C++ 프로그래밍

코딩 테스트

- 정리 및 요약

목차

원문 범위와 출처 읽는 법 5

01 개발환경·변수·기본 문법 9

02 함수·포인터·참조·메모리 15

03 클래스·생성·복사·const 21

04 상속·다형성 27

05 연산자 오버로딩·N차원 배열 프로젝트 33

06 표준 입출력·파일·엑셀 프로젝트 39

07 템플릿·가변 인자·메타프로그래밍 45

08 STL 컨테이너·알고리즘·문자열 51

09 예외 처리 58

10 우측값·이동·완벽한 전달 63

11 스마트 포인터·함수 객체 69

12 쓰레드·동기화·atomic·비동기·ThreadPool 75

13 유니폼 초기화·constexpr·decltype 81

14 type_traits·정규표현식·난수·시간·파일시스템·유틸리티 86

15 Visual Studio 디버깅·Make 92

16 리스트, 스택, 큐 98

17 트리, 힙, 그래프 103

18 해시 테이블과 블룸 필터 108

19 분할 정복 113

20 그리디 알고리즘 118

21 그래프 알고리즘 I 123

22 그래프 알고리즘 II 130

23 동적 계획법 I 136

24 동적 계획법 II 141

25 실습 문제 풀이 146

원문 기반 복습 질문과 정답 153

01 개발환경·변수·기본 문법 153

02 함수·포인터·참조·메모리 154

03 클래스·생성·복사·const 155

04 상속·다형성 156

05 연산자 오버로딩·N차원 배열 프로젝트 157

06 표준 입출력·파일·엑셀 프로젝트 158

07 템플릿·가변 인자·메타프로그래밍 159

08 STL 컨테이너·알고리즘·문자열 160

09 예외 처리 161

10 우측값·이동·완벽한 전달 162

11 스마트 포인터·함수 객체 163

12 쓰레드·동기화·atomic·비동기·ThreadPool 164

13 유니폼 초기화·constexpr·decltype 165

14 type_traits·정규표현식·난수·시간·파일시스템·유틸리티 166

15 Visual Studio 디버깅·Make 167

16 리스트, 스택, 큐 168

17 트리, 힙, 그래프 169

18 해시 테이블과 블룸 필터 170

19 분할 정복 171

20 그리디 알고리즘 172

21 그래프 알고리즘 I 173

22 그래프 알고리즘 II 174

23 동적 계획법 I 175

24 동적 계획법 II 176

25 실습 문제 풀이 177

원문 범위와 출처 읽는 법

첨부한 C++ 책 2개와 강의 자료 13개만을 사용했습니다. 주제가 겹치는 내용은 25개 통합 단원으로 묶었습니다. 통합 단원 번호는 편집 번호이며 원문의 장 번호와 다릅니다.

각 항목 끝에는 [문장 번호]와 원본 파일 코드·절·쪽수를 표시했습니다. B는 책의 인쇄 쪽수이며 원본 PDF 면 번호에서 5를 뺀 값입니다. K는 PDF 면 번호, L로 시작하는 코드는 강의 슬라이드 번호입니다. 별도 근거 검증표의 같은 번호로 문장을 찾을 수 있습니다.

다루는 내용은 원문 도입부와 본문의 학습 범위입니다. 원문에 요약 절이 있는 경우 이를 우선했으며, 본문을 재압축한 단원은 편집 요약으로 표시했습니다. 코드와 수식은 원문 발췌이며, 책에 없는 예시나 보충 설명을 덧붙이지 않았습니다.

K는 목차부터 시작하는 스캔 자료입니다. 책의 표지·서지 정보는 원문에서 확인 불가이며, 마지막 PDF 530면은 찾아보기 첫 면입니다. 찾아보기의 나머지 면은 첨부에서 확인 불가입니다. 부록 A.2.2의 제목·도입 일부도 수록 면 사이에서 확인 불가이므로 확인되는 풀이만 정리했습니다. 본문·부록 중 수록된 면만 사용했습니다. (K PDF p.444-446, 530)

별도 「확인 및 시험」은 단원마다 한 페이지입니다. 출처는 파일 코드와 절·쪽수로 표시하며, 같은 문장 번호의 근거 검증표에서도 확인할 수 있습니다. 자세한 복습 질문과 정답 근거는 이 노트 뒤에 있습니다.

원본 파일 코드

코드	첨부 파일명	총 PDF 면
B	cpp.pdf	983
K	Cpp코테.pdf	530
L00	Lec00_CPP_Lecture_Introduction.pdf	24
L01	Lec01_CPP_Introduction_and_Variable.pdf	53
L02D	Lec02_Additional_Debugging_in_VS.pdf	13
L02	Lec02_CPP_Basic_Syntax.pdf	41
L03	Lec03_Function.pdf	67
L04	Lec04_Pointer_Reference.pdf	60
L05A	Lec05_1_OOP_Class_Part1.pdf	63
L05B	Lec05_2_OOP_Class_Part2.pdf	40
L06	Lec06_Inheritance.pdf	53
L07	Lec07_Polymorphism.pdf	47
L08	Lec08_Operator_Overloading.pdf	48
L09	Lec09_Generic_Programming_Template.pdf	33
L99	Lec99_Summary.pdf	10

통합 단원과 원문 범위

단원	제목	원본 PDF 면 범위
01	개발환경·변수·기본 문법	B PDF 6-34 | L00 PDF 1-24 | L01 PDF 1-53 | L02 PDF 1-41
02	함수·포인터·참조·메모리	B PDF 35-61 | L03 PDF 1-67 | L04 PDF 1-60
03	클래스·생성·복사·const	B PDF 62-192 | L05A PDF 1-63 | L05B PDF 1-40
04	상속·다형성	B PDF 275-344 | L06 PDF 1-53 | L07 PDF 1-47
05	연산자 오버로딩·N차원 배열 프로젝트	B PDF 193-274 | L08 PDF 1-48
06	표준 입출력·파일·엑셀 프로젝트	B PDF 345-414
07	템플릿·가변 인자·메타프로그래밍	B PDF 415-498 | L09 PDF 1-33
08	STL 컨테이너·알고리즘·문자열	B PDF 499-617 | L99 PDF 8
09	예외 처리	B PDF 618-634
10	우측값·이동·완벽한 전달	B PDF 635-675
11	스마트 포인터·함수 객체	B PDF 676-723
12	쓰레드·동기화·atomic·비동기·ThreadPool	B PDF 724-830
13	유니폼 초기화·constexpr·decltype	B PDF 831-870
14	type_traits·정규표현식·난수·시간·파일시스템·유틸리티	B PDF 871-960
15	Visual Studio 디버깅·Make	L02D PDF 1-13 | B PDF 961-983
16	리스트, 스택, 큐	K PDF 10-61
17	트리, 힙, 그래프	K PDF 62-105
18	해시 테이블과 블룸 필터	K PDF 106-145

단원	제목	원본 PDF 면 범위
19	분할 정복	K PDF 146-193
20	그리디 알고리즘	K PDF 194-231
21	그래프 알고리즘 I	K PDF 232-269
22	그래프 알고리즘 II	K PDF 270-313
23	동적 계획법 I	K PDF 314-373
24	동적 계획법 II	K PDF 374-425
25	실습 문제 풀이	K PDF 426-529

01 개발환경·변수·기본 문법

B PDF 6-34 | L00 PDF 1-24 | L01 PDF 1-53 | L02 PDF 1-41

다루는 내용

강의 초반부는 메모리 레이아웃과 명령어에 따른 메모리의 변화를 이해하는 것을 주요 목표로 한다. [U01-001] (L00 Course introduction, 슬라이드 5)

배열, **명령문**과 연산자, 조건문과 반복문을 다룬다. [U01-002] (L02 Today, 슬라이드 2)

핵심 개념

개발환경과 빌드

컴파일러는 소스 코드를 오브젝트 코드로 변환하는 도구이다. [U01-003] (L01 Terminologies, 슬라이드 5)

링커는 오브젝트 코드를 실행파일(exe)로 변환하는 도구이다. [U01-004] (L01 Terminologies, 슬라이드 5)

IDE (Integrated Development Environment)는 텍스트 에디터, **컴파일러**, **링커**, 디버거를 묶은 개발환경이다. [U01-005] (L01 Terminologies, 슬라이드 5)

Compiler Warnings는 코드에 잠재적인 문제가 있을 것으로 예상될 때 발생한다. [U01-006] (L01 Compiler Warnings, 슬라이드 15)

프로그램 구성과 입출력

키워드는 언어 자체에서 예약된 단어이며, 식별자는 **변수**·함수·타입 등 개발자가 지정하는 부분이다. [U01-007] (L01 C++ Keyword, Identifier, Operator, 슬라이드 20)

전처리기는 **컴파일** 이전에 처리되며 "#"으로 시작한다. [U01-008] (L01 C++ preprocessor, 슬라이드 21)

main() 함수는 프로그램의 진입점이다. [U01-009] (L01 main() function, 슬라이드 25)

cout과 <<는 표준 출력 스트림과 삽입 연산자이며, cin과 >>는 표준 입력 스트림과 추출 연산자이다. [U01-010] (L01 Basic I/O, 슬라이드 29)

주석은 프로그래머에게 정보를 제공하기 위한 것이며 전처리 단계에서 무시(삭제)된다. [U01-011] (L01 C++ Comments, 슬라이드 24)

이름공간(namespace)

이름공간(namespace)은 정의된 **객체**가 어디 소속인지 지정하며, 소속된 **이름공간**이 다르면 같은 이름도 다른 것으로 취급한다. [U01-012] (B 2장·이름공간(namespace), 인쇄 p.16)

std는 C++ 표준 라이브러리의 모든 함수와 **객체** 등이 정의된 **이름공간**이다. [U01-013] (B 2장·이름공간(namespace), 인쇄 p.16)

이름 없는 **이름공간**에 정의된 것들은 해당 파일 안에서만 접근할 수 있다. [U01-014] (B 2장·이름 없는 이름공간, 인쇄 p.20)

책은 using namespace std; 대신 std::를 직접 붙여 소속을 명시하는 방식을 권장한다. [U01-015] (B 2장·이름공간(namespace), 인쇄 p.19)

변수·초기화·상수

메모리는 읽고 쓸 수 있는 바이트 공간의 집합이며, 강의에서는 메인 메모리(RAM)를 지칭한다. [U01-016] (L01 Memory, 슬라이드 36)

변수는 메모리의 주소에 붙이는 이름이며, **변수**를 만들면 타입에 따라 메모리 공간이 확보된다. [U01-017] (L01 Variable, 슬라이드 41)

변수를 정의하면서 초기값을 설정하는 것을 **초기화**라고 한다. [U01-018] (L01 Definition and Initialization, 슬라이드 42)

sizeof 연산자는 타입 또는 **변수**의 바이트 단위 크기를 리턴한다. [U01-019] (L01 Using the sizeof operator, 슬라이드 47)

상수는 **변수**와 유사하지만 **초기화** 이후 변할 수 없는 값이다. [U01-020] (L01 Constant, 슬라이드 49)

배열

배열은 동일한 타입인 요소들의 집합인 복합(compound) 데이터 타입이다. [U01-021] (L02 Array, 슬라이드 5)

배열은 길이가 고정되어 있고 연속된 메모리 주소에 저장된다. [U01-022] (L02 Arrays, 슬라이드 6)

배열 인덱스는 0부터 시작하여 마지막 인덱스는 size-1이며, out of bound를 체크하지 않는다. [U01-023] (L02 Arrays, 슬라이드 6)

표현식·명령문·연산자

표현식은 코드의 가장 작은 구성요소이다. [U01-024] (L02 Expressions, 슬라이드 14)

명령문은 명령을 수행하는 코드 단위이며 세미콜론(";")으로 끝난다. [U01-025] (L02 Statements, 슬라이드 15)

대입 연산자는 오른쪽의 값을 계산하여 왼쪽에 대입하며, 왼쪽은 대입이 가능해야 한다. [U01-026] (L02 Operations (1), 슬라이드 16)

산술 연산자는 +, -, *, /, %(mod)이며 증감 연산자는 Prefix와 Postfix로 구분한다. [U01-027] (L02 Operations (2), 슬라이드 17)

논리 연산자 !, &&, ||의 Short-circuit evaluation은 결과 파악이 가능하면 나머지 연산을 하지 않는다. [U01-028] (L02 Operations (3), 슬라이드 18)

조건문과 블록

if문은 **표현식**이 참일 경우에만 실행하는 **명령문**이다. [U01-029] (L02 Conditional statement, 슬라이드 25)

블록은 "{"로 시작하여 "}"로 끝나며, 블록 내에서 선언된 지역**변수**는 그 블록 내에서만 접근할 수 있다. [U01-030] (L02 Conditional statement, 슬라이드 26)

if-else문은 **표현식**의 참·거짓 여부에 따라 분기하고, switch문은 switch, case, default를 사용한 분기문이다. [U01-031] (L02 Conditional statement, 슬라이드 27,29)

?: 연산자는 조건 **표현식**이 참이면 expr1의 값을, 거짓이면 expr2의 값을 리턴하는 삼항 연산자이다. [U01-032] (L02 Conditional statement, 슬라이드 30)

반복문

강의는 반복문으로 for문, while문, do-while문을 구분한다. [U01-033] (L02 (Summary) Controlling flow, 슬라이드 41)

C++에서는 **변수**를 사용하기 직전 어느 위치에서든지 선언할 수 있고, 책의 for문 예제는 **초기화** 부분에서 int i를 선언한다. [U01-034] (B 2장·C++ 와 C 언어의 공통 문법 구조, 인쇄 p.25)

continue는 이후 문장을 실행하지 않고 다음 iteration으로 곧바로 넘어간다. [U01-035] (L02 Looping Statement, 슬라이드 39)

break는 이후 문장을 실행하지 않고 루프 밖으로 바로 빠져나간다. [U01-036] (L02 Looping Statement, 슬라이드 39)

동작 방식·절차

빌드 과정

1. 각 .cpp 소스 코드를 **컴파일러**가 .obj 오브젝트 코드로 변환한다. [U01-037] (L01 C++ Build Process, 슬라이드 6)

2. 링커가 오브젝트 코드와 라이브러리를 연결하여 실행 프로그램(.exe)을 만든다. [U01-038] (L01 C++ Build Process, 슬라이드 6)

배열의 정의·초기화·사용

1. 고정 크기의 **배열**을 정의한다. [U01-039] (L02 (Summary) Arrays, 슬라이드 11)

2. 리스트 **초기화** "{}"를 사용하여 **초기화**한다. [U01-040] (L02 (Summary) Arrays, 슬라이드 11)

3. 인덱스로 개별 요소에 접근하여 사용한다. [U01-041] (L02 (Summary) Arrays, 슬라이드 11)

주요 문법·표·수식

첫 C++ 프로그램 - 원문 발췌 (원문 발췌)

```
#include <iostream>
int main() {
  std::cout << "Hello, World!!" << std::endl;
  return 0;
}
```

[U01-042] (B 2장·첫 C++ 프로그램 분석하기, 인쇄 p.15)

iostream 헤더 파일은 C++의 표준 입출력에 필요한 것들을 포함한다. [U01-043] (B 2장·첫 C++ 프로그램 분석하기, 인쇄 p.15)

byte와 bit: 1 byte = 8 bit = 2^8 = 256개의 숫자 표현 가능; 양수 범위 0~255 [U01-044] (L01 Bit, Byte, Hex, 슬라이드 35)

오류가 나타나는 단계와 성격

구분	원문 설명
Compiler Errors [U01-045] (L01 Compiler Errors, 슬라이드 14)	프로그래밍 언어의 규칙을 위반하는 경우이다. [U01-046] (L01 Compiler Errors, 슬라이드 14)
Linker Errors [U01-047] (L01 Linker Errors, 슬라이드 16)	obj 파일의 링크 과정에서 오류가 있는 경우이다. [U01-048] (L01 Linker Errors, 슬라이드 16)
Runtime Errors [U01-049] (L01 Runtime Errors, 슬라이드 16)	프로그램의 실행 도중 발생하는 오류이다. [U01-050] (L01 Runtime Errors, 슬라이드 16)
Logical Errors [U01-051] (L01 Logical Errors, 슬라이드 17)	프로그램의 동작에 관한 논리적 오류이다. [U01-052] (L01 Logical Errors, 슬라이드 17)

핵심 용어 표

용어	원문에 나온 정의·설명
컴파일러	**컴파일러**는 소스 코드를 오브젝트 코드로 변환하는 도구이다. [U01-003] (L01 Terminologies, 슬라이드 5)
링커	**링커**는 오브젝트 코드를 실행파일(exe)로 변환하는 도구이다. [U01-004] (L01 Terminologies, 슬라이드 5)
이름공간(namespace)	**이름공간**(namespace)은 정의된 **객체**가 어디 소속인지 지정하며, 소속된 **이름공간**이 다르면 같은 이름도 다른 것으로 취급한다. [U01-012] (B 2장·이름공간 (namespace), 인쇄 p.16)
변수	**변수**는 메모리의 주소에 붙이는 이름이며, **변수**를 만들면 타입에 따라 메모리 공간이 확보된다. [U01-017] (L01 Variable, 슬라이드 41)
초기화	**변수**를 정의하면서 초기값을 설정하는 것을 **초기화**라고 한다. [U01-018] (L01 Definition and Initialization, 슬라이드 42)
상수	**상수**는 **변수**와 유사하지만 **초기화** 이후 변할 수 없는 값이다. [U01-020] (L01 Constant, 슬라이드 49)
배열	**배열**은 동일한 타입인 요소들의 집합인 복합(compound) 데이터 타입이다. [U01-021] (L02 Array, 슬라이드 5)
표현식	**표현식**은 코드의 가장 작은 구성요소이다. [U01-024] (L02 Expressions, 슬라이드 14)
명령문	**명령문**은 명령을 수행하는 코드 단위이며 세미콜론(";")으로 끝난다. [U01-025] (L02 Statements, 슬라이드 15)

단원 요약

요약 기준: 원문 Summary: L02 슬라이드 11·21·41을 중심으로 정리

배열은 동일한 타입 요소들의 집합이며, 고정 크기로 정의하고 인덱스를 통해 사용한다. [U01-053] (L02 (Summary) Arrays, 슬라이드 11)

표현식은 가장 작은 단위이며 **명령문**은 세미콜론으로 끝을 표기하는 코드의 구성 단위이다. [U01-054] (L02 (Summary) Statements and Operators, 슬라이드 21)

제어 흐름에는 조건문과 반복문이 있으며, continue와 break를 사용한다. [U01-055] (L02 (Summary) Controlling flow, 슬라이드 41)

02 함수·포인터·참조·메모리

B PDF 35-61 | L03 PDF 1-67 | L04 PDF 1-60

다루는 내용

함수의 정의·프로토타입·매개**변수**·return·기본 인수·오버로딩·호출 동작·inline·재귀를 다룬다. [U02-001] (L03 Today, 슬라이드 3)

포인터의 정의·주소 접근·**역참조**·동적 할당과 **참조자** 및 전달 방식의 차이를 다룬다. [U02-002] (L04 (Summary) Pointers·References, 슬라이드 4,41)

핵심 개념

함수 정의·프로토타입·반환

함수 정의에 필요한 요소는 이름, 매개**변수** 리스트, 리턴 타입, 본문(body)이다. [U02-003] (L03 Function Definition, 슬라이드 10)

함수 프로토타입은 함수를 사용하기 전에 입력 매개**변수**와 반환형을 미리 알려주는 **명령문**이다. [U02-004] (L03 Function Prototypes, 슬라이드 20)

return문은 함수의 결과값을 전달하며 함수를 즉각적으로 종료한다. [U02-005] (L03 Function Return, 슬라이드 28)

반환값이 없는 경우 반환형을 void로 명시한다. [U02-006] (L03 Function Definition, 슬라이드 13)

인수·전달·기본 인수·오버로딩

호출에서 전달하는 값은 인수(argument), 함수 정의의 전달 항목은 인자 또는 매개**변수**(parameter)이며 개수·순서·타입이 일치해야 한다. [U02-007] (L03 Function Parameters, 슬라이드 23)

pass-by-value에서는 데이터 값이 복사되어 전달되므로 함수 안의 사본을 바꾸어도 원본은 바뀌지 않는다. [U02-008] (L03 Pass-by-value, 슬라이드 25)

기본 인수는 인수가 주어지지 않았을 때 사용할 기본값으로, 프로토타입 또는 정의부 중 한쪽에만 선언하며 여러 개면 오른쪽부터 선언한다. [U02-009] (L03 Default Argument Values, 슬라이드 29)

함수 오버로딩은 서로 다른 매개**변수** 리스트를 갖는 동일한 이름의 함수를 정의하는 것이다. [U02-010] (L03 Function Overloading, 슬라이드 32)

반환 타입만 다른 **함수 오버로딩**은 불가능하다. [U02-011] (L03 Function Overloading, 슬라이드 34)

호출·범위·inline·재귀

함수의 매개**변수**와 지역**변수**는 함수 실행 중에만 존재한다. [U02-012] (L03 Local/Global Scope, 슬라이드 36)

static 지역**변수**는 지역 밖에 정의한 것처럼 활용할 수 있지만 scope 밖에서 접근할 수는 없다. [U02-013] (L03 Local/Global Scope, 슬라이드 37)

강의의 inline 설명은 **컴파일** 단계에서 함수 호출을 함수 내 **명령문**으로 대체하며 바이너리 파일 용량이 커질 수 있다고 정리한다. [U02-014] (L03 inline function, 슬라이드 61)

재귀함수는 스스로를 호출하는 함수이며, 재귀 호출을 끝내는 base case가 반드시 실행되어야 한다. [U02-015] (L03 Recursive Function, 슬라이드 63)

포인터·주소·역참조·배열

포인터는 주소값을 저장하는 **변수**이다. [U02-016] (L04 Accessing Pointer Address, 슬라이드 10)

포인터는 타입 뒤에 *를 붙여 정의하며, **nullptr**은 아무것도 가리키지 않는 **상태**를 의미한다. [U02-017] (L04 Definition of Pointers, 슬라이드 9)

주소 연산자 &는 적용한 피연산자의 주소값을 반환한다. [U02-018] (L04 Accessing Pointer Address, 슬라이드 10)

역참조는 * 연산자로 **포인터**의 주소에 저장된 데이터에 접근하는 것이다. [U02-019] (L04 Dereferencing a Pointer, 슬라이드 15)

배열 이름은 첫 요소의 주소를 가리키며, **배열**과 **포인터**는 정의 후 주소 변경 가능 여부와 sizeof 반환값에 차이가 있다. [U02-020] (L04 Arrays and Pointers, 슬라이드 26)

참조자와 수명

참조자는 **변수**의 별명이며 선언과 동시에 **초기화**되어야 하고, 한 번 **초기화**되면 다른 **변수**의 **참조자**가 될 수 없다. [U02-021] (L04 Reference, 슬라이드 42)

상수 참조자로 선언하면 리터럴도 참조할 수 있다. [U02-022] (B 3장·상수에 대한 참조자, 인쇄 p.38)

레퍼런스의 **배열**은 불가능하지만 **배열**의 레퍼런스는 가능하며, **배열** 크기에 맞는 별명이 되어야 한다. [U02-023] (B 3장·레퍼런스의 배열과 배열의 레퍼런스, 인쇄 p.38-40)

댕글링 레퍼런스(Dangling reference)는 레퍼런스는 있지만 원래 참조하던 것이 사라진 레퍼런스이다. [U02-024] (B 3장·지역변수의 레퍼런스를 리턴?, 인쇄 p.42)

상수 레퍼런스로 함수의 리턴값을 받으면 그 값의 수명은 레퍼런스가 사라질 때까지 연장된다. [U02-025] (B 3장·참조자가 아닌 값을 리턴하는 함수를 참조자로 받기, 인쇄 p.45)

동적 메모리와 반환 시 주의

동적 메모리 할당은 런타임에 힙 메모리를 할당하는 것이다. [U02-026] (L04 Dynamic Memory Allocation, 슬라이드 18)

힙 메모리는 스스로 해제되지 않으므로 사용 후 해제하지 않으면 메모리 누수가 발생한다. [U02-027] (L04 Dynamic Memory Allocation, 슬라이드 18)

new와 delete가 짝을 이루고, new []와 delete []가 짝을 이룬다. [U02-028] (B 4장·new 로 배열 할당하기, 인쇄 p.51)

함수 내부에서 동적으로 할당한 메모리의 주소는 반환할 수 있지만 지역**변수**에 대한 **포인터**를 반환해서는 안 된다. [U02-029] (L04 Returning a Pointer, 슬라이드 36-37)

const 포인터와 함수 전달

pointers to const는 데이터가 const이고, **포인터**는 다른 데이터를 가리킬 수 있다. [U02-030] (L04 Const and Pointers, 슬라이드 29)

const pointers는 **포인터**가 const이고 데이터는 변할 수 있으며, const pointers to const는 둘 다 const이다. [U02-031] (L04 Const and Pointers, 슬라이드 30-31)

포인터를 전달하면 주소값을 통해 함수 범위 밖의 메모리에 접근할 수 있다. [U02-032] (L03 Pass-by-address, 슬라이드 55)

const **참조자**를 사용하는 전달은 원본을 수정하지 않고 복사 오버헤드가 클 때, 매개**변수**가 null이 되지 않음이 보장될 때 사용한다. [U02-033] (L04 Pointers vs References, 슬라이드 56)

동물 관리 예제와 객체지향 도입

동물 관리 예제는 Animal의 이름·나이·체력·배부른 정도·깨끗한 정도를 저장하고, play와 show_stat으로 **상태** 변경·출력을 수행한다. [U02-034] (B 4장·돌아온 마이펫, 인쇄 p.51)

책은 new Animal 뒤에 **초기화** 함수를 따로 호출해야 하는 문제와, Bird·Fish에 공통 부분을 반복 작성하는 문제를 **객체**지향 도입에 연결한다. [U02-035] (B 4장·돌아온 마이펫, 인쇄 p.55-56)

동작 방식·절차

강의의 함수 호출 메모리 설명 순서

1. 함수를 호출하면 호출 영역을 구분하는 구분선이 생긴다. [U02-036] (L03 (Summary) Functions, 슬라이드 64)

2. 그 영역에는 매개**변수**를 포함한 지역**변수** 공간이 들어간다. [U02-037] (L03 (Summary) Functions, 슬라이드 64)

3. 함수가 끝나면 그 공간은 자동으로 사라진다. [U02-038] (L03 (Summary) Functions, 슬라이드 64)

new·사용·delete의 원문 순서

1. new로 공간을 할당하고 반환된 주소값을 **포인터**에 저장한다. [U02-039] (B 4장·new 와 delete 의 사용, 인쇄 p.47)

2. 포인터를 **역참조**하여 값을 저장하고 출력한다. [U02-040] (B 4장·new 와 delete 의 사용, 인쇄 p.47)

3. 마지막에 delete로 할당된 공간을 해제한다. [U02-041] (B 4장·new 와 delete 의 사용, 인쇄 p.47)

주요 문법·표·수식

참조자 매개변수 - 원문 발췌 (원문 발췌)

```
int change_val(int &p) {
  p = 3;

  return 0;
}
```

[U02-042] (B 3장·참조자를 함수 인자로 받기, 인쇄 p.35)

함수 안에서 p = 3;을 수행하는 것은 main 함수의 number에 number = 3;을 수행하는 것과 같다. [U02-043] (B 3장·참조자를 함수 인자로 받기, 인쇄 p.35)

new와 delete - 원문 발췌 (원문 발췌)

```
#include <iostream>
int main() {
  int* p = new int;
  *p = 10;
  std::cout << *p << std::endl;
  delete p;
  return 0;
}
```

[U02-044] (B 4장·new 와 delete 의 사용, 인쇄 p.46)

new는 메모리를 할당하고 delete는 메모리를 해제한다. [U02-045] (B 4장·C++ 의 세계로, 인쇄 p.46)

강의의 전달 방식별 사용 조건

방식	원문 사용 조건
Pass-by-value [U02-046] (L04 Pointers vs References, 슬라이드 51)	원본을 수정하지 않을 때, 크기가 작은 기본 자료형을 전달한다. [U02-047] (L04 Pointers vs References, 슬라이드 51)
Pass-by-address [U02-048] (L04 Pointers vs References, 슬라이드 52)	원본 수정이 필요하고, **포인터**가 **nullptr**이어도 상관없을 때 사용한다. [U02-049] (L04 Pointers vs References, 슬라이드 52)
Pass-by-reference [U02-050] (L04 Pointers vs References, 슬라이드 55)	원본을 수정하고, 매개**변수**가 null이 되지 않음이 보장될 때 사용한다. [U02-051] (L04 Pointers vs References, 슬라이드 55)

핵심 용어 표

용어	원문에 나온 정의·설명
함수 프로토타입	**함수 프로토타입**은 함수를 사용하기 전에 입력 매개**변수**와 반환형을 미리 알려주는 **명령문**이다. [U02-004] (L03 Function Prototypes, 슬라이드 20)
pass-by-value	**pass-by-value**에서는 데이터 값이 복사되어 전달되므로 함수 안의 사본을 바꾸어도 원본은 바뀌지 않는다. [U02-008] (L03 Pass-by-value, 슬라이드 25)
함수 오버로딩	**함수 오버로딩**은 서로 다른 매개**변수** 리스트를 갖는 동일한 이름의 함수를 정의하는 것이다. [U02-010] (L03 Function Overloading, 슬라이드 32)
포인터	**포인터**는 주소값을 저장하는 **변수**이다. [U02-016] (L04 Accessing Pointer Address, 슬라이드 10)
역참조	**역참조**는 * 연산자로 **포인터**의 주소에 저장된 데이터에 접근하는 것이다. [U02-019] (L04 Dereferencing a Pointer, 슬라이드 15)
참조자	**참조자**는 **변수**의 별명이며 선언과 동시에 **초기화**되어야 하고, 한 번 **초기화**되면 다른 **변수**의 **참조자**가 될 수 없다. [U02-021] (L04 Reference, 슬라이드 42)
동적 메모리 할당	**동적 메모리 할당**은 런타임에 힙 메모리를 할당하는 것이다. [U02-026] (L04 Dynamic Memory Allocation, 슬라이드 18)
댕글링 레퍼런스	**댕글링 레퍼런스**(Dangling reference)는 레퍼런스는 있지만 원래 참조하던 것이 사라진 레퍼런스이다. [U02-024] (B 3장·지역변수의 레퍼런스를 리턴?, 인쇄 p.42)
재귀함수	**재귀함수**는 스스로를 호출하는 함수이며, 재귀 호출을 끝내는 base case가 반드시 실행되어야 한다. [U02-015] (L03 Recursive Function, 슬라이드 63)

단원 요약

요약 기준: 원문 Summary: L03 슬라이드 64 및 L04 슬라이드 39·57

함수는 매개**변수**를 복사하여 전달하고, **포인터**와 **참조자**를 사용하여 다른 메모리 영역의 **변수**를 변경할 수 있다. [U02-052] (L03 (Summary) Functions, 슬라이드 64)

포인터는 *, 주소 접근은 &, **역참조**는 *를 사용하며 동적 메모리는 new와 delete로 다룬다. [U02-053] (L04 (Summary) Pointers, 슬라이드 39)

참조자는 **변수**의 별명이며, 전달 방식에는 값·**포인터**·**참조자** 및 const를 사용하는 구분이 있다. [U02-054] (L04 (Summary) References, 슬라이드 57)

03 클래스·생성·복사·const

B PDF 62-192 | L05A PDF 1-63 | L05B PDF 1-40

다루는 내용

절차적 프로그래밍과 **객체**지향 프로그래밍, **클래스·객체·생성자·소멸자·생성자** 오버로딩을 다룬다. [U03-001] (L05A Lecture Plan, 슬라이드 2,4)

문자열 **클래스** 프로젝트는 배운 **클래스**의 중요한 내용을 복습하고 점검하기 위한 것이다. [U03-002] (B 5장·내가 만들어보는 문자열 클래스, 인쇄 p.138)

핵심 개념

객체지향·캡슐화·정보은닉

절차적 프로그래밍은 데이터와 작업이 분리되고, **객체**지향 프로그래밍은 데이터와 기능을 **캡슐화**한다. [U03-003] (L05A (Summary) Object Oriented Programming, 슬라이드 10)

캡슐화는 **클래스**가 데이터와 데이터를 기반으로 하는 기능을 모두 포함하는 것이다. [U03-004] (L05A Object Oriented Programming, 슬라이드 8)

정보은닉에서는 내부 구현을 감추고 외부로 노출된 인터페이스를 사용하게 한다. [U03-005] (L05A Object Oriented Programming, 슬라이드 8)

강의는 **객체**지향 프로그래밍의 한계로 성능 손실 가능성과 설계에 필요한 추가 수고를 제시한다. [U03-006] (L05A (Summary) Object Oriented Programming, 슬라이드 10)

클래스·객체·멤버 접근

클래스는 **객체**가 생성되기 위한 틀이며 **객체**의 데이터와 기능을 정의하는 사용자 정의 자료형이다. [U03-007] (L05A Classes and Objects, 슬라이드 14)

객체는 **클래스**로부터 생성된 실체이다. [U03-008] (L05A Classes and Objects, 슬라이드 15)

public은 **객체**에서 접근할 수 있고 private은 **클래스** 멤버들만 접근할 수 있다. [U03-009] (L05A Class Member Access Modifier, 슬라이드 24)

멤버함수는 멤버**변수**에 직접 접근할 수 있으며, **클래스** 안 또는 ClassName::MethodName 형식으로 **클래스** 밖에 구현할 수 있다. [U03-010] (L05A Implementing Member Methods, 슬라이드 29)

명세와 구현을 분리할 때 **클래스** 선언은 .h에, 구현은 .cpp에 작성하고 Include guard로 헤더의 중복 포함을 방지한다. [U03-011] (L05A Implementing Member Methods, 슬라이드 29,32)

class의 기본 접근권한은 private이고 struct의 기본 접근권한은 public이다. [U03-012] (L05A Struct vs Class, 슬라이드 36)

생성자·소멸자·기본 생성자

생성자는 **객체**가 생성될 때 자동 호출되는 특수한 멤버함수로, **클래스**와 이름이 같고 반환형이 없다. [U03-013] (L05A Constructors, 슬라이드 41)

소멸자는 **객체**가 소멸할 때 자동 호출되며 **클래스** 이름 앞에 ~를 붙이고 반환형·매개**변수**가 없다. [U03-014] (L05A Destructor, 슬라이드 44)

생성자는 매개**변수**가 다른 고유한 형태로 오버로딩할 수 있으나 **소멸자**는 오버로딩할 수 없다. [U03-015] (L05A Overloading Constructor·Destructor, 슬라이드 44,53)

기본 **생성자**는 인자가 없는 **생성자**이며, **생성자**를 직접 구현하지 않으면 **컴파일러**가 만들어 준다. [U03-016] (L05A Default Constructor, 슬라이드 47)

인자가 있는 **생성자**만 구현한 경우 기본 **생성자**는 자동 생성되지 않는다. [U03-017] (L05A Declaring a Class, 슬라이드 49)

초기화 리스트·위임·기본 매개변수

생성자 초기화 리스트는 **생성자** 본체에서 값을 대입하는 대신 생성과 동시에 값을 지정한다. [U03-018] (L05A Constructor Initialization List, 슬라이드 56)

클래스 안의 레퍼런스 **변수**나 **상수**는 **생성자**의 **초기화** 리스트로 **초기화**해야 한다. [U03-019] (B 5장 ·const, const, const!·생성자의 초기화 리스트, 인쇄 p.113)

생성자 위임은 다른 **생성자**를 멤버 **초기화** 리스트 위치에서 호출하여 코드 반복을 줄이는 방법이다. [U03-020] (L05A Delegating Constructors, 슬라이드 57)

생성자도 함수이므로 기본 매개**변수**를 사용할 수 있다. [U03-021] (L05A Default Constructor Parameters, 슬라이드 60)

복사 생성·얕은 복사·깊은 복사

복사 생성자는 기존 **객체**를 기반으로 새로운 **객체**가 생성될 때 어떻게 복사할지를 정의한다. [U03-022] (L05B Copy Constructor, 슬라이드 6)

강의는 값으로 매개**변수** 전달, 값으로 결과 반환, 기존 **객체**를 기반으로 새 **객체** 생성의 경우에 복사 생성이 일어난다고 설명한다. [U03-023] (L05B Copy Constructor, 슬라이드 6)

직접 구현하지 않으면 기본 **복사 생성자**가 자동 생성되며 멤버**변수**의 값을 복사하여 대입한다. [U03-024] (L05B Copy Constructor, 슬라이드 10)

얕은 복사는 **포인터**가 가리키는 데이터가 아니라 **포인터** 주소값 자체를 복사한다. [U03-025] (L05B Shallow vs Deep Copy, 슬라이드 14)

깊은 복사는 새로운 힙 공간을 할당한 뒤 동일한 데이터를 복사하여 복사 생성하는 방식이다. [U03-026] (L05B Shallow vs Deep Copy, 슬라이드 20)

강의의 얕은 복사 예제는 같은 메모리를 두 번 해제하려 하면서 오류가 발생한다. [U03-027] (L05B Shallow vs Deep Copy, 슬라이드 18)

this·const·static·friend

this 포인터는 멤버함수를 호출한 **객체**의 주소값이다. [U03-028] (L05B this Pointer, 슬라이드 26)

값이 바뀌지 않음을 보장하는 멤버함수는 선언 뒤에 const를 붙여 const **객체**에서 호출할 수 있게 한다. [U03-029] (L05B const Object, 슬라이드 31)

static **클래스** 멤버**변수**와 멤버함수는 **객체**가 아니라 **클래스**에 속한다. [U03-030] (L05B Static Class Members, 슬라이드 32)

friend는 특정 함수나 **클래스**에 private 멤버 접근을 허용하며, 관계는 비대칭이고 전이되지 않는다. [U03-031] (L05B Friends of a Class, 슬라이드 37)

MyString 프로젝트의 데이터와 연산

MyString은 문자열 데이터를 가리키는 **포인터**와 문자열 길이를 보관하여 문자열 크기 변경과 반복적인 길이 계산을 다룬다. [U03-032] (B 5장·내가 만들어보는 문자열 클래스·문자열 클래스를 만들자, 인쇄 p.139)

MyString은 string_length를 별도로 보관하므로 실제 문자에 해당하는 내용만 string_content에 저장한다. [U03-033] (B 5장·내가 만들어보는 문자열 클래스, 인쇄 p.140)

memory_capacity는 현재 할당된 메모리 공간의 크기이며, reserve는 미리 공간을 예약하고 capacity는 할당된 크기를 알려준다. [U03-034] (B 5장·내가 만들어보는 문자열 클래스·assign 함수, 인쇄 p.145-146)

assign은 기존 문자열을 지정한 문자열로 바꾸며, insert는 지정 위치 앞에 문자열을 삽입하고 erase는 지정 위치에서 문자를 지운다. [U03-035] (B 5장·내가 만들어보는 문자열 클래스·assign·insert·erase, 인쇄 p.143,151,164)

find는 find_from에서 시작하여 처음 발견한 문자열의 위치를 반환하며, 포함되지 않으면 -1을 반환한다. [U03-036] (B 5장·내가 만들어보는 문자열 클래스·find 함수, 인쇄 p.165)

compare의 크기 비교는 문자열을 사전식으로 **배열**했을 때 어느 문자열이 뒤에 오는지를 판단하는 것이다. [U03-037] (B 5장·내가 만들어보는 문자열 클래스·compare, 인쇄 p.171)

explicit과 mutable

생성자를 **explicit**으로 선언하면 그 **생성자**를 이용한 암시적 변환을 막을 수 있다. [U03-038] (B 5장·클래스의 explicit 과 mutable 키워드·explicit, 인쇄 p.183)

mutable로 선언한 멤버**변수**는 const 멤버함수 안에서도 값을 바꿀 수 있다. [U03-039] (B 5장·클래스의 explicit 과 mutable 키워드·mutable, 인쇄 p.183)

책의 Server 예제는 const인 GetUserInfo에서 캐쉬를 갱신하기 위해 Cache 멤버를 **mutable**로 선언한다. [U03-040] (B 5장·클래스의 explicit 과 mutable 키워드·mutable, 인쇄 p.186-187)

동작 방식·절차

MyString 복사 생성자의 처리 순서

1. 원본의 문자열 길이를 복사하고 그 길이를 할당 용량으로 설정한다. [U03-041] (B 5장·내가 만들어보는 문자열 클래스, 인쇄 p.148)

2. 문자열 길이만큼 새 char **배열**을 할당한다. [U03-042] (B 5장·내가 만들어보는 문자열 클래스, 인쇄 p.148)

3. 원본의 문자를 새 **배열**에 차례로 복사한다. [U03-043] (B 5장·내가 만들어보는 문자열 클래스, 인쇄 p.148)

insert에서 새 할당이 필요한 경우의 순서

1. 기존 주소를 보관하고 새 메모리를 할당한다. [U03-044] (B 5장·내가 만들어보는 문자열 클래스·insert, 인쇄 p.152)

2. 삽입 위치 앞부분, 삽입할 문자열, 기존 문자열의 나머지 뒷부분 순서로 복사한다. [U03-045] (B 5장·내가 만들어보는 문자열 클래스·insert, 인쇄 p.152)

3. 기존 메모리를 해제하고 문자열 길이를 갱신한 뒤 *this를 반환한다. [U03-046] (B 5장·내가 만들어보는 문자열 클래스·insert, 인쇄 p.152)

주요 문법·표·수식

MyString 복사 생성자 - 원문 발췌 (원문 발췌)

```
MyString::MyString(const MyString& str) {
  string_length = str.string_length;
  memory_capacity = str.string_length;
  string_content = new char[string_length];

  for (int i = 0; i != string_length; i++) {
    string_content[i] = str.string_content[i];
  }
}
```

[U03-047] (B 5장·내가 만들어보는 문자열 클래스, 인쇄 p.148)

얕은 복사와 깊은 복사의 대상

방식	복사 내용
얕은 복사 [U03-048] (L05B Shallow vs Deep Copy, 슬라이드 14)	얕은 복사는 **포인터**가 가리키는 데이터가 아니라 **포인터** 주소값 자체를 복사한다. [U03-025] (L05B Shallow vs Deep Copy, 슬라이드 14)
깊은 복사 [U03-049] (L05B Shallow vs Deep Copy, 슬라이드 20)	**깊은 복사**는 새로운 힙 공간을 할당한 뒤 동일한 데이터를 복사하여 복사 생성하는 방식이다. [U03-026] (L05B Shallow vs Deep Copy, 슬라이드 20)

핵심 용어 표

용어	원문에 나온 정의·설명
클래스	**클래스**는 **객체**가 생성되기 위한 틀이며 **객체**의 데이터와 기능을 정의하는 사용자 정의 자료형이다. [U03-007] (L05A Classes and Objects, 슬라이드 14)
객체	**객체**는 **클래스**로부터 생성된 실체이다. [U03-008] (L05A Classes and Objects, 슬라이드 15)
캡슐화	**캡슐화**는 **클래스**가 데이터와 데이터를 기반으로 하는 기능을 모두 포함하는 것이다. [U03-004] (L05A Object Oriented Programming, 슬라이드 8)
생성자	**생성자**는 **객체**가 생성될 때 자동 호출되는 특수한 멤버함수로, **클래스**와 이름이 같고 반환형이 없다. [U03-013] (L05A Constructors, 슬라이드 41)
소멸자	**소멸자**는 **객체**가 소멸할 때 자동 호출되며 **클래스** 이름 앞에 ~를 붙이고 반환형·매개**변수**가 없다. [U03-014] (L05A Destructor, 슬라이드 44)
복사 생성자	**복사 생성자**는 기존 **객체**를 기반으로 새로운 **객체**가 생성될 때 어떻게 복사할지를 정의한다. [U03-022] (L05B Copy Constructor, 슬라이드 6)
깊은 복사	**깊은 복사**는 새로운 힙 공간을 할당한 뒤 동일한 데이터를 복사하여 복사 생성하는 방식이다. [U03-026] (L05B Shallow vs Deep Copy, 슬라이드 20)
this 포인터	**this 포인터**는 멤버함수를 호출한 **객체**의 주소값이다. [U03-028] (L05B this Pointer, 슬라이드 26)
explicit	**생성자**를 **explicit**으로 선언하면 그 **생성자**를 이용한 암시적 변환을 막을 수 있다. [U03-038] (B 5장·클래스의 explicit 과 mutable 키워드·explicit, 인쇄 p.183)
mutable	**mutable**로 선언한 멤버**변수**는 const 멤버함수 안에서도 값을 바꿀 수 있다. [U03-039] (B 5장·클래스의 explicit 과 mutable 키워드·mutable, 인쇄 p.183)

단원 요약

요약 기준: 원문 Summary: L05A 슬라이드 38·62 및 L05B 슬라이드 23·40

클래스는 **객체**의 틀이며, 접근제한자로 멤버 접근을 제한하고 .h 명세와 .cpp 구현을 분리한다. [U03-050] (L05A (Summary) Class and Object, 슬라이드 38)

생성자 초기화 리스트는 생성과 동시에 값을 지정하고, **생성자** 위임과 기본 매개**변수**를 사용할 수 있다. [U03-051] (L05A (Summary) Constructor Overloading, 슬라이드 62)

기본 **복사 생성자**는 멤버**변수**를 그대로 대입하며, **깊은 복사**는 새 힙 공간을 할당하여 복사 생성한다. [U03-052] (L05B (Summary) Copy Constructor, 슬라이드 23)

this는 현재 **객체**의 주소값이고, const·static·friend는 멤버의 변경 가능성·**클래스** 소속·접근 허용에 관계된다. [U03-053] (L05B (Summary) this, const, static, friend, 슬라이드 40)

04 상속·다형성

B PDF 275-344 | L06 PDF 1-53 | L07 PDF 1-47

다루는 내용

상속·**유도 클래스**·protected·**생성자**와 **소멸자**·**기본 클래스 생성자**와 멤버함수의 관계를 다룬다. [U04-001] (L06 Inheritance, 슬라이드 8)

다형성·**동적 바인딩**·가상함수·override/final·**순수가상함수**·추상**클래스**·인터페이스를 다룬다. [U04-002] (L07 Polymorphism, 슬라이드 4)

핵심 개념

상속과 클래스 관계

상속은 **기본 클래스**를 기반으로 새 **클래스**를 만드는 기능이다. [U04-003] (L06 (Summary) Inheritance, 슬라이드 53)

기본 클래스(base class/parent class/super class)는 **상속**의 대상이 되는 **클래스**이다. [U04-004] (L06 Terminology, 슬라이드 16)

유도 클래스(derived class/child class/sub class)는 **기본 클래스**로부터 생성되며 데이터와 행동을 **상속**받는 **클래스**이다. [U04-005] (L06 Terminology, 슬라이드 16)

Manager는 Employee라는 is-a 관계이고, EmployeeList가 Employee를 포함하는 관계는 has-a이다. [U04-006] (B 7장·가상 함수와 다형성·is-a와 has-a, 인쇄 p.298,300)

protected 멤버는 기본·**유도 클래스** 내부에서 접근 가능하지만 그 **객체**를 통한 외부 접근은 불가능하며, private은 **유도 클래스**에서 직접 접근할 수 없다. [U04-007] (L06 Protected Member, 슬라이드 23)

상속에서의 생성·복사·소멸

유도 클래스 객체 생성 시 **기본 클래스 생성자**가 먼저 호출되고 **유도 클래스 생성자**가 뒤에 호출된다. [U04-008] (L06 Constructors and Destructors, 슬라이드 27)

소멸자는 **유도 클래스**, **기본 클래스** 순서로 호출되어 **생성자**와 반대 순서이다. [U04-009] (L06 Constructors and Destructors, 슬라이드 31)

유도 클래스의 **초기화** 리스트에서 원하는 **기본 클래스 생성자**를 호출하며, 명시하지 않으면 **기본 클래스**의 인자 없는 **생성자**가 호출된다. [U04-010] (L06 Passing Arguments to Base Class Constructor, 슬라이드 38)

사용자가 **유도 클래스**의 **복사 생성자**를 구현한 경우 **기본 클래스**의 **복사 생성자**를 명시적으로 호출해야 하며, 명시하지 않으면 **기본 클래스**의 인자 없는 **생성자**가 호출된다. [U04-011] (L06 Copy Constructor, 슬라이드 45)

정적·동적 바인딩과 다형성

정적 바인딩은 **컴파일** 시 타입을 기준으로 호출 함수를 결정한다. [U04-012] (L07 Polymorphism, 슬라이드 13)

동적 바인딩은 런타임에 실제 메모리에 저장된 타입을 기준으로 호출 함수를 결정한다. [U04-013] (L07 Polymorphism, 슬라이드 13)

다형성(polymorphism)은 하나의 메소드를 호출했음에도 여러 가지 다른 작업을 하는 것이다. [U04-014] (B 7장·가상 함수와 다형성, 인쇄 p.321)

C++의 런타임 **다형성** 구현에는 **상속**, **기본 클래스 포인터** 또는 **참조자**, 가상함수가 필요하다. [U04-015] (L07 Polymorphism, 슬라이드 6)

재정의로 가려진 **기본 클래스**의 멤버함수는 Base::Member() 형식으로 호출할 수 있다. [U04-016] (L06 Using Member Function of Base Class, 슬라이드 47)

virtual·override·final

오버라이드할 **기본 클래스**의 함수 앞에 virtual을 붙이면 가상함수가 된다. [U04-017] (L07 Virtual Function, 슬라이드 15)

유도 클래스를 **기본 클래스 포인터**로 해제하는 강의 예제는 기본 **소멸자**를 가상으로 선언하면 유도·기본 **소멸자**가 모두 호출된다. [U04-018] (L07 Virtual Destructor, 슬라이드 18)

override 지정자는 오버라이딩 의도를 표시하고, 함수 원형이 맞지 않는 실수를 **컴파일** 단계에서 찾도록 한다. [U04-019] (L07 override Specifier, 슬라이드 25,28)

클래스의 final은 추가 **상속**을 막고, 멤버함수의 final은 **유도 클래스**에서 가상함수를 오버라이드하지 못하게 한다. [U04-020] (L07 final Specifier, 슬라이드 29)

추상 클래스·순수가상함수·인터페이스

추상 클래스(Abstract Class)는 **객체**를 생성할 수 없으며 **상속** 계층에서 **기본 클래스**로 사용되는 **클래스**이다. [U04-021] (L07 Pure Virtual Functions and Abstract Classes, 슬라이드 33)

가상 멤버함수의 선언 뒤에 =0을 붙이면 **순수가상함수**가 되며, 이를 하나 이상 갖는 **클래스**는 **추상 클래스**이다. [U04-022] (L07 Pure Virtual Functions and Abstract Classes, 슬라이드 34)

순수가상함수를 오버라이드하지 않는 **유도 클래스**도 **추상 클래스**로 간주된다. [U04-023] (L07 Pure Virtual Functions and Abstract Classes, 슬라이드 35)

추상 클래스 자체의 **객체**를 만들 수 없어도 그 **포인터·참조자**로 오버라이드된 함수를 **동적 바인딩**할 수 있다. [U04-024] (L07 Pure Virtual Functions and Abstract Classes, 슬라이드 38)

강의는 **순수가상함수**만 가진 **추상 클래스**를 인터페이스 **클래스**라고 부른다. [U04-025] (L07 Abstract Classes and Interface, 슬라이드 40)

가상함수 테이블과 캐스팅

가상함수 테이블(vtable)은 함수 이름과 실제로 대응하는 함수를 테이블로 저장한다. [U04-026] (B 7장·상속에 관련된 잡다한 내용들·가상 함수의 구현 원리, 인쇄 p.327)

강의의 구현 설명에서는 **객체**가 호출 정보를 가진 vtable을 가리키는 **포인터**(vptr)를 포함한다. [U04-027] (L07 vtable, 슬라이드 44)

책은 작동이 보장되지 않는 강제 다운캐스팅을 권장하지 않으며, dynamic_cast를 **상속** 관계의 **포인터** 사이 캐스팅 수단으로 소개한다. [U04-028] (B 7장·가상 함수와 다형성·dyanmic_cast, 인쇄 p.307)

다중 상속·이름 충돌·가상 상속

다중 상속(multiple inheritance)은 한 **클래스**가 여러 **클래스**를 **상속**받는 것이다. [U04-029] (B 7장·상속에 관련된 잡다한 내용들·다중 상속, 인쇄 p.332)

다중 상속에서 기반 **클래스 생성자**의 호출 순서는 **상속**하는 순서에 따른다. [U04-030] (B 7장·상속에 관련된 잡다한 내용들·다중 상속, 인쇄 p.334)

두 기반 **클래스**에 같은 이름의 멤버가 있으면 파생 **객체**에서 그 이름으로 접근할 때 모호성 오류가 발생할 수 있다. [U04-031] (B 7장·상속에 관련된 잡다한 내용들·다중 상속 시 주의할 점, 인쇄 p.335)

다이아몬드 **상속**은 **상속**되는 두 **클래스**가 공통 기반 **클래스**를 포함하여 그 내용이 중복되는 형태이다. [U04-032] (B 7장·상속에 관련된 잡다한 내용들·다중 상속 시 주의할 점, 인쇄 p.336)

공통 기반 Human을 virtual로 **상속**하면 Me에 Human을 한 번만 포함하도록 할 수 있으며 Me **생성자**에서 Human **생성자**도 호출해야 한다. [U04-033] (B 7장·상속에 관련된 잡다한 내용들·가상 상속, 인쇄 p.337)

책의 사원 관리와 설계 방식 비교

사원 관리 예제는 Employee와 Manager를 Employee*로 관리하고 calculate_pay와 print_info를 가상함수로 만들어 실제 종류에 맞게 호출한다. [U04-034] (B 7장·가상 함수와 다형성·Employee 문제, 인쇄 p.318-319,321)

차량 예제의 브리지 패턴은 하나의 범주를 Engine* 같은 멤버 **포인터**로 두고, 중첩된 일반화는 한 계층 아래 다른 계층의 파생 **클래스**를 둔다. [U04-035] (B 7장·다중 상속은 언제 사용해야 할까?, 인쇄 p.338)

같은 차량 예제의 **다중 상속** 방식은 환경과 동력원에 해당하는 **클래스**를 함께 **상속**받는다. [U04-036] (B 7장·다중 상속은 언제 사용해야 할까?, 인쇄 p.338-339)

책은 브리지 패턴·중첩된 일반화·**다중 상속** 중 절대적으로 우월한 방식은 없으며 상황에 맞게 고른다고 정리한다. [U04-037] (B 7장·다중 상속은 언제 사용해야 할까?, 인쇄 p.339)

동작 방식·절차

유도 클래스의 생성과 소멸 순서

1. 생성 시 **기본 클래스 생성자**를 먼저 호출한다. [U04-038] (L06 Constructors and Destructors, 슬라이드 27)

2. 이후 **유도 클래스 생성자**를 호출한다. [U04-039] (L06 Constructors and Destructors, 슬라이드 27)

3. 소멸 시에는 **유도 클래스 소멸자**를 먼저 호출한다. [U04-040] (L06 Constructors and Destructors, 슬라이드 31)

4. 이후 **기본 클래스 소멸자**를 호출한다. [U04-041] (L06 Constructors and Destructors, 슬라이드 31)

가상함수 호출의 원문 설명 순서

1. 포인터의 기반 **클래스**에서 함수의 정의를 찾는다. [U04-042] (B 7장·가상 함수의 구현 원리, 인쇄 p.328)

2. 가상함수이면 직접 실행하는 대신 가상함수 테이블에서 대응 함수를 선택한다. [U04-043] (B 7장·가상 함수의 구현 원리, 인쇄 p.328)

3. 실행 시 실제 **객체**의 가상함수 테이블을 참조하여 오버라이드된 함수를 호출한다. [U04-044] (B 7장·가상 함수의 구현 원리, 인쇄 p.328-329)

주요 문법·표·수식

순수가상함수와 가상 소멸자 - 원문 발췌 (원문 발췌)

```
class Animal {
 public:
  Animal() {}
  virtual ~Animal() {}
  virtual void speak() = 0;
};
```

[U04-045] (B 7장·순수 가상 함수와 추상 클래스, 인쇄 p.329)

순수가상함수의 오버라이딩 - 원문 발췌 (원문 발췌)

```
class Dog : public Animal {
 public:
  Dog() : Animal() {}
  void speak() override { std::cout << "왈왈" << std::endl; }
};
```

[U04-046] (B 7장·순수 가상 함수와 추상 클래스, 인쇄 p.329)

차량 예제: 브리지 패턴의 **클래스** 수: N + M; N: 차량이 작동하는 환경의 수, M: 동력원의 종류.
[U04-047] (B 7장·다중 상속은 언제 사용해야 할까?, 인쇄 p.337-338)

차량 예제: 중첩된 일반화·**다중 상속**의 조합: N × M; 각 환경과 동력원의 모든 조합에 해당하는 파생 **클래스**의 최대 수. [U04-048] (B 7장·다중 상속은 언제 사용해야 할까?, 인쇄 p.338)

바인딩의 결정 시점과 기준

구분	시점	기준
정적 바인딩 [U04-049] (L07 Polymorphism, 슬라이드 13)	**컴파일** 시 [U04-050] (L07 Polymorphism, 슬라이드 13)	타입을 기준으로 호출 함수를 결정한다. [U04-051] (L07 Polymorphism, 슬라이드 13)
동적 바인딩 [U04-052] (L07 Polymorphism, 슬라이드 13)	런타임 [U04-053] (L07 Polymorphism, 슬라이드 13)	실제 메모리에 저장된 타입을 기준으로 결정한다. [U04-054] (L07 Polymorphism, 슬라이드 13)

핵심 용어 표

용어	원문에 나온 정의·설명
상속	**상속**은 **기본 클래스**를 기반으로 새 **클래스**를 만드는 기능이다. [U04-003] (L06 (Summary) Inheritance, 슬라이드 53)
기본 클래스	**기본 클래스**(base class/parent class/super class)는 **상속**의 대상이 되는 **클래스**이다. [U04-004] (L06 Terminology, 슬라이드 16)
유도 클래스	**유도 클래스**(derived class/child class/sub class)는 **기본 클래스**로부터 생성되며 데이터와 행동을 **상속**받는 **클래스**이다. [U04-005] (L06 Terminology, 슬라이드 16)
정적 바인딩	**정적 바인딩**은 **컴파일** 시 타입을 기준으로 호출 함수를 결정한다. [U04-012] (L07 Polymorphism, 슬라이드 13)
동적 바인딩	**동적 바인딩**은 런타임에 실제 메모리에 저장된 타입을 기준으로 호출 함수를 결정한다. [U04-013] (L07 Polymorphism, 슬라이드 13)
다형성	**다형성**(polymorphism)은 하나의 메소드를 호출했음에도 여러 가지 다른 작업을 하는 것이다. [U04-014] (B 7장·가상 함수와 다형성, 인쇄 p.321)
추상 클래스	**추상 클래스**(Abstract Class)는 **객체**를 생성할 수 없으며 **상속** 계층에서 **기본 클래스**로 사용되는 **클래스**이다. [U04-021] (L07 Pure Virtual Functions and Abstract Classes, 슬라이드 33)
순수가상함수	가상 멤버함수의 선언 뒤에 =0을 붙이면 **순수가상함수**가 되며, 이를 하나 이상 갖는 **클래스**는 **추상 클래스**이다. [U04-022] (L07 Pure Virtual Functions and Abstract Classes, 슬라이드 34)
다중 상속	**다중 상속**(multiple inheritance)은 한 **클래스**가 여러 **클래스**를 **상속**받는 것이다. [U04-029] (B 7장·상속에 관련된 잡다한 내용들·다중 상속, 인쇄 p.332)

단원 요약

요약 기준: 편집 요약 (상속 부분은 L06 슬라이드 53의 원문 Summary 기준)

상속은 **기본 클래스**를 기반으로 새 **클래스**를 만들고, **유도 클래스**의 **초기화** 리스트로 **기본 클래스**를 어떻게 생성할지 명시하는 기능이다. [U04-055] (L06 (Summary) Inheritance, 슬라이드 53)

동적 바인딩은 런타임에 실제 메모리에 저장된 타입을 기준으로 호출 함수를 결정한다. [U04-013] (L07 Polymorphism, 슬라이드 13)

가상 멤버함수의 선언 뒤에 =0을 붙이면 **순수가상함수**가 되며, 이를 하나 이상 갖는 **클래스**는 **추상 클래스**이다. [U04-022] (L07 Pure Virtual Functions and Abstract Classes, 슬라이드 34)

다중 상속은 이름 **충돌**과 기반 **클래스** 중복에 주의하며, 설계 방식은 상황에 맞게 선택한다. [U04-056] (B 7장·상속에 관련된 잡다한 내용들, 인쇄 p.335-336,339)

05 연산자 오버로딩·N차원 배열 프로젝트

B PDF 193-274 | L08 PDF 1-48

다루는 내용

멤버함수·전역함수·스트림·대입·첨자 연산자의 오버로딩을 다룬다. [U05-001] (L08 Operator Overloading, 슬라이드 4)

N차원 **배열** 프로젝트는 **연산자 오버로딩**을 종합하여 []로 원소에 접근하는 **배열 클래스**를 구현한다. [U05-002] (B 6장·N 차원 배열 만들기 프로젝트, 인쇄 p.239-240)

핵심 개념

연산자 오버로딩의 규칙

연산자 오버로딩은 +, =, * 등의 연산자를 사용자 정의 타입에 대해 사용할 수 있게 하는 기능이다. [U05-003] (L08 Operator Overloading, 슬라이드 6)

연산자 오버로딩은 연산 우선순위나 단항·이항·삼항 구분을 바꾸지 못하며, 기본 타입의 연산자나 새 연산자를 정의할 수 없다. [U05-004] (L08 Operator Overloading, 슬라이드 11)

[ ], ( ), ->, = 등은 멤버함수로만 오버로딩할 수 있다. [U05-005] (L08 Operator Overloading, 슬라이드 11)

operator+와 operator=를 정의해도 operator+=는 자동 생성되지 않으므로 별도로 구현해야 한다. [U05-006] (B 6장·연산자 오버로딩, 인쇄 p.201)

멤버함수·전역함수·friend

멤버함수 방식의 이항 연산자는 왼쪽 **객체**의 operator 함수에 오른쪽 **객체**를 인자로 넘긴다. [U05-007] (L08 Binary operator, as Member Function, 슬라이드 14)

전역함수 방식의 이항 연산자는 왼쪽 피연산자도 매개**변수**로 전달한다. [U05-008] (L08 Binary operator, as Global Function, 슬라이드 22)

멤버함수만으로는 p1*3이 가능해도 3*p1을 처리할 수 없는 경우가 있어 전역함수를 추가 구현한다. [U05-009] (L08 Limitation of Member Function·Binary operator, as Global Function, 슬라이드 20,27)

friend로 선언한 **전역 연산자 함수**는 **클래스**의 private 멤버에 접근할 수 있다. [U05-010] (L08 Binary operator, as Global Function, 슬라이드 23)

클래스 내부에 friend 함수의 코드가 있어도 그 함수가 멤버함수인 것은 아니다. [U05-011] (L08 Binary operator, as Global Function, 슬라이드 24-25)

비교·사칙연산·대입

MyString의 operator==는 compare의 결과를 논리 부정하여 같을 때 true를 반환하도록 한다. [U05-012] (B 6장·MyString 의 == 연산자 오버로딩, 인쇄 p.189)

책의 Complex 사칙연산 함수는 원래 **객체**를 바꾸지 않고 계산 결과의 값을 반환한다. [U05-013] (B 6장·복소수 클래스 만들기, 인쇄 p.196)

자동 생성된 **대입 연산자**는 얕은 복사를 수행하므로 **깊은 복사**가 필요한 경우 직접 정의해야 한다. [U05-014] (L08 Assignment Operator Overloading, 슬라이드 37)

대입 연산자는 자기 대입을 검사하고, **포인터** 멤버의 데이터가 **배열**이면 해제 후 재할당을 고려한다. [U05-015] (L08 Assignment Operator Overloading, 슬라이드 39)

유도 클래스의 **대입 연산자**는 **기본 클래스**의 **대입 연산자**를 호출한 다음 **유도 클래스** 속성을 대입한다. [U05-016] (L08 Copy/Move Constructor & Overloaded Operator =, 슬라이드 40)

스트림 삽입·추출과 첨자

<<는 cout **객체**(ostream **클래스**)의 오버로딩된 연산자이다. [U05-017] (L08 OStream, 슬라이드 30)

스트림 삽입 연산자는 전역함수로 선언하고 ostream의 **참조자**를 반환하여 chain insertion을 가능하게 한다. [U05-018] (L08 Stream Insertion and Extraction Overloading, 슬라이드 31)

스트림 추출 연산자의 오른쪽 매개**변수**는 const가 아니다. [U05-019] (L08 Stream Insertion and Extraction Operators Overloading, 슬라이드 32)

첨자 연산자는 멤버함수로 오버로딩하여 경계 검사 등의 기능을 확장할 수 있다. [U05-020] (L08 Subscript([ ]) operator overloading, 슬라이드 43)

첨자 연산자로 **배열** 내부 값을 수정하려면 참조형을 반환하고, const **객체**를 위한 접근도 제공한다. [U05-021] (L08 Subscript([ ]) operator overloading, 슬라이드 45-46)

Wrapper·타입 변환·전위/후위

Wrapper 클래스는 기본 자료형을 **클래스**로 포장하여 각 자료형을 **객체**로 사용하는 **클래스**이다. [U05-022] (B 6장·int Wrapper 클래스 - 타입 변환 연산자, 인쇄 p.233)

타입 변환 연산자는 operator 뒤에 변환할 타입을 쓰며 **생성자**처럼 반환 타입을 따로 쓰지 않는다. [U05-023] (B 6장·int Wrapper 클래스 - 타입 변환 연산자, 인쇄 p.234)

전위 증감은 매개**변수** 없는 operator++()/operator--()로, **후위 증감**은 의미 없는 int 매개**변수**를 가진 형태로 구분한다. [U05-024] (B 6장·전위/후위 증감 연산자, 인쇄 p.236)

전위 증감 연산자는 값을 바꾼 자기 자신을 반환한다. [U05-025] (B 6장·전위/후위 증감 연산자, 인쇄 p.236)

후위 증감 연산자는 값이 바뀌기 이전의 **객체**를 반환한다. [U05-026] (B 6장·전위/후위 증감 연산자, 인쇄 p.236)

C++ 스타일의 캐스팅

static_cast는 언어가 지원하는 일반적인 타입 변환이고, const_cast는 **객체**의 **상수**성(const)을 없애는 타입 변환이다. [U05-027] (B 6장·C++ 스타일의 캐스팅, 인쇄 p.241)

책은 dynamic_cast를 파생 **클래스** 사이의 다운캐스팅으로 소개하고, reinterpret_cast는 서로 관련 없는 **포인터** 사이의 캐스팅 등에 사용한다고 설명한다. [U05-028] (B 6장·C++ 스타일의 캐스팅, 인쇄 p.241)

N차원 배열의 데이터 구조와 재귀

프로젝트의 Array는 차원 수 dim, 각 차원의 크기 size, 시작 Address **포인터** top을 보관한다. [U05-029] (B 6장·N 차원 배열 만들기, 인쇄 p.246)

Address는 level과 void* next를 가지며, 마지막 레벨에서는 int **배열**을, 그 앞 레벨에서는 다음 Address **배열**을 가리킨다. [U05-030] (B 6장·N 차원 배열 만들기, 인쇄 p.243-244)

initialize_address는 마지막 레벨이면 int **배열**을 만들고, 그 전이면 다음 Address **배열**을 만들어 각 원소를 재귀적으로 **초기화**한다. [U05-031] (B 6장·N 차원 배열 만들기, 인쇄 p.247-248)

생성은 위에서 아래로 메모리를 확장하고, 소멸은 아래에서 위로 메모리를 해제한다. [U05-032] (B 6장·N 차원 배열 만들기, 인쇄 p.249)

중간 객체·타입 변환·반복자

연속된 []의 중간 단계는 Int Wrapper **객체**로 표현하고, 실제 데이터에 도달하면 int처럼 읽거나 대입하도록 한다. [U05-033] (B 6장·N 차원 배열 만들기·operator[] 문제, 인쇄 p.251-252,255)

반복자(iterator)는 **배열**의 모든 원소를 순차적으로 접근할 수 있게 하며, ++로 다음 원소를 가리킨다. [U05-034] (B 6장·N 차원 배열 만들기 프로젝트·반복자, 인쇄 p.260)

Iterator는 location **배열**로 현재 원소의 좌표를 보관한다. [U05-035] (B 6장·N 차원 배열 만들기·반복자, 인쇄 p.260)

begin은 첫 원소, end는 마지막 원소 바로 다음을 뜻하므로 end의 값을 참조하려 하면 오류가 발생한다. [U05-036] (B 6장·N 차원 배열 만들기·반복자, 인쇄 p.261)

동작 방식·절차

후위 증가 연산자의 원문 처리 순서

1. 증가 전 자기 자신으로 임시 **객체**를 복사 생성한다. [U05-037] (B 6장·전위/후위 증감 연산자, 인쇄 p.238)

2. 현재 **객체**의 값을 증가시킨다. [U05-038] (B 6장·전위/후위 증감 연산자, 인쇄 p.238)

3. 증가 전 **상태**를 보관한 임시 **객체**를 반환한다. [U05-039] (B 6장·전위/후위 증감 연산자, 인쇄 p.238)

N차원 Address 초기화 순서

1. 현재 Address가 마지막 레벨이면 해당 크기의 int **배열**을 할당하고 재귀를 끝낸다. [U05-040] (B 6장·N 차원 배열 만들기, 인쇄 p.247)

2. 마지막 레벨이 아니면 다음 Address **배열**을 할당한다. [U05-041] (B 6장·N 차원 배열 만들기, 인쇄 p.247)

3. 새 각 원소의 level을 다음 단계로 정하고 initialize_address를 재귀 호출한다. [U05-042] (B 6장·N 차원 배열 만들기, 인쇄 p.248)

주요 문법·표·수식

대입 연산자 - 원문 발췌 (원문 발췌)

```
Complex& Complex::operator=(const Complex& c)

{
  real = c.real;
  img = c.img;
  return *this;
}
```

[U05-043] (B 6장·대입 연산자 함수, 인쇄 p.197)

후위 증가 연산자 - 원문 발췌 (원문 발췌)

```
Test operator++(int) {
  Test temp(*this);
  x++;
  std::cout << "후위 증감 연산자" << std::endl;
  return temp;
}
```

[U05-044] (B 6장·전위/후위 증감 연산자, 인쇄 p.238)

N차원 **배열**을 하나의 1차원 **배열**로 할당하는 크기: x1 * x2 * ... * xn; 원문에서 사용자가 원하는 **배열**을 arr[x1][x2]...[xn]으로 두었을 때의 크기. [U05-045] (B 6장·N 차원 배열 만들기, 인쇄 p.242)

이항 연산자의 피연산자 전달 방식

구현 형태	피연산자 전달
멤버함수 [U05-046] (L08 Binary operator, as Member Function, 슬라이드 14)	멤버함수 방식의 이항 연산자는 왼쪽 **객체**의 operator 함수에 오른쪽 **객체**를 인자로 넘긴다. [U05-007] (L08 Binary operator, as Member Function, 슬라이드 14)
전역함수 [U05-047] (L08 Binary operator, as Global Function, 슬라이드 22)	전역함수 방식의 이항 연산자는 왼쪽 피연산자도 매개**변수**로 전달한다. [U05-008] (L08 Binary operator, as Global Function, 슬라이드 22)

핵심 용어 표

용어	원문에 나온 정의·설명
연산자 오버로딩	**연산자 오버로딩**은 +, =, * 등의 연산자를 사용자 정의 타입에 대해 사용할 수 있게 하는 기능이다. [U05-003] (L08 Operator Overloading, 슬라이드 6)
멤버 연산자 함수	멤버함수 방식의 이항 연산자는 왼쪽 **객체**의 operator 함수에 오른쪽 **객체**를 인자로 넘긴다. [U05-007] (L08 Binary operator, as Member Function, 슬라이드 14)
전역 연산자 함수	전역함수 방식의 이항 연산자는 왼쪽 피연산자도 매개**변수**로 전달한다. [U05-008] (L08 Binary operator, as Global Function, 슬라이드 22)
대입 연산자	자동 생성된 **대입 연산자**는 얕은 복사를 수행하므로 **깊은 복사**가 필요한 경우 직접 정의해야 한다. [U05-014] (L08 Assignment Operator Overloading, 슬라이드 37)
Wrapper 클래스	**Wrapper 클래스**는 기본 자료형을 **클래스**로 포장하여 각 자료형을 **객체**로 사용하는 **클래스**이다. [U05-022] (B 6장·int Wrapper 클래스 - 타입 변환 연산자, 인쇄 p.233)
타입 변환 연산자	**타입 변환 연산자**는 operator 뒤에 변환할 타입을 쓰며 **생성자**처럼 반환 타입을 따로 쓰지 않는다. [U05-023] (B 6장·int Wrapper 클래스 - 타입 변환 연산자, 인쇄 p.234)
전위 증감	**전위 증감** 연산자는 값을 바꾼 자기 자신을 반환한다. [U05-025] (B 6장·전위/후위 증감 연산자, 인쇄 p.236)
후위 증감	**후위 증감** 연산자는 값이 바뀌기 이전의 **객체**를 반환한다. [U05-026] (B 6장·전위/후위 증감 연산자, 인쇄 p.236)
반복자(iterator)	**반복자**(iterator)는 **배열**의 모든 원소를 순차적으로 접근할 수 있게 하며, ++로 다음 원소를 가리킨다. [U05-034] (B 6장·N 차원 배열 만들기 프로젝트·반복자, 인쇄 p.260)

단원 요약

요약 기준: 원문 정리: B 인쇄 p.238~239(PDF243~244) 및 L08 슬라이드 48

책은 동등한 두 **객체**의 이항 연산자는 외부함수로, 한 **객체**만 바뀌는 이항 연산자와 단항 연산자는 멤버함수로 구현하는 것을 권한다. [U05-048] (B 6장·잡다한 연산자들의 오버로딩·정리, 인쇄 p.238-239)

스트림 연산자는 연속 삽입을 위해 **참조자**를 반환하며, **깊은 복사**가 필요한 **대입 연산자**는 직접 오버로딩한다. [U05-049] (L08 (Summary)Operator Overloading, 슬라이드 48)

첨자 연산자는 일반적으로 const 멤버와 참조 반환 함수를 각각 오버로딩한다. [U05-050] (L08 (Summary)Operator Overloading, 슬라이드 48)

06 표준 입출력·파일·엑셀 프로젝트

B PDF 345-414

다루는 내용

C++ 입출력 라이브러리의 **클래스**와 표준 스트림·파일 스트림을 다룬다. [U06-001] (B C++ 표준 입출력 라이브러리 / C++ 파일 입출력, 인쇄 p.340,356)

배운 내용을 바탕으로 콘솔용 Excel을 만들고, 셀과 표를 확장한다. [U06-002] (B 엑셀 만들기 프로젝트, 인쇄 p.374,394)

핵심 개념

입출력 클래스와 스트림

ios_base는 스트림의 입출력 형식 관련 데이터를 처리하고, ios는 **스트림 버퍼**를 **초기화**하며 입출력 작업의 **상태**를 처리한다. [U06-003] (B C++ 입출력 라이브러리, 인쇄 p.341)

스트림 버퍼는 데이터를 내보내거나 받아들이기 전에 임시로 저장하는 곳이다. [U06-004] (B C++ 입출력 라이브러리, 인쇄 p.341)

istream은 실제로 입력을 수행하며, operator>>가 정의되어 있고 cin은 istream **클래스**의 **객체** 중 하나이다. [U06-005] (B istream 클래스, 인쇄 p.342)

operator>>로 문자열을 입력하면 공백문자에 따라 분리해서 입력받는다. [U06-006] (B istream 클래스, 인쇄 p.343-344)

상태 플래그와 입력 복구

타입에 맞지 않는 값을 넣어 오류가 발생하면 failbit가 켜지고, 입력값을 받지 않고 리턴한다. [U06-007] (B istream 클래스, 인쇄 p.346)

fail 함수는 failbit가 true이거나 badbit가 true이면 true를 리턴한다. [U06-008] (B istream 클래스, 인쇄 p.348)

clear()에 인자를 주지 않으면 플래그를 goodbit으로 **초기화**하고, ignore는 최대 첫 번째 인자만큼, 두 번째 인자가 나올 때까지 그 문자를 포함하여 버퍼에서 무시한다. [U06-009] (B istream 클래스, 인쇄 p.348)

형식 플래그·조작자·스트림 버퍼

인자 두 개를 취하는 setf는 두 번째 인자의 내용을 **초기화**하고 첫 번째 인자의 형식 플래그를 적용한다. [U06-010] (B 형식 플래그(format flag) 와 조작자(Manipulator), 인쇄 p.350)

조작자는 스트림을 조작하여 입력 혹은 출력 방식을 바꾸는 함수이다. [U06-011] (B 형식 플래그(format flag) 와 조작자(Manipulator), 인쇄 p.351)

std::endl은 개행문자를 출력하고 버퍼를 모두 내보내는 flush를 수행한다. [U06-012] (B 형식 플래그(format flag) 와 조작자(Manipulator), 인쇄 p.352)

streambuf는 입력 버퍼와 출력 버퍼를 각각 get area와 put area라고 부르며, 버퍼의 시작·다음으로 읽을 문자·끝을 가리키는 **포인터**를 둔다. [U06-013] (B 스트림 버퍼에 대해, 인쇄 p.353)

rdbuf는 입력을 수행하던 **streambuf 객체**의 **포인터**를 반환하고, snextc는 위치지정자를 한 칸 전진시킨 후 해당 문자를 엿본다. [U06-014] (B 스트림 버퍼에 대해, 인쇄 p.354)

파일 연결과 읽기

ifstream과 ofstream은 각각 istream과 ostream을 **상속**받으며 fstream 라이브러리에 포함된다. [U06-015] (B C++ 파일 입출력 / fstream, 인쇄 p.356)

ifstream **생성자**에 파일 경로를 전달해 파일을 열고, **is_open**으로 파일이 열렸는지 확인한다. [U06-016] (B C++ 파일 입출력 / fstream, 인쇄 p.357-358)

ifstream **소멸자**는 자동으로 close를 수행하며, 같은 **객체**를 다른 파일에 연결할 때는 close 후 open을 사용한다. [U06-017] (B C++ 파일 입출력 / fstream, 인쇄 p.358-359)

binary 옵션은 이진 그대로의 값을 받는 설정이며, read의 두 번째 인자로 읽을 바이트 수를 전달한다. [U06-018] (B C++ 파일 입출력 / fstream, 인쇄 p.360-361)

seekg는 파일 위치지정자를 옮기며, tellg는 시작지점으로부터 위치지정자의 위치를 반환한다. [U06-019] (B 파일 전체 읽기, 인쇄 p.362)

ifstream의 getline은 개행문자나 지정한 문자가 나올 때까지 최대 지정한 크기-1만큼 읽고, 버퍼가 먼저 차면 failbit를 켠다. [U06-020] (B 파일 전체를 한 줄씩 읽기, 인쇄 p.363-364)

getline(in, s)은 istream **객체**와 string **객체**를 받아 버퍼 크기를 지정하지 않고 개행문자 혹은 파일 끝까지 입력받는다. [U06-021] (B 파일 전체를 한 줄씩 읽기, 인쇄 p.365)

출력 모드와 문자열 스트림

ofstream은 파일이 없으면 생성하고, 특별한 설정 없이 기존 파일을 열면 내용을 지우고 새 내용으로 덮어 쓴다. [U06-022] (B 파일에 쓰기, 인쇄 p.366)

ios::app은 기존 파일의 뒤에 새 내용을 붙여 쓰고, ios::ate는 파일을 열 때 위치지정자를 파일 끝에 둔다. [U06-023] (B 파일에 쓰기, 인쇄 p.367)

std::istringstream은 문자열을 입력 스트림처럼 사용하고, std::ostringstream에 출력한 값은 str 함수로 불러온다. [U06-024] (B 문자열 스트림(std::stringstream), 인쇄 p.370-371)

엑셀 프로젝트의 자료구조와 클래스

프로젝트의 **Vector**는 가변 길이 **배열**로 임의 위치의 원소에 접근하며, Stack은 항상 최상단의 데이터에만 접근한다. [U06-025] (B 엑셀 만들기 프로젝트, 인쇄 p.374)

Cell은 한 칸을 뜻하고, Table은 필요할 때 동적으로 생성한 Cell **객체**의 **포인터**를 2차원 **배열**로 보관한다. [U06-026] (B 엑셀 만들기 프로젝트, 인쇄 p.383-385)

Table의 print_table은 순수 가상함수이며, TxtTable·CSVTable·HtmlTable은 각 형식의 문자열로 표를 출력한다. [U06-027] (B 엑셀 만들기 프로젝트, 인쇄 p.387,390-391)

확장한 Cell은 stringify와 to_numeric을 순수 가상함수로 두며, StringCell·NumberCell·DateCell은 문자열·정수·시간 정보를 보관한다. [U06-028] (B 엑셀 만들기 프로젝트 2부 / Cell 클래스 확장, 인쇄 p.394-395)

수식과 명령 처리

ExprCell 예제는 숫자를 한 자리 정수로, 셀 이름을 두 글자로 제한한다. [U06-029] (B 엑셀 만들기 프로젝트 2부 / parse_expression 함수, 인쇄 p.402)

중위 표기법은 피연산자 사이에 연산자를 넣으며, **후위 표기법**은 연산자를 피연산자 뒤에 두고 괄호 없이 표현한다. [U06-030] (B 엑셀 만들기 프로젝트 2부 / 중위 표기법과 후위 표기법, 인쇄 p.398)

Excel **클래스**는 사용자 입력을 받아 테이블을 생성·관리하며, sets·setn·setd·sete·out·exit 명령을 처리한다. [U06-031] (B 엑셀 만들기 프로젝트 2부 / 엑셀 프로그램, 인쇄 p.405-407)

동작 방식·절차

파일 전체를 한 번에 읽기

1. seekg로 위치지정자를 파일 끝으로 옮기고 tellg로 크기를 읽는다. [U06-032] (B 파일 전체를 한 번에 읽기, 인쇄 p.361-362)

2. 문자열을 그 크기로 resize한 뒤 위치지정자를 파일 맨 앞으로 옮긴다. [U06-033] (B 파일 전체를 한 번에 읽기, 인쇄 p.361-362)

3. read로 파일 전체 내용을 문자열에 저장한다. [U06-034] (B 파일 전체를 한 번에 읽기, 인쇄 p.363)

후위 표기법의 계산

1. 피연산자를 만나면 **스택**에 push한다. [U06-035] (B 엑셀 만들기 프로젝트 2부 / 중위 표기법과 후위 표기법, 인쇄 p.398)

2. 연산자를 만나면 두 개를 pop하여 연산하고 결과를 다시 push한다. [U06-036] (B 엑셀 만들기 프로젝트 2부 / 중위 표기법과 후위 표기법, 인쇄 p.398)

3. 모든 계산이 끝나면 **스택**의 최종 결과값을 pop하여 리턴한다. [U06-037] (B 엑셀 만들기 프로젝트 2부 / to_numeric, 인쇄 p.401)

중위 표기법을 후위 표기법으로 변환

1. 피연산자는 exp_vec에 넣고, 여는 괄호는 **스택**에 push한다. [U06-038] (B 엑셀 만들기 프로젝트 2부 / parse_expression 함수, 인쇄 p.401)

2. 닫는 괄호를 만나면 여는 괄호가 나올 때까지 pop되는 연산자를 exp_vec에 넣는다. [U06-039] (B 엑셀 만들기 프로젝트 2부 / parse_expression 함수, 인쇄 p.401)

3. 연산자는 자신보다 우선순위가 낮은 연산자가 최상단에 오거나 **스택**이 빌 때까지 pop하여 exp_vec에 넣고, 자신을 push한다. [U06-040] (B 엑셀 만들기 프로젝트 2부 / parse_expression 함수, 인쇄 p.401)

4. 원문 구현은 수식 전체를 괄호로 감싸 마지막 괄호를 처리할 때 남은 연산자를 pop한다. [U06-041] (B 엑셀 만들기 프로젝트 2부 / parse_expression 함수, 인쇄 p.404)

주요 문법·표·수식

파일 전체 읽기: 원문 발췌 (원문 발췌)

```
in.seekg(0, std::ios::end);
int size = in.tellg();
s.resize(size);
in.seekg(0, std::ios::beg);
in.read(&s[0], size);
```

[U06-042] (B 파일 전체를 한 번에 읽기, 인쇄 p.361)

원문의 중위·후위 표기 예: 3 + 4 * 5 + 4 * (7 - 2) → 3 4 5 * + 4 7 2 - * +; 원문은 두 표기가 정확히 동일한 수식이라고 설명한다. [U06-043] (B 엑셀 만들기 프로젝트 2부 / 중위 표기법과 후위 표기법, 인쇄 p.398)

스트림 상태 플래그: 원문 설명

플래그	원문의 설명
goodbit [U06-044] (B istream 클래스, 인쇄 p.345)	스트림에 입출력 작업이 가능할 때 [U06-045] (B istream 클래스, 인쇄 p.345)
badbit [U06-046] (B istream 클래스, 인쇄 p.345)	스트림에 복구 불가능한 오류 발생 시 [U06-047] (B istream 클래스, 인쇄 p.345)
failbit [U06-048] (B istream 클래스, 인쇄 p.345)	스트림에 복구 가능한 오류 발생 시 [U06-049] (B istream 클래스, 인쇄 p.345)
eofbit [U06-050] (B istream 클래스, 인쇄 p.345)	입력 작업 시에 EOF 도달 시 [U06-051] (B istream 클래스, 인쇄 p.345)

핵심 용어 표

용어	원문에 나온 정의·설명
스트림 버퍼	**스트림 버퍼**는 데이터를 내보내거나 받아들이기 전에 임시로 저장하는 곳이다. [U06-004] (B C++ 입출력 라이브러리, 인쇄 p.341)
조작자(Manipulator)	**조작자**는 스트림을 조작하여 입력 혹은 출력 방식을 바꾸는 함수이다. [U06-011] (B 형식 플래그(format flag) 와 조작자(Manipulator), 인쇄 p.351)
streambuf	**streambuf**는 입력 버퍼와 출력 버퍼를 각각 get area와 put area라고 부르며, 버퍼의 시작·다음으로 읽을 문자·끝을 가리키는 **포인터**를 둔다. [U06-013] (B 스트림 버퍼에 대해, 인쇄 p.353)
is_open	ifstream **생성자**에 파일 경로를 전달해 파일을 열고, **is_open**으로 파일이 열렸는지 확인한다. [U06-016] (B C++ 파일 입출력 / fstream, 인쇄 p.357-358)
Vector	프로젝트의 **Vector**는 가변 길이 **배열**로 임의 위치의 원소에 접근하며, Stack은 항상 최상단의 데이터에만 접근한다. [U06-025] (B 엑셀 만들기 프로젝트, 인쇄 p.374)
후위 표기법(postfix notation)	중위 표기법은 피연산자 사이에 연산자를 넣으며, **후위 표기법**은 연산자를 피연산자 뒤에 두고 괄호 없이 표현한다. [U06-030] (B 엑셀 만들기 프로젝트 2부 / 중위 표기법과 후위 표기법, 인쇄 p.398)

단원 요약

요약 기준: 편집 요약

ios_base는 스트림의 입출력 형식 관련 데이터를 처리하고, ios는 **스트림 버퍼**를 **초기화**하며 입출력 작업의 **상태**를 처리한다. [U06-003] (B C++ 입출력 라이브러리, 인쇄 p.341)

clear()에 인자를 주지 않으면 플래그를 goodbit으로 **초기화**하고, ignore는 최대 첫 번째 인자만큼, 두 번째 인자가 나올 때까지 그 문자를 포함하여 버퍼에서 무시한다. [U06-009] (B istream 클래스, 인쇄 p.348)

std::istringstream은 문자열을 입력 스트림처럼 사용하고, std::ostringstream에 출력한 값은 str 함수로 불러온다. [U06-024] (B 문자열 스트림(std::stringstream), 인쇄 p.370-371)

확장한 Cell은 stringify와 to_numeric을 순수 가상함수로 두며, StringCell·NumberCell·DateCell은 문자열·정수·시간 정보를 보관한다. [U06-028] (B 엑셀 만들기 프로젝트 2부 / Cell 클래스 확장, 인쇄 p.394-395)

07 템플릿·가변 인자·메타프로그래밍

B PDF 415-498 | L09 PDF 1-33

다루는 내용

템플릿으로 타입에 관계없이 코드를 작성하고 함수 템플릿과 **클래스** 템플릿을 다룬다. [U07-001] (L09 Generic Programming과 템플릿, 슬라이드 3)

가변 길이 템플릿으로 임의 개수의 인자를 받고 **TMP**로 **컴파일**타임 연산을 다룬다. [U07-002] (B 가변 길이 템플릿 / 템플릿 메타프로그래밍, 인쇄 p.445,463)

핵심 개념

일반화·템플릿·인스턴스화

Generic Programming은 타입에 관계없이 동작하는 일반적인 코드를 작성하는 방법이다. [U07-003] (L09 Generic Programming, 슬라이드 5)

매크로(#define)는 코드를 단순 대체하며, 매크로문을 괄호로 감싸고 세미콜론을 포함하지 않는 것이 기본 약속이다. [U07-004] (L09 Generic Programming using Macro, 슬라이드 6,11)

템플릿은 원하는 타입을 넣으면 **컴파일러**가 해당 타입으로 코드를 생성하는 틀이다. [U07-005] (B C++ 템플릿(template), 인쇄 p.413,416)

template <typename T>의 T는 타입 이름을 받는 템플릿 인자이며, template <class T>도 같은 의미이다. [U07-006] (B C++ 템플릿(template), 인쇄 p.415-416)

클래스 템플릿에 인자를 전달하여 실제 코드를 생성하는 것을 **클래스 템플릿 인스턴스화**(class template instantiation)라고 한다. [U07-007] (B C++ 템플릿(template), 인쇄 p.418)

템플릿은 인스턴스화되지 않은 **상태**에서는 **컴파일** 시 실제 코드로 변환되지 않는다. [U07-008] (B C++ 템플릿(template), 인쇄 p.418)

특수화와 함수 템플릿

템플릿 특수화(template specialization)는 일부 경우를 따로 처리하는 것이며, 전달할 템플릿 인자가 없어도 template<>를 남겨야 한다. [U07-009] (B 템플릿 특수화(template specialization), 인쇄 p.419)

원문의 **Vector**<bool>은 여러 bool을 정수의 비트에 묶어서 저장하도록 따로 특수화한다. [U07-010] (B 템플릿 특수화(template specialization), 인쇄 p.419-420)

함수 템플릿 max(a, b)는 a와 b의 타입을 보고 **컴파일러**가 템플릿 인자를 추론하여 인스턴스화한다. [U07-011] (B 함수 템플릿(Function template), 인쇄 p.428)

원문의 bubble_sort가 작동하려면 컨테이너에 size·swap·operator[]가 정의되어 있어야 한다. [U07-012] (B 함수 템플릿(Function template), 인쇄 p.429)

템플릿이 사용하는 멤버가 전달한 타입에 없으면 런타임이 아니라 **컴파일**할 때 오류가 발생한다. [U07-013] (B 함수 템플릿(Function template), 인쇄 p.432)

함수 **객체**(Function Object), 또는 Functor는 함수는 아니지만 함수처럼 사용하는 **객체**이며 원문에서는 operator()를 정의한다. [U07-014] (B 함수 객체(Function Object - Functor)의 도입, 인쇄 p.436)

값 인자·배열·디폴트 인자

템플릿 인자는 타입 외의 값도 받을 수 있고, 원문의 add_num은 타입 T와 int num을 받는다. [U07-015] (B 타입이 아닌 템플릿 인자(non-type template arguments), 인쇄 p.438)

std::array는 원소 타입과 크기를 템플릿 인자로 명시하며, 원문 예제에서는 런타임에 동적으로 크기를 할당하지 않는다. [U07-016] (B 타입이 아닌 템플릿 인자(non-type template arguments), 인쇄 p.440)

템플릿 인자에도 = 뒤에 디폴트 값을 지정할 수 있으며 타입 역시 디폴트로 지정할 수 있다. [U07-017] (B 디폴트 템플릿 인자, 인쇄 p.442-443)

파라미터 팩과 Fold

템플릿 **파라미터 팩**과 함수 **파라미터 팩**은 각각 0개 이상의 템플릿 인자와 함수 인자를 나타낸다. [U07-018] (B 가변 길이 템플릿, 인쇄 p.446)

원문의 print는 첫 인자를 출력한 뒤 print(args...)로 나머지 인자를 재귀적으로 전달한다. [U07-019] (B 가변 길이 템플릿, 인쇄 p.446-447)

재귀 호출을 끝내는 print를 뒤에 정의하면 원문 예제는 **컴파일** 오류가 발생하므로 템플릿 함수의 작성 순서에 유의한다. [U07-020] (B 순서를 바꾼다면?, 인쇄 p.448-449)

StrCat의 두 번째 구현은 전체 문자열 길이를 먼저 구해 reserve로 공간을 할당한 뒤 append로 문자열을 붙인다. [U07-021] (B 임의의 개수의 문자열을 합치는 함수, 인쇄 p.451-452)

파라미터 팩에 sizeof...을 사용하면 전체 인자의 개수를 리턴한다. [U07-022] (B sizeof..., 인쇄 p.453)

Fold 식은 **파라미터 팩**의 연산을 표현하며, Fold 식을 쓸 때 반드시 괄호 ()로 감싸야 한다. [U07-023] (B Fold Expression, 인쇄 p.455-456)

TMP의 구조와 타입 표현

템플릿 메타프로그래밍(Template Meta Programming, **TMP**)은 타입을 가지고 **컴파일**타임에 생성되는 코드로 프로그래밍하는 것이다. [U07-024] (B 템플릿 메타프로그래밍(Template Meta Programming - TMP), 인쇄 p.463-464)

Factorial은 재귀적인 템플릿 구조를 사용하고 Factorial<1>의 특수화로 재귀 구조를 끝낸다. [U07-025] (B 템플릿 메타프로그래밍 1부, 인쇄 p.464-465)

GCD는 유클리드 호제법을 템플릿으로 표현하고, Ratio는 분자와 분모를 템플릿 인자로 받아 유리수를 나타낸다. [U07-026] (B 템플릿 메타프로그래밍 1부, 인쇄 p.466-467)

using은 typedef 대신 타입 이름을 정의하는 데 사용할 수 있다. [U07-027] (B 템플릿 메타프로그래밍 1부, 인쇄 p.468)

의존 타입(dependent type)은 템플릿 인자에 따라 달라질 수 있는 타입이며, 원문은 의존하는 식별자가 타입임을 알리기 위해 typename을 붙인다. [U07-028] (B 템플릿 메타프로그래밍 2부 / 지난 강의 생각해보기 문제, 인쇄 p.477-478)

단위 라이브러리와 auto

Dim은 단위 정보를 타입으로 나타내고 quantity는 실제 데이터 q와 차원 정보 dim_type을 둔다. [U07-029] (B 단위(Unit) 라이브러리, 인쇄 p.481-483)

quantity의 +와 -는 같은 단위가 필요하고, *와 /는 다른 차원을 받아 새 차원을 만든다. [U07-030] (B 단위(Unit) 라이브러리, 인쇄 p.483,486)

auto를 사용하면 **컴파일러**가 **초기화**하는 식에서 해당 타입을 **컴파일** 시 추론한다. [U07-031] (B 타입을 알아서 추측해라! - auto 키워드, 인쇄 p.488-489)

동작 방식·절차

강의자료의 함수 템플릿 작성

1. 타입명을 T로 대체한다. [U07-032] (L09 Generic Programming using Template, 슬라이드 16)

2. template <typename T> 또는 template <class T>로 T가 템플릿 인수임을 명시한다. [U07-033] (L09 Generic Programming using Template, 슬라이드 16)

원문 StrCat의 처리 순서

1. GetStringSize로 합쳐질 문자열의 총길이를 계산한다. [U07-034] (B 임의의 개수의 문자열을 합치는 함수, 인쇄 p.452)

2. reserve로 concat_str에 필요한 공간을 미리 할당한다. [U07-035] (B 임의의 개수의 문자열을 합치는 함수, 인쇄 p.452)

3. AppendToString이 append 후 나머지 인자들을 재귀적으로 전달하고 인자가 없는 함수로 종료한다. [U07-036] (B 임의의 개수의 문자열을 합치는 함수, 인쇄 p.452)

주요 문법·표·수식

가변 길이 print: 원문 발췌 (원문 발췌)

```
template <typename T>
void print(T arg) {
  std::cout << arg << std::endl;
}

template <typename T, typename... Types>
void print(T arg, Types... args) {
  std::cout << arg << ", ";
  print(args...);
}
```

[U07-037] (B 가변 길이 템플릿, 인쇄 p.445)

Factorial 템플릿과 특수화: 원문 발췌 (원문 발췌)

```
template <int N>
struct Factorial {
  static const int result = N * Factorial<N - 1>::result;
};

template <>
struct Factorial<1> {
  static const int result = 1;
};
```

[U07-038] (B 템플릿 메타프로그래밍 1부, 인쇄 p.464)

단항 좌측 Fold: 원문 전개: (... + nums) → ((((1 + 4) + 2) + 3) + 10); 원문의 인자 1, 4, 2, 3, 10을 사용한 전개이다. [U07-039] (B Fold Expression, 인쇄 p.455)

강의자료의 Generic Programming 구현 방법

구현	자료의 설명
매크로(#define) [U07-040] (L09 Generic Programming using Macro, 슬라이드 6)	코드의 단순 대체 [U07-041] (L09 Generic Programming using Macro, 슬라이드 6)
템플릿 [U07-042] (L09 Generic Programming using Template, 슬라이드 15)	**컴파일러**가 설계도를 기반으로 함수·**클래스**를 생성하며, 사용하지 않는 경우 생성하지 않는다. [U07-043] (L09 Generic Programming using Template, 슬라이드 15)

핵심 용어 표

용어	원문에 나온 정의·설명
Generic Programming	**Generic Programming**은 타입에 관계없이 동작하는 일반적인 코드를 작성하는 방법이다. [U07-003] (L09 Generic Programming, 슬라이드 5)
템플릿 인스턴스화	**클래스** 템플릿에 인자를 전달하여 실제 코드를 생성하는 것을 **클래스 템플릿 인스턴스화**(class template instantiation)라고 한다. [U07-007] (B C++ 템플릿(template), 인쇄 p.418)
템플릿 특수화	**템플릿 특수화**(template specialization)는 일부 경우를 따로 처리하는 것이며, 전달할 템플릿 인자가 없어도 template<>를 남겨야 한다. [U07-009] (B 템플릿 특수화(template specialization), 인쇄 p.419)
파라미터 팩	템플릿 **파라미터 팩**과 함수 **파라미터 팩**은 각각 0개 이상의 템플릿 인자와 함수 인자를 나타낸다. [U07-018] (B 가변 길이 템플릿, 인쇄 p.446)
TMP	템플릿 메타프로그래밍(Template Meta Programming, **TMP**)은 타입을 가지고 **컴파일**타임에 생성되는 코드로 프로그래밍하는 것이다. [U07-024] (B 템플릿 메타프로그래밍(Template Meta Programming - TMP), 인쇄 p.463-464)
의존 타입(dependent type)	**의존 타입**(dependent type)은 템플릿 인자에 따라 달라질 수 있는 타입이며, 원문은 의존하는 식별자가 타입임을 알리기 위해 typename을 붙인다. [U07-028] (B 템플릿 메타프로그래밍 2부 / 지난 강의 생각해보기 문제, 인쇄 p.477-478)
auto	**auto**를 사용하면 **컴파일러**가 **초기화**하는 식에서 해당 타입을 **컴파일** 시 추론한다. [U07-031] (B 타입을 알아서 추측해라! - auto 키워드, 인쇄 p.488-489)

단원 요약

요약 기준: 편집 요약

템플릿은 원하는 타입을 넣으면 **컴파일러**가 해당 타입으로 코드를 생성하는 틀이다. [U07-005] (B C++ 템플릿(template), 인쇄 p.413,416)

템플릿 특수화(template specialization)는 일부 경우를 따로 처리하는 것이며, 전달할 템플릿 인자가 없어도 template<>를 남겨야 한다. [U07-009] (B 템플릿 특수화(template specialization), 인쇄 p.419)

템플릿 **파라미터 팩**과 함수 **파라미터 팩**은 각각 0개 이상의 템플릿 인자와 함수 인자를 나타낸다. [U07-018] (B 가변 길이 템플릿, 인쇄 p.446)

템플릿 메타프로그래밍(Template Meta Programming, **TMP**)은 타입을 가지고 **컴파일**타임에 생성되는 코드로 프로그래밍하는 것이다. [U07-024] (B 템플릿 메타프로그래밍(Template Meta Programming - TMP), 인쇄 p.463-464)

08 STL 컨테이너·알고리즘·문자열

B PDF 499-617 | L99 PDF 8

다루는 내용

STL의 컨테이너·**반복자**·**알고리즘**을 다룬다. [U08-001] (B C++ 표준 템플릿 라이브러리, 인쇄 p.494)

표준 문자열 라이브러리 string과 **string_view**를 살펴본다. [U08-002] (B C++ 문자열의 모든 것(string과 string_view), 인쇄 p.591)

핵심 개념

STL과 vector

STL은 임의 타입의 **객체**를 보관하는 컨테이너, 원소에 접근하는 **반복자**, **반복자**로 작업을 수행하는 **알고리즘**으로 구성된다. [U08-003] (B C++ 표준 템플릿 라이브러리(Standard Template Library - STL), 인쇄 p.494)

시퀀스 컨테이너는 **객체**를 순차적으로 보관하고, 연관 컨테이너는 키를 바탕으로 대응되는 값을 찾는다. [U08-004] (B C++ STL 컨테이너 - 벡터(std::vector), 인쇄 p.495)

vector는 가변 길이 **배열**이며 원소가 메모리상 연속적으로 저장된다. [U08-005] (B C++ STL 컨테이너 - 벡터(std::vector), 인쇄 p.495)

vector의 임의 위치 접근은 O(1), 맨 뒤 추가·제거는 amortized O(1), 임의 위치 추가·제거는 O(n)으로 정리한다. [U08-006] (B C++ STL 컨테이너 - 벡터(std::vector), 인쇄 p.500)

vector가 할당한 공간을 다 채우면 큰 공간을 새로 할당하고 기존 원소들을 복사한다. [U08-007] (B C++ STL 컨테이너 - 벡터(std::vector), 인쇄 p.498)

반복자와 범위 기반 for

반복자(iterator)는 컨테이너 원소에 접근할 수 있는 **포인터**와 같은 **객체**이다. [U08-008] (B 반복자(iterator), 인쇄 p.500)

begin()은 첫 원소를, end()는 마지막 원소 한 칸 뒤를 가리키며, begin()==end()이면 빈 컨테이너를 나타낸다. [U08-009] (B 반복자(iterator), 인쇄 p.500)

반복자의 * 연산자는 가리키는 원소의 레퍼런스를 반환하고, vector의 insert는 지정 **반복자** 앞에 추가하며 erase는 그 원소를 지운다. [U08-010] (B 반복자(iterator), 인쇄 p.502,504)

원문은 vector에서 erase·insert 후 기존 **반복자**를 계속 사용하는 문제를 다룬다. [U08-011] (B 반복자(iterator), 인쇄 p.504-506)

const_iterator는 가리키는 원소의 값을 바꿀 수 없으며 cbegin·cend로 얻고, 역**반복자**는 뒤에서 앞으로 이동한다. [U08-012] (B 반복자(iterator), 인쇄 p.507-509)

범위 기반 for에서 값으로 받으면 원소가 복사되며, const **auto**&로 받으면 원소를 **상수** 레퍼런스로 접근한다. [U08-013] (B 범위 기반 for 문(range based for loop), 인쇄 p.511,513)

list와 deque

list는 양방향 연결 구조로 []·at를 제공하지 않으며, 원하는 위치를 가리킨 **상태**에서 원소 추가·제거를 O(1)에 수행한다. [U08-014] (B 리스트(list), 인쇄 p.513-514)

list의 **반복자**는 한 칸씩 양방향 이동하는 BidirectionalIterator이고, vector의 **반복자**는 임의 위치에 접근하는 **RandomAccessIterator**이다. [U08-015] (B 리스트(list), 인쇄 p.515)

deque는 양 끝의 추가·제거와 임의 위치 접근을 O(1)에 수행하며, 임의 위치 추가·제거는 O(n)이다. [U08-016] (B 덱(deque - double ended queue), 인쇄 p.517)

deque의 원소는 일정 크기의 블록에 나뉘어 있고, 각 블록의 위치를 보관하기 위한 메모리가 필요하다. [U08-017] (B 덱(deque - double ended queue), 인쇄 p.517-518)

set·map과 중복 키

set은 정렬 **상태**를 유지하며 중복된 원소를 넣지 않고, find는 원소가 있으면 **반복자**, 없으면 end()를 반환한다. [U08-018] (B 셋(set), 인쇄 p.525,528-529)

set에서 두 원소가 같다는 것은 양쪽 방향의 operator< 결과가 모두 false라는 뜻이다. [U08-019] (B 여러분이 만든 클래스 객체를 셋에 넣고 싶을 때, 인쇄 p.532)

set의 비교는 operator< 또는 두 번째 템플릿 인자로 전달하는 비교 함수 **객체**로 지정할 수 있다. [U08-020] (B 여러분이 만든 클래스 객체를 셋에 넣고 싶을 때, 인쇄 p.530,535-536)

map은 키와 대응하는 값을 보관하며, **반복자**의 first는 키이고 second는 값이다. [U08-021] (B 맵(map), 인쇄 p.536,538)

map의 []는 없는 키를 참조하면 값의 디폴트 **생성자**를 호출하여 원소를 추가하고, 기존 키이면 대응 값을 바꿀 수 있다. [U08-022] (B 맵(map), 인쇄 p.538,540,542)

multiset과 multimap은 중복을 허용하고, multimap은 []를 제공하지 않으며 equal_range로 해당 키의 원소 범위를 얻는다. [U08-023] (B 멀티셋(multiset)과 멀티맵(multimap), 인쇄 p.542,544-545)

unordered 컨테이너와 해시

unordered_set·unordered_map은 원소가 정렬되지 않으며, 해시 함수를 사용해 원소를 저장할 위치를 정한다. [U08-024] (B 정렬되지 않은 셋과 맵(unordered_set, unordered_map), 인쇄 p.546,548)

해시 함수는 임의 크기의 데이터를 고정 크기의 데이터에 대응시키고, 서로 다른 원소가 같은 해시값을 가지면 **해시 충돌**이라 한다. [U08-025] (B 해시 함수(Hash function), 인쇄 p.548)

unordered_set·unordered_map의 삽입·탐색은 평균 O(1), 최악 O(N)이고, rehash는 모든 원소를 다시 insert하여 O(N)이 걸린다. [U08-026] (B 해시 함수(Hash function), 인쇄 p.549)

사용자 **클래스**의 unordered 컨테이너에는 해시 함수와 **충돌** 시 비교할 operator==가 필요하다. [U08-027] (B 내가 만든 클래스를 unordered_set/unordered_map의 원소로 넣고 싶을 때, 인쇄 p.551)

알고리즘과 서술자

알고리즘은 보통 시작 **반복자**와 끝 바로 뒤 **반복자**를 받으며, **서술자**(Predicate)에는 보통 bool을 반환하는 함수 **객체**를 전달한다. [U08-028] (B C++ 표준 알고리즘 라이브러리, 인쇄 p.556)

sort는 기본적으로 오름차순으로 정렬하며 **RandomAccessIterator**를 요구하고, 비교 방법을 추가 인자로 받을 수 있다. [U08-029] (B 정렬(sort, stable_sort, partial_sort), 인쇄 p.558)

stable_sort는 크기가 같은 원소들의 기존 순서를 보존하며, partial_sort는 전체 원소 중 앞의 지정 구간을 작은 원소 순으로 정렬한다. [U08-030] (B 정렬(sort, stable_sort, partial_sort), 인쇄 p.556,561)

remove는 원소를 이동할 뿐 실제 삭제하지 않으므로, 컨테이너에서 지우려면 erase를 호출한다. [U08-031] (B 원소 제거(remove, remove_if), 인쇄 p.568)

remove_if는 조건을 나타내는 함수 **객체**를 받으며, 원문은 그 **객체**가 여러 번 복사될 수 있어 내부 **상태**에 의존하는 결과를 주의하도록 한다. [U08-032] (B remove_if에 조건 추가하기, 인쇄 p.569,572-573)

람다·변환·탐색

람다는 이름 없는 함수 **객체**를 만들며, 캡쳐 목록으로 외부 **변수**를 사용하고 &로 참조를 캡쳐한다. [U08-033] (B 람다 함수(lambda function), 인쇄 p.575-576,579)

transform은 각 원소에 함수를 적용한 결과를 결과 시작 **반복자**부터 저장하며, 결과를 담는 공간이 충분해야 한다. [U08-034] (B 원소 수정하기(transform), 인쇄 p.582)

find는 값이 같은 첫 원소를 찾고, find_if는 조건이 참인 원소를 찾는다. [U08-035] (B 원소를 탐색하는 함수(find, find_if, any_of, all_of 등등), 인쇄 p.585,588)

any_of는 범위에서 하나라도 조건을 충족하면 true, all_of는 모든 원소가 조건을 충족해야 true를 반환한다. [U08-036] (B 원소를 탐색하는 함수(find, find_if, any_of, all_of 등등), 인쇄 p.589)

문자열·인코딩·string_view

std::string은 basic_string<char>의 인스턴스화 버전이며, basic_string은 문자를 연속 저장하고 Traits는 기본 문자열 연산을 정의한다. [U08-037] (B basic_string, 인쇄 p.495,591-592)

짧은 문자열 최적화(**SSO**)는 짧은 문자열 데이터를 따로 할당하는 대신 **객체** 자체에 저장하는 방식이다. [U08-038] (B 짧은 문자열 최적화(SSO), 인쇄 p.595)

원문의 "hello"s는 string으로 추론되는 리터럴이며, Raw string literal은 이스케이프 없이 문자들을 그대로 넣고 구분자를 지정할 수 있다. [U08-039] (B 리터럴 연산자 / 그 외의 여러 가지 리터럴 정의 방법, 인쇄 p.597-599)

유니코드는 문자에 고유 값을 부여하며, UTF-8은 1~4바이트, UTF-16은 2 또는 4바이트, UTF-32는 4바이트로 문자를 표현한다. [U08-040] (B C++에서 한글 다루기, 인쇄 p.600)

std::string의 size()는 실제 문자 수가 아니라 char 원소 수를 반환하므로, UTF-8 문자열의 문자 수와 다를 수 있다. [U08-041] (B UTF-8 인코딩, 인쇄 p.602)

string_view는 문자열을 소유하지 않고 참조해서 읽으며, 가리키는 문자열이 소멸한 뒤 사용하면 Undefined behavior가 발생한다. [U08-042] (B string_view / 소유하지 않고 읽기만 한다!, 인쇄 p.609)

string의 substr은 새 부분 문자열을 만들어 O(n), **string_view**의 substr은 다른 view를 만들어 O(1)에 수행된다. [U08-043] (B string_view / 소유하지 않고 읽기만 한다!, 인쇄 p.609)

동작 방식·절차

원문의 erase-remove 처리

1. remove가 남길 원소를 앞으로 옮긴 뒤 그 끝을 가리키는 **반복자**를 반환한다. [U08-044] (B 원소 제거(remove, remove_if), 인쇄 p.567-568)

2. remove 결과부터 기존 end까지 erase하여 실제 원소를 지운다. [U08-045] (B 원소 제거(remove, remove_if), 인쇄 p.568)

multimap의 같은 키 원소 순회

1. equal_range(key)로 시작과 끝 바로 뒤 **반복자**를 담은 pair를 얻는다. [U08-046] (B 멀티셋(multiset)과 멀티맵(multimap), 인쇄 p.545)

2. range.first부터 range.second 직전까지 **반복자**를 증가시키며 각 원소를 읽는다. [U08-047] (B 멀티셋(multiset)과 멀티맵(multimap), 인쇄 p.546)

주요 문법·표·수식

반복자 범위 출력: 원문 발췌 (원문 발췌)

```
template <typename Iter>
void print(Iter begin, Iter end) {
  while (begin != end) {
    std::cout << *begin << " ";
    begin++;
  }
  std::cout << std::endl;
}
```

[U08-048] (B 정렬(sort, stable_sort, partial_sort), 인쇄 p.557)

람다와 remove_if: 원문 발췌 (원문 발췌)

```
std::cout << "벡터에서홀수인원소제거---" << std::endl;
vec.erase(std::remove_if(vec.begin(), vec.end(),
                         [](int i) -> bool { return i % 2 == 1; }),
          vec.end());
print(vec.begin(), vec.end());
```

[U08-049] (B 람다 함수(lambda function), 인쇄 p.575)

partial_sort 복잡도: O(N log M); N: 전체 원소 수, M: 정렬하려는 부분의 크기. [U08-050] (B 정렬(sort, stable_sort, partial_sort), 인쇄 p.561)

시퀀스 컨테이너의 원문 비교

컨테이너	특징·연산
vector [U08-051] (B 벡터(std::vector), 인쇄 p.500)	연속 저장; 임의 접근 O(1); 임의 위치 추가·제거 O(n) [U08-052] (B 벡터(std::vector), 인쇄 p.495,500)
list [U08-053] (B 리스트(list), 인쇄 p.514)	양방향 연결; []·at 없음; 위치를 찾은 뒤 추가·제거 O(1) [U08-054] (B 리스트(list), 인쇄 p.513-514)
deque [U08-055] (B 덱(deque), 인쇄 p.517)	양 끝 추가·제거 O(1); 임의 접근 O(1); 임의 위치 추가·제거 O(n) [U08-056] (B 덱(deque), 인쇄 p.517)

핵심 용어 표

용어	원문에 나온 정의·설명
STL	**STL**은 임의 타입의 **객체**를 보관하는 컨테이너, 원소에 접근하는 **반복자**, **반복자**로 작업을 수행하는 **알고리즘**으로 구성된다. [U08-003] (B C++ 표준 템플릿 라이브러리(Standard Template Library - STL), 인쇄 p.494)
반복자(iterator)	**반복자**(iterator)는 컨테이너 원소에 접근할 수 있는 **포인터**와 같은 **객체**이다. [U08-008] (B 반복자(iterator), 인쇄 p.500)
RandomAccessIterator	list의 **반복자**는 한 칸씩 양방향 이동하는 BidirectionalIterator이고, vector의 **반복자**는 임의 위치에 접근하는 **RandomAccessIterator**이다. [U08-015] (B 리스트(list), 인쇄 p.515)
해시 충돌(hash collision)	해시 함수는 임의 크기의 데이터를 고정 크기의 데이터에 대응시키고, 서로 다른 원소가 같은 해시값을 가지면 **해시 충돌**이라 한다. [U08-025] (B 해시 함수(Hash function), 인쇄 p.548)
서술자(Predicate)	**알고리즘**은 보통 시작 **반복자**와 끝 바로 뒤 **반복자**를 받으며, **서술자**(Predicate)에는 보통 bool을 반환하는 함수 **객체**를 전달한다. [U08-028] (B C++ 표준 알고리즘 라이브러리, 인쇄 p.556)
SSO	짧은 문자열 최적화(**SSO**)는 짧은 문자열 데이터를 따로 할당하는 대신 **객체** 자체에 저장하는 방식이다. [U08-038] (B 짧은 문자열 최적화(SSO), 인쇄 p.595)
유니코드(Unicode)	**유니코드**는 문자에 고유 값을 부여하며, UTF-8은 1~4바이트, UTF-16은 2 또는 4바이트, UTF-32는 4바이트로 문자를 표현한다. [U08-040] (B C++에서 한글 다루기, 인쇄 p.600)
string_view	**string_view**는 문자열을 소유하지 않고 참조해서 읽으며, 가리키는 문자열이 소멸한 뒤 사용하면 Undefined behavior가 발생한다. [U08-042] (B string_view / 소유하지 않고 읽기만 한다!, 인쇄 p.609)

단원 요약

요약 기준: 원문 Summary

STL은 컨테이너·**알고리즘**·**반복자**로 구성되며 **반복자**는 **역참조**를 통해 값에 접근한다. [U08-057] (L99 Summary of 2023-2 C++ Programming Class / STL, 슬라이드 8)

std::string은 basic_string의 char 인자 인스턴스화 버전이며 char_traits로 원하는 기능을 가진 문자열을 만들 수 있다. [U08-058] (B 뭘 배웠지?, 인쇄 p.612)

문자 인코딩 방식에 따라 문자별로 인코딩되는 길이가 다를 수 있다. [U08-059] (B 뭘 배웠지?, 인쇄 p.612)

string_view로 불필요한 복사를 막고 const char*과 const string& 사이의 처리를 할 수 있다. [U08-060] (B 뭘 배웠지?, 인쇄 p.612)

09 예외 처리

B PDF 618-634

다루는 내용

C++에서 **예외**를 발생시키고 처리하는 방식을 다룬다. [U09-001] (B C++에서의 예외 처리, 인쇄 p.613)

스택 풀기, 여러 종류의 **예외**, 모든 **예외** 받기와 **noexcept**를 다룬다. [U09-002] (B C++에서의 예외 처리, 인쇄 p.618,621,626-627)

핵심 개념

예외와 기존 처리 방식

예외(exception)는 정상적인 상황에서 벗어난 **예외**적인 상황이다. [U09-003] (B 예외란?, 인쇄 p.613)

원문은 범위를 벗어난 vector 접근과 큰 메모리 할당을 실행 중 오류가 발생할 수 있는 예로 든다. [U09-004] (B 예외란?, 인쇄 p.613)

기존 방식은 작업 후 결과값을 확인하며, 깊은 함수에서 발생한 문제를 바깥에서 확인하려면 각 함수가 처리 결과를 전달해야 한다. [U09-005] (B 기존의 예외 처리 방식, 인쇄 p.614-615)

throw와 표준 예외 객체

C++은 **throw**로 **예외**가 발생했다는 사실을 명시적으로 나타낼 수 있다. [U09-006] (B 예외 발생시키기 - throw, 인쇄 p.615)

throw 다음에는 **예외**로 전달할 **객체**를 쓰며, 원문은 std::out_of_range **객체**를 던진다. [U09-007] (B 예외 발생시키기 - throw, 인쇄 p.616)

표준 라이브러리에는 out_of_range·overflow_error·length_error·runtime_error 등의 **예외**가 정의되어 있다. [U09-008] (B 예외 발생시키기 - throw, 인쇄 p.616)

throw를 실행하면 그 위치에서 즉시 함수를 빠져나가 **예외** 처리 부분으로 이동하므로 아래 문장들은 실행하지 않는다. [U09-009] (B 예외 발생시키기 - throw, 인쇄 p.616)

예외를 처리하는 부분까지 함수를 빠져나가는 동안 **스택**에 생성된 **객체**들의 **소멸자**를 호출한다. [U09-010] (B 예외 발생시키기 - throw, 인쇄 p.616)

try·catch와 예외 내용

try 안에는 **예외**가 발생할 만한 코드를 넣으며, **예외**가 없으면 **catch**는 무시한다. [U09-011] (B 예외 처리하기 - try와 catch, 인쇄 p.618)

catch는 던진 **예외**를 받는 부분이며, 괄호 안에 정의된 **예외**의 꼴에 맞는 **객체**를 받는다. [U09-012] (B 예외 처리하기 - try와 catch, 인쇄 p.618)

out_of_range의 **what**()으로 **예외**에 관해 저장된 문자열을 확인할 수 있다. [U09-013] (B 예외 처리하기 - try와 catch, 인쇄 p.618)

스택 풀기와 생성자

스택 풀기(stack unwinding)는 **catch**로 이동하면서 **스택**상에 정의된 **객체**들을 소멸시키는 과정이다. [U09-014] (B 스택 풀기(stack unwinding), 인쇄 p.621)

원문의 호출 예에서는 func3에서 던진 **예외**가 func2·func1을 지나 main의 **catch**까지 전달된다. [U09-015] (B 스택 풀기(stack unwinding), 인쇄 p.619-620)

생성자에서 **예외**가 발생하면 해당 **객체**의 **소멸자**가 호출되지 않으므로, 원문은 이전에 획득한 **자원**을 **catch**에서 해제하도록 주의시킨다. [U09-016] (B 스택 풀기(stack unwinding), 인쇄 p.621)

여러 종류와 catch 순서

하나의 **try**에 여러 **catch**를 붙여 char·int·string·const char* 등 여러 종류의 **예외**를 받을 수 있다. [U09-017] (B 여러 종류의 예외 받기, 인쇄 p.621-623)

catch는 가장 먼저 대입될 수 있는 **객체**를 받으므로, Parent **catch**가 앞에 있으면 Child **예외**도 먼저 받는다. [U09-018] (B 여러 종류의 예외 받기, 인쇄 p.624)

원문은 파생 **클래스 예외**를 받는 **catch**를 기반 **클래스 catch**보다 앞에 두어 각각 처리한다. [U09-019] (B 여러 종류의 예외 받기, 인쇄 p.624-625)

모든 예외와 noexcept

던진 **예외**를 받는 **catch**가 없으면 원문 예제는 프로그램이 비정상 종료된다. [U09-020] (B 모든 예외 받기, 인쇄 p.626)

catch(...)는 **try** 안에서 발생한 모든 **예외**를 받지만 특정 타입의 **객체**로 받을 수는 없다. [U09-021] (B 모든 예외 받기, 인쇄 p.627)

noexcept는 함수가 **예외**를 발생시키지 않음을 명시하는 키워드이다. [U09-022] (B 예외를 발생시키지 않는 함수 - noexcept, 인쇄 p.627)

noexcept 함수가 실제 **예외**를 발생시키면 **예외**가 제대로 처리되지 않고 프로그램이 종료된다. [U09-023] (B 예외를 발생시키지 않는 함수 - noexcept, 인쇄 p.628)

원문은 C++ 11부터 **소멸자**가 기본적으로 **noexcept**이므로 **소멸자**에서 **예외**를 던지지 않도록 주의시킨다. [U09-024] (B 예외를 발생시키지 않는 함수 - noexcept, 인쇄 p.629)

동작 방식·절차

원문의 Vector 예외 처리

1. **try** 안에서 vec.at(index)를 호출한다. [U09-025] (B 예외 처리하기 - try와 catch, 인쇄 p.617)

2. index가 범위를 벗어나면 std::out_of_range를 **throw**한다. [U09-026] (B 예외 처리하기 - try와 catch, 인쇄 p.617)

3. **catch**(std::out_of_range& e)가 받아 e.**what**()을 출력한다. [U09-027] (B 예외 처리하기 - try와 catch, 인쇄 p.617)

원문의 스택 풀기

1. func3에서 **예외**를 던진다. [U09-028] (B 스택 풀기(stack unwinding), 인쇄 p.619)

2. **catch**가 없는 func2·func1을 빠져나가면서 각 함수의 **객체**를 소멸시킨다. [U09-029] (B 스택 풀기(stack unwinding), 인쇄 p.619-621)

3. main에 있는 **catch**에서 전달된 **예외**를 처리한다. [U09-030] (B 스택 풀기(stack unwinding), 인쇄 p.620)

주요 문법·표·수식

out_of_range 처리: 원문 발췌 (원문 발췌)

```
try {
  data = vec.at(index);
} catch (std::out_of_range& e) {
  std::cout << "예외발생! " << e.what() << std::endl;
}
```

[U09-031] (B 예외 처리하기 - try와 catch, 인쇄 p.617)

예외 처리 구문의 역할

구문	원문의 역할
throw [U09-032] (B 예외 발생시키기 - throw, 인쇄 p.616)	**예외 객체**를 던지고 처리 부분으로 이동한다. [U09-033] (B 예외 발생시키기 - throw, 인쇄 p.616)
try [U09-034] (B 예외 처리하기 - try와 catch, 인쇄 p.618)	**try** 안에는 **예외**가 발생할 만한 코드를 넣으며, **예외**가 없으면 **catch**는 무시한다. [U09-011] (B 예외 처리하기 - try와 catch, 인쇄 p.618)
catch [U09-035] (B 예외 처리하기 - try와 catch, 인쇄 p.618)	**catch**는 던진 **예외**를 받는 부분이며, 괄호 안에 정의된 **예외**의 꼴에 맞는 **객체**를 받는다. [U09-012] (B 예외 처리하기 - try와 catch, 인쇄 p.618)

핵심 용어 표

용어	원문에 나온 정의·설명
예외(exception)	**예외**(exception)는 정상적인 상황에서 벗어난 **예외**적인 상황이다. [U09-003] (B 예외란?, 인쇄 p.613)
throw	**throw** 다음에는 **예외**로 전달할 **객체**를 쓰며, 원문은 std::out_of_range **객체**를 던진다. [U09-007] (B 예외 발생시키기 - throw, 인쇄 p.616)
try	**try** 안에는 **예외**가 발생할 만한 코드를 넣으며, **예외**가 없으면 **catch**는 무시한다. [U09-011] (B 예외 처리하기 - try와 catch, 인쇄 p.618)
catch	**catch**는 던진 **예외**를 받는 부분이며, 괄호 안에 정의된 **예외**의 꼴에 맞는 **객체**를 받는다. [U09-012] (B 예외 처리하기 - try와 catch, 인쇄 p.618)
what()	out_of_range의 **what**()으로 **예외**에 관해 저장된 문자열을 확인할 수 있다. [U09-013] (B 예외 처리하기 - try와 catch, 인쇄 p.618)
스택 풀기(stack unwinding)	**스택 풀기**(stack unwinding)는 **catch**로 이동하면서 **스택**상에 정의된 **객체**들을 소멸시키는 과정이다. [U09-014] (B 스택 풀기(stack unwinding), 인쇄 p.621)
noexcept	**noexcept**는 함수가 **예외**를 발생시키지 않음을 명시하는 키워드이다. [U09-022] (B 예외를 발생시키지 않는 함수 - noexcept, 인쇄 p.627)

단원 요약

요약 기준: 편집 요약

C++은 **throw**로 **예외**가 발생했다는 사실을 명시적으로 나타낼 수 있다. [U09-006] (B 예외 발생시키기 - throw, 인쇄 p.615)

catch는 던진 **예외**를 받는 부분이며, 괄호 안에 정의된 **예외**의 꼴에 맞는 **객체**를 받는다. [U09-012] (B 예외 처리하기 - try와 catch, 인쇄 p.618)

스택 풀기(stack unwinding)는 **catch**로 이동하면서 **스택**상에 정의된 **객체**들을 소멸시키는 과정이다. [U09-014] (B 스택 풀기(stack unwinding), 인쇄 p.621)

noexcept 함수가 실제 **예외**를 발생시키면 **예외**가 제대로 처리되지 않고 프로그램이 종료된다. [U09-023] (B 예외를 발생시키지 않는 함수 - noexcept, 인쇄 p.628)

10 우측값·이동·완벽한 전달

B PDF 635-675

다루는 내용

우측값 레퍼런스와 이동 **생성자**를 다룬다. [U10-001] (B 우측값 레퍼런스와 이동 생성자, 인쇄 p.630)

좌측값을 이동할 수 있게 하는 move와 인자 전달을 위한 forward를 다룬다. [U10-002] (B move 문법과 완벽한 전달, 인쇄 p.649,655,669)

핵심 개념

복사 생략과 값의 종류

복사 생략(copy elision)은 **컴파일러**가 불필요한 복사 생성을 생략하는 작업이다. [U10-003] (B 복사 생략(Copy Elision), 인쇄 p.631)

원문의 MyString 덧셈 예는 반환된 임시 문자열을 복사할 때 추가적인 **자원**이 소모되는 문제를 다룬다. [U10-004] (B 복사 생략(Copy Elision), 인쇄 p.634-635)

nullptr는 원문에서 **포인터** 주소값 0을 명확히 나타내며 기존 NULL을 대체하는 키워드로 설명한다. [U10-005] (B 복사 생략(Copy Elision), 인쇄 p.634)

원문은 주소값을 취할 수 있는 값을 좌측값(lvalue), 주소값을 취할 수 없는 값을 우측값(rvalue)으로 설명한다. [U10-006] (B 좌측값(lvalue)과 우측값(rvalue), 인쇄 p.635)

& 하나로 정의하는 좌측값 레퍼런스(lvalue reference)는 좌측값을 참조하며, 좌측값 레퍼런스 자체도 좌측값이다. [U10-007] (B 좌측값(lvalue)과 우측값(rvalue), 인쇄 p.636)

const T&는 **예외**적으로 우측값도 참조할 수 있지만 그 임시 **객체**의 값을 변경할 수 없다. [U10-008] (B 그렇다면 이동은 어떻게?, 인쇄 p.638)

우측값 레퍼런스와 이동 생성자

우측값 레퍼런스는 &&로 정의하며, 원문의 MyString(MyString&& str)은 우측값을 인자로 받는다. [U10-009] (B 우측값 레퍼런스, 인쇄 p.641)

우측값 레퍼런스 매개**변수** str은 이름이 있으므로 **표현식** 자체는 좌측값이다. [U10-010] (B 우측값 레퍼런스, 인쇄 p.641)

원문의 이동 **생성자**는 문자열 전체를 새로 복사하는 대신 string_content의 주소를 새 **객체**에 전달한다. [U10-011] (B 우측값 레퍼런스, 인쇄 p.642)

이동한 원본의 string_content를 **nullptr**로 바꾸어 원본 **소멸자**가 새 **객체**도 가리키는 메모리를 해제하지 않게 한다. [U10-012] (B 우측값 레퍼런스, 인쇄 p.642)

원문의 일반적인 int&& 예제는 우측값을 받을 수 있고 좌측값 a를 직접 받을 수 없다. [U10-013] (B 우측값 레퍼런스, 인쇄 p.642)

우측값 레퍼런스는 참조하는 임시 **객체**가 소멸되지 않도록 붙들 수 있다. [U10-014] (B 우측값 레퍼런스, 인쇄 p.642)

원문 vector 예제는 이동 **생성자**에 **noexcept**를 명시하면 재할당할 때 이동 **생성자**를 사용함을 보여준다. [U10-015] (B 이동 생성자 작성 시 주의할 점, 인쇄 p.643,645,647)

move와 이동 대입

std::move는 <utility>에서 제공되며, 이동 자체를 수행하지 않고 받은 **객체**를 우측값으로 변환한다. [U10-016] (B move 함수(move semantics), 인쇄 p.655-657)

실제 이동은 우측값을 받는 이동 **생성자**나 이동 **대입 연산자**가 호출될 때 수행된다. [U10-017] (B move 함수(move semantics), 인쇄 p.661)

원문의 MyString 이동 대입은 주소·길이·용량을 전달하고 원본의 주소를 **nullptr**로, 길이와 용량을 0으로 바꾼다. [U10-018] (B move 함수(move semantics), 인쇄 p.658-660)

이동 **대입 연산자**를 정의하지 않은 원문 예제에서는 move를 써도 일반적인 **대입 연산자**의 복사가 수행된다. [U10-019] (B move 함수(move semantics), 인쇄 p.661)

원문의 my_swap은 a로 tmp를 이동 생성하고 b를 a로, tmp를 b로 이동 대입한다. [U10-020] (B move 함수(move semantics), 인쇄 p.660)

wrapper와 완벽한 전달

완벽한 전달(perfect forwarding)은 wrapper가 받은 인자를 원래 호출과 같은 형태로 다른 함수에 전달하는 문제를 다룬다. [U10-021] (B 완벽한 전달(perfect forwarding), 인쇄 p.661-663,670)

emplace_back은 **생성자** 인자를 직접 받아 내부에서 **객체**를 생성하여 vector 뒤에 추가한다. [U10-022] (B 완벽한 전달(perfect forwarding), 인쇄 p.662)

wrapper(T u)는 원문의 좌측값·**상수 좌측값**·**우측값** 호출에서 모두 좌측값 레퍼런스 g를 호출하여 원래 형태를 보존하지 못한다. [U10-023] (B 완벽한 전달(perfect forwarding), 인쇄 p.663)

wrapper(T& u)는 원문의 우측값 A()를 받지 못하고, const T&와의 오버로드도 우측값을 **상수** 좌측값 레퍼런스로 전달한다. [U10-024] (B 완벽한 전달(perfect forwarding), 인쇄 p.664-666)

보편적 레퍼런스와 forward

원문은 템플릿 인자 T에 대한 T&&를 **보편적 레퍼런스**(Universal reference)라 부르며 좌측값과 우측값을 모두 받을 수 있다고 설명한다. [U10-025] (B 보편적 레퍼런스(Universal reference), 인쇄 p.668)

레퍼런스 겹침 규칙에서 &와 겹치면 &가 되고, &&와 &&가 겹친 경우만 &&가 된다. [U10-026] (B 보편적 레퍼런스(Universal reference), 인쇄 p.669)

원문의 wrapper에서는 a에 대해 T가 A&, ca에 대해 const A&, A()에 대해 A로 추론된다. [U10-027] (B 보편적 레퍼런스(Universal reference), 인쇄 p.669)

std::forward<T>(u)는 원래 우측값인 인자는 우측값으로, 좌측값인 인자는 좌측값으로 전달하여 원문과 같은 g 오버로드가 호출되게 한다. [U10-028] (B 보편적 레퍼런스(Universal reference), 인쇄 p.669-670)

동작 방식·절차

원문 MyString의 이동 생성

1. 원본의 길이·문자열 주소·용량을 새 **객체**에 복사한다. [U10-029] (B 우측값 레퍼런스, 인쇄 p.640)

2. 원본 문자열 주소를 **nullptr**로 바꾼다. [U10-030] (B 우측값 레퍼런스, 인쇄 p.640)

3. 소멸자는 문자열 주소가 **nullptr**가 아닐 때에만 delete[]를 수행한다. [U10-031] (B 우측값 레퍼런스, 인쇄 p.642)

원문의 완벽한 전달 wrapper

1. T&&로 인자를 받아 원래 인자의 형태에 맞게 T를 추론한다. [U10-032] (B 보편적 레퍼런스(Universal reference), 인쇄 p.668-669)

2. 이름 있는 매개**변수** u를 **std::forward**<T>(u)를 거쳐 g에 전달한다. [U10-033] (B 보편적 레퍼런스(Universal reference), 인쇄 p.667,669)

3. 원래 호출과 wrapper 호출이 같은 g 오버로드를 선택한다. [U10-034] (B 보편적 레퍼런스(Universal reference), 인쇄 p.670)

주요 문법·표·수식

MyString 이동 생성자: 원문 발췌 (원문 발췌)

```
MyString::MyString(MyString &&str) {
  std::cout << "이동생성자호출!" << std::endl;
  string_length = str.string_length;
  string_content = str.string_content;
  memory_capacity = str.memory_capacity;

  // 임시객체소멸시에메모리를해제하지
  // 못하게한다.
  str.string_content = nullptr;
}
```

[U10-035] (B 우측값 레퍼런스, 인쇄 p.640)

move를 사용하는 my_swap: 원문 발췌 (원문 발췌)

```
template <typename T>
void my_swap(T &a, T &b) {
  T tmp(std::move(a));
  a = std::move(b);
  b = std::move(tmp);
}
```

[U10-036] (B move 함수(move semantics), 인쇄 p.659)

원문의 레퍼런스 겹침 규칙

겹치는 형태	결과
int& & [U10-037] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)	int& [U10-038] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)
int& && [U10-039] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)	int& [U10-038] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)
int&& & [U10-040] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)	int& [U10-038] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)
int&& && [U10-041] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)	int&& [U10-042] (B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)

핵심 용어 표

용어	원문에 나온 정의·설명
복사 생략(copy elision)	**복사 생략**(copy elision)은 **컴파일러**가 불필요한 복사 생성을 생략하는 작업이다. [U10-003] (B 복사 생략(Copy Elision), 인쇄 p.631)
nullptr	**nullptr**는 원문에서 **포인터** 주소값 0을 명확히 나타내며 기존 NULL을 대체하는 키워드로 설명한다. [U10-005] (B 복사 생략(Copy Elision), 인쇄 p.634)
좌측값·우측값	원문은 주소값을 취할 수 있는 값을 좌측값(lvalue), 주소값을 취할 수 없는 값을 우측값(rvalue)으로 설명한다. [U10-006] (B 좌측값(lvalue)과 우측값(rvalue), 인쇄 p.635)
우측값 레퍼런스	**우측값 레퍼런스**는 &&로 정의하며, 원문의 MyString(MyString&& str)은 우측값을 인자로 받는다. [U10-009] (B 우측값 레퍼런스, 인쇄 p.641)
std::move	**std::move**는 <utility>에서 제공되며, 이동 자체를 수행하지 않고 받은 **객체**를 우측값으로 변환한다. [U10-016] (B move 함수(move semantics), 인쇄 p.655-657)
완벽한 전달(perfect forwarding)	**완벽한 전달**(perfect forwarding)은 wrapper가 받은 인자를 원래 호출과 같은 형태로 다른 함수에 전달하는 문제를 다룬다. [U10-021] (B 완벽한 전달(perfect forwarding), 인쇄 p.661-663,670)
보편적 레퍼런스(Universal reference)	원문은 템플릿 인자 T에 대한 T&&를 **보편적 레퍼런스**(Universal reference)라 부르며 좌측값과 우측값을 모두 받을 수 있다고 설명한다. [U10-025] (B 보편적 레퍼런스(Universal reference), 인쇄 p.668)
std::forward	**std::forward**<T>(u)는 원래 우측값인 인자는 우측값으로, 좌측값인 인자는 좌측값으로 전달하여 원문과 같은 g 오버로드가 호출되게 한다. [U10-028] (B 보편적 레퍼런스(Universal reference), 인쇄 p.669-670)

단원 요약

요약 기준: 편집 요약

복사 생략(copy elision)은 **컴파일러**가 불필요한 복사 생성을 생략하는 작업이다. [U10-003] (B 복사 생략(Copy Elision), 인쇄 p.631)

원문의 이동 **생성자**는 문자열 전체를 새로 복사하는 대신 string_content의 주소를 새 **객체**에 전달한다. [U10-011] (B 우측값 레퍼런스, 인쇄 p.642)

std::move는 <utility>에서 제공되며, 이동 자체를 수행하지 않고 받은 **객체**를 우측값으로 변환한다. [U10-016] (B move 함수(move semantics), 인쇄 p.655-657)

std::forward<T>(u)는 원래 우측값인 인자는 우측값으로, 좌측값인 인자는 좌측값으로 전달하여 원문과 같은 g 오버로드가 호출되게 한다. [U10-028] (B 보편적 레퍼런스(Universal reference), 인쇄 p.669-670)

11 스마트 포인터·함수 객체

B PDF 676-723

다루는 내용

C++의 **자원** 관리와 **unique_ptr·shared_ptr·weak_ptr**를 다룬다. [U11-001] (B 스마트 포인터, 인쇄 p.671,674,703)

호출 가능한 모든 것을 포괄하는 **Callable**과 function·mem_fn·bind를 다룬다. [U11-002] (B 함수 객체, 인쇄 p.705,718)

핵심 개념

자원 관리와 RAII

원문은 할당한 메모리와 연 파일을 **자원**(resource)으로 들며, 사용이 끝난 **자원**은 반환해야 한다고 설명한다. [U11-003] (B 자원(resource) 관리의 중요성, 인쇄 p.671)

메모리를 사용한 뒤 해제하지 않는 경우를 메모리 누수(memory leak)라고 한다. [U11-004] (B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.674)

예외로 인해 delete가 있는 문장까지 실행되지 않으면 **자원** 해제를 놓칠 수 있다. [U11-005] (B 자원(resource) 관리의 중요성, 인쇄 p.673-674)

RAII(Resource Acquisition Is Initialization)는 **자원** 관리를 **스택**에 할당한 **객체**를 통해 수행하는 방식이다. [U11-006] (B Resource Acquisition Is Initialization - RAII, 인쇄 p.674)

스마트 **포인터**(smart pointer)는 자신이 소멸할 때 가리키는 **자원**도 해제하도록 하는 **포인터 객체**이다. [U11-007] (B Resource Acquisition Is Initialization - RAII, 인쇄 p.674)

double free는 이미 소멸한 **객체**를 다시 소멸시켜 발생하는 버그이다. [U11-008] (B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675)

unique_ptr의 소유권·이동·생성

unique_ptr는 특정 **객체**의 유일한 소유권을 나타내며, 자신이 소멸할 때 가리키던 **객체**도 소멸한다. [U11-009] (B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675,682)

unique_ptr는 -> 연산자를 오버로드하여 일반 **포인터**처럼 사용할 수 있다. [U11-010] (B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.676)

= delete는 함수 사용을 명시적으로 금지하며, **unique_ptr**의 **복사 생성자**는 유일한 소유권을 위해 삭제되어 있다. [U11-011] (B 삭제된 함수, 인쇄 p.678)

unique_ptr는 이동 생성이 가능하며, **std::move**로 소유권을 옮기면 원래 **포인터**는 아무것도 가리키지 않는다. [U11-012] (B unique_ptr 소유권 이전하기, 인쇄 p.679-680)

get은 실제 **객체**의 주소를 반환하며, 원문은 소유권을 옮기지 않고 잠시 접근할 함수에는 그 주소를 전달한다. [U11-013] (B unique_ptr를 함수 인자로 전달하기, 인쇄 p.682)

make_unique는 **클래스 생성자**에 인자를 완벽하게 전달하여 **unique_ptr**를 생성한다. [U11-014] (B unique_ptr를 쉽게 생성하기, 인쇄 p.683)

unique_ptr를 vector에 넣는 원문 예는 push_back(**std::move**(pa)) 또는 내부에서 직접 생성하는 emplace_back을 사용한다. [U11-015] (B unique_ptr를 원소로 가지는 컨테이너, 인쇄 p.685-687)

shared_ptr와 제어 블록

shared_ptr는 여러 스마트 **포인터**가 같은 **객체**를 소유할 수 있게 하며, 참조 개수가 0이 되면 **객체**를 해제한다. [U11-016] (B shared_ptr, 인쇄 p.688-689)

use_count는 현재 **shared_ptr**의 참조 개수를 알려준다. [U11-017] (B shared_ptr, 인쇄 p.691)

shared_ptr들은 동적으로 할당한 **제어 블록**(control block)을 공유하여 참조 개수 등의 정보를 공유한다. [U11-018] (B shared_ptr, 인쇄 p.692)

make_shared는 **객체**와 **shared_ptr**의 **제어 블록**을 한 번에 동적 할당하여 **shared_ptr**를 반환한다. [U11-019] (B make_shared로 생성하자, 인쇄 p.693)

같은 일반 **포인터** 주소로 **shared_ptr**를 각각 생성하면 **제어 블록**이 따로 만들어져 같은 **객체**를 중복 해제할 수 있다. [U11-020] (B shared_ptr 생성 시 주의할 점, 인쇄 p.693-695)

enable_shared_from_this를 **상속**한 **클래스**의 shared_from_this는 이미 있는 **제어 블록**을 사용하며, 그 **객체**를 가리키는 **shared_ptr**가 먼저 있어야 한다. [U11-021] (B enable_shared_from_this, 인쇄 p.696-697)

순환 참조와 weak_ptr

shared_ptr가 서로를 소유하는 순환 참조에서는 참조 개수가 0이 되지 않아 **소멸자**가 호출되지 않는 문제가 생긴다. [U11-022] (B 서로 참조하는 shared_ptr, 인쇄 p.698-699)

weak_ptr는 **객체**를 참조하지만 **shared_ptr**의 참조 개수를 늘리지 않아 순환 참조 문제를 해결하는 데 사용한다. [U11-023] (B weak_ptr, 인쇄 p.699-700)

weak_ptr는 **shared_ptr** 또는 다른 **weak_ptr**로 생성하며 일반 **포인터** 주소로 만들 수 없다. [U11-024] (B weak_ptr, 인쇄 p.702)

weak_ptr의 lock은 **객체**가 살아 있으면 그 **객체**의 **shared_ptr**, 이미 해제되었으면 빈 **shared_ptr**를 반환한다. [U11-025] (B weak_ptr, 인쇄 p.702)

제어 블록은 **weak_ptr**도 더 이상 가리키지 않을 때 해제할 수 있으므로 참조 개수와 약한 참조 개수(weak count)를 기록한다. [U11-026] (B weak_ptr, 인쇄 p.703)

Callable과 function

Callable은 ()를 붙여 호출할 수 있는 모든 것을 뜻하며, 함수·함수 **객체**·람다가 예로 나온다. [U11-027] (B Callable, 인쇄 p.705-706)

std::function은 **Callable**을 **객체** 형태로 보관하며, 템플릿 인자로 반환형과 함수 인자를 나타내는 함수 타입을 받는다. [U11-028] (B std::function, 인쇄 p.706-707)

멤버 함수를 **std::function**으로 보관할 때 원래 인자에 호출할 **객체**를 받는 인자를 더하고 &로 멤버 함수 주소를 명시한다. [U11-029] (B 멤버 함수를 가지는 std::function, 인쇄 p.709-710)

mem_fn·bind·ref

mem_fn은 전달한 멤버 함수를 함수 **객체**로 만들어 반환하며, 원문은 transform에 사용한다. [U11-030] (B 멤버 함수들을 함수 객체로 - mem_fn, 인쇄 p.714)

std::bind는 원래 함수의 인자를 특정 값으로 붙이고, placeholders로 새 함수 **객체**의 인자를 전달하거나 순서를 바꾼다. [U11-031] (B std::bind, 인쇄 p.715-716)

bind에 전달한 값은 복사되므로 참조 전달이 필요하면 std::ref를 사용하고, const 레퍼런스에는 std::cref를 사용한다. [U11-032] (B std::bind, 인쇄 p.717-718)

동작 방식·절차

unique_ptr 소유권 이전

1. **unique_ptr**가 **객체**의 유일한 소유권을 갖게 한다. [U11-033] (B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675)

2. 원문은 **std::move**로 다른 **unique_ptr**에 이동 생성한다. [U11-034] (B unique_ptr 소유권 이전하기, 인쇄 p.679)

3. 이전 후 원래 **unique_ptr**는 **nullptr**이므로 그 **객체**에 접근하지 않는다. [U11-035] (B unique_ptr 소유권 이전하기, 인쇄 p.680)

weak_ptr로 객체 접근

1. lock으로 **shared_ptr**를 얻는다. [U11-036] (B weak_ptr, 인쇄 p.702)

2. 반환값을 if로 확인한다. [U11-037] (B weak_ptr, 인쇄 p.702-703)

3. **객체**가 살아 있으면 접근하고, 빈 **shared_ptr**이면 이미 소멸된 것으로 처리한다. [U11-038] (B weak_ptr, 인쇄 p.702)

주요 문법·표·수식

weak_ptr의 lock과 확인: 원문 발췌 (원문 발췌)

```
void access_other() {
  std::shared_ptr<A> o = other.lock();
  if (o) {
    std::cout << "접근: " << o->name() << std::endl;
  } else {
    std::cout << "이미소멸됨ㅠ" << std::endl;
  }
}
```

[U11-039] (B weak_ptr, 인쇄 p.701)

Callable을 std::function에 보관: 원문 발췌 (원문 발췌)

```
std::function<int(const std::string&)> f1 = some_func1;
std::function<void(char)> f2 = S();
std::function<void()> f3 = []() { std::cout << "Func3 호출! " << std::endl; };

f1("hello");
f2('c');
f3();
```

[U11-040] (B std::function, 인쇄 p.707)

스마트 포인터의 소유·참조 방식

종류	원문의 설명
unique_ptr [U11-041] (B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675)	특정 **객체**의 유일한 소유권 [U11-042] (B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675)
shared_ptr [U11-043] (B shared_ptr, 인쇄 p.688)	여러 **포인터**가 같은 **객체**를 소유 [U11-044] (B shared_ptr, 인쇄 p.688)
weak_ptr [U11-045] (B weak_ptr, 인쇄 p.700)	참조 개수를 늘리지 않으며 **shared_ptr**로 변환하여 접근 [U11-046] (B weak_ptr, 인쇄 p.700)

핵심 용어 표

용어	원문에 나온 정의·설명
자원(resource)	원문은 할당한 메모리와 연 파일을 **자원**(resource)으로 들며, 사용이 끝난 **자원**은 반환해야 한다고 설명한다. [U11-003] (B 자원(resource) 관리의 중요성, 인쇄 p.671)
RAII	**RAII**(Resource Acquisition Is Initialization)는 **자원** 관리를 **스택**에 할당한 **객체**를 통해 수행하는 방식이다. [U11-006] (B Resource Acquisition Is Initialization - RAII, 인쇄 p.674)
unique_ptr	**unique_ptr**는 특정 **객체**의 유일한 소유권을 나타내며, 자신이 소멸할 때 가리키던 **객체**도 소멸한다. [U11-009] (B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675,682)
제어 블록(control block)	**shared_ptr**들은 동적으로 할당한 **제어 블록**(control block)을 공유하여 참조 개수 등의 정보를 공유한다. [U11-018] (B shared_ptr, 인쇄 p.692)
shared_ptr	**shared_ptr**는 여러 스마트 **포인터**가 같은 **객체**를 소유할 수 있게 하며, 참조 개수가 0이 되면 **객체**를 해제한다. [U11-016] (B shared_ptr, 인쇄 p.688-689)
weak_ptr	**weak_ptr**는 **객체**를 참조하지만 **shared_ptr**의 참조 개수를 늘리지 않아 순환 참조 문제를 해결하는 데 사용한다. [U11-023] (B weak_ptr, 인쇄 p.699-700)
Callable	**Callable**은 ()를 붙여 호출할 수 있는 모든 것을 뜻하며, 함수·함수 **객체**·람다가 예로 나온다. [U11-027] (B Callable, 인쇄 p.705-706)
std::function	**std::function**은 **Callable**을 **객체** 형태로 보관하며, 템플릿 인자로 반환형과 함수 인자를 나타내는 함수 타입을 받는다. [U11-028] (B std::function, 인쇄 p.706-707)

단원 요약

요약 기준: 편집 요약

unique_ptr는 특정 **객체**의 유일한 소유권을 나타내며, 자신이 소멸할 때 가리키던 **객체**도 소멸한다. [U11-009] (B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675,682)

shared_ptr는 여러 스마트 **포인터**가 같은 **객체**를 소유할 수 있게 하며, 참조 개수가 0이 되면 **객체**를 해제한다. [U11-016] (B shared_ptr, 인쇄 p.688-689)

weak_ptr는 **객체**를 참조하지만 **shared_ptr**의 참조 개수를 늘리지 않아 순환 참조 문제를 해결하는 데 사용한다. [U11-023] (B weak_ptr, 인쇄 p.699-700)

std::function은 **Callable**을 **객체** 형태로 보관하며, 템플릿 인자로 반환형과 함수 인자를 나타내는 함수 타입을 받는다. [U11-028] (B std::function, 인쇄 p.706-707)

12 쓰레드·동기화·atomic·비동기·ThreadPool

B PDF 724-830

다루는 내용

thread **객체**를 생성하면 인자로 받은 함수를 새로운 쓰레드에서 실행한다. [U12-001] (B 16장 / C++ 에서 쓰레드 생성하기, 인쇄 p.729)

쓰레드풀 사용자는 쓰레드들을 생성해 두고 수행할 일을 쓰레드풀에 추가한다. [U12-002] (B 16장 / ThreadPool, 인쇄 p.806)

핵심 개념

C++ 쓰레드와 실행

프로세스는 운영체제에서 실행되는 프로그램의 최소 단위이다. [U12-003] (B 16장 / 멀티 쓰레드 프로그램, 인쇄 p.720)

같은 **프로세스** 안의 쓰레드들은 메모리를 공유한다. [U12-004] (B 16장 / 멀티 쓰레드 프로그램, 인쇄 p.722)

thread **객체**를 생성하면 인자로 받은 함수를 새로운 쓰레드에서 실행한다. [U12-001] (B 16장 / C++ 에서 쓰레드 생성하기, 인쇄 p.729)

쓰레드가 CPU 코어에 어떻게 할당되고 언제 컨텍스트 스위치를 할지는 운영체제가 결정한다. [U12-005] (B 16장 / C++ 에서 쓰레드 생성하기, 인쇄 p.729)

join 은 해당 쓰레드가 실행을 종료하면 리턴한다. [U12-006] (B 16장 / join, 인쇄 p.730)

쓰레드를 detach 하면 main 함수에서는 그 쓰레드가 종료될 때까지 기다리지 않는다. [U12-007] (B 16장 / detach, 인쇄 p.733)

thread **생성자**의 첫 인자는 **Callable** 이며, 이어서 그 함수에 전달할 인자들을 적는다. [U12-008] (B 16장 / 쓰레드에 인자 전달하기, 인쇄 p.735)

this_thread::get_id 로 현재 쓰레드의 아이디를 얻을 수 있다. [U12-009] (B 16장 / 쓰레드에 인자 전달하기, 인쇄 p.735)

뮤텍스·조건 변수

원문의 counter 예제는 mutex 의 lock 과 unlock 사이에서 result += 1 을 수행한다. [U12-010] (B 16장 / 뮤텍스와 조건 변수, 인쇄 p.744)

뮤텍스의 lock 과 unlock 사이에서 한 쓰레드만 실행할 수 있는 코드 부분을 임계 영역(critical section)이라고 부른다. [U12-011] (B 16장 / 뮤텍스(mutex), 인쇄 p.745)

lock_guard 는 **생성자**에서 뮤텍스를 lock 하고 소멸할 때 unlock 한다. [U12-012] (B 16장 / lock_guard, 인쇄 p.748)

원문의 데드락 예제에서는 worker1 이 m1 을, worker2 가 m2 를 잠근 채 서로 상대가 가진 뮤텍스의 해제를 기다려 진행하지 못한다. [U12-013] (B 16장 / 데드락(deadlock), 인쇄 p.749-750)

여러 Lock 을 획득해야 한다면 정해진 순서로 획득한다. [U12-014] (B 16장 / Lock 들을 언제나 정해진 순서로 획득해라, 인쇄 p.754)

생산자(producer)는 처리할 일을 받아오는 쓰레드이고, 소비자(consumer)는 받은 일을 처리하는 쓰레드이다. [U12-015] (B 16장 / 생산자와 소비자 패턴, 인쇄 p.754)

조건 **변수**(**condition_variable**)는 특정 조건이 만족될 때까지 쓰레드를 기다리게 하는 데 사용한다. [U12-016] (B 16장 / condition_variable, 인쇄 p.760)

wait 에 전달한 조건이 거짓이면 unique_lock 을 unlock 한 뒤 sleep 한다. [U12-017] (B 16장 / condition_variable, 인쇄 p.763)

unique_lock 은 unlock 후 다시 lock 할 수 있고, cv->wait 는 unique_lock 을 인자로 받는다. [U12-018] (B 16장 / condition_variable, 인쇄 p.764)

notify_one 은 기다리는 쓰레드 중 하나를 깨워 조건을 다시 검사하게 한다. [U12-019] (B 16장 / condition_variable, 인쇄 p.764)

원문의 생산자·소비자 예제는 생산자들을 **join** 한 뒤 notify_all 로 잠든 소비자들을 모두 깨운다. [U12-020] (B 16장 / condition_variable, 인쇄 p.764)

원자성·메모리 순서

원자적 연산은 중간에 다른 쓰레드가 끼어들 여지가 없으며, 연산을 했거나 하지 않았거나만 존재한다. [U12-021] (B 16장 / 원자성(atomicity), 인쇄 p.774-775)

atomic 의 템플릿 인자로 원자적으로 만들고 싶은 타입을 전달한다. [U12-022] (B 16장 / 원자성 (atomicity), 인쇄 p.776)

memory_order_relaxed 는 단일 쓰레드 관점의 결과가 같다면 서로 다른 **변수**의 relaxed 메모리 연산을 재배치할 수 있다. [U12-023] (B 16장 / memory_order_relaxed, 인쇄 p.779)

memory_order_release 는 이전 메모리 명령이 뒤로 재배치되는 것을 금지하고,
memory_order_acquire 는 뒤의 메모리 명령이 앞으로 재배치되는 것을 금지한다. [U12-024] (B 16장 / release 와 acquire, 인쇄 p.783)

memory_order_seq_cst 를 사용하는 메모리 연산들은 모든 쓰레드에서 동일한 연산 순서로 관찰되며, atomic 의 기본 memory_order 이다. [U12-025] (B 16장 / memory_order_seq_cst, 인쇄 p.787)

비동기 연산 도구

프로그램 실행이 여러 갈래로 갈라져 동시에 진행되는 것을 비동기적(asynchronous) 실행이라고 부른다. [U12-026] (B 16장 / 비동기 연산을 위한 도구들, 인쇄 p.793)

promise 가 future 에 값을 전달하기 전까지 wait 가 기다리며, get 으로 전달된 **객체**를 얻는다. [U12-027] (B 16장 / std::promise 와 std::future, 인쇄 p.795)

future 의 get 은 안의 데이터를 이동하므로 두 번 호출하면 안 된다. [U12-028] (B 16장 / std::promise 와 std::future, 인쇄 p.795)

promise 의 set_exception 으로 전달된 **예외**는 future 의 get 을 호출할 때 던져진다. [U12-029] (B 16장 / 예외 전달, 인쇄 p.797)

wait_for 은 정해진 시간 동안 기다리며, future_status 는 ready, timeout, deferred 의 세 **상태**를 갖는다. [U12-030] (B 16장 / wait_for, 인쇄 p.797,799)

여러 쓰레드에서 future 를 get 해야 하는 경우 shared_future 를 사용한다. [U12-031] (B 16장 / shared_future, 인쇄 p.799)

packaged_task 는 전달받은 함수를 실행하여 리턴값을 promise 에 설정하고, get_future 로 대응하는 future 를 얻는다. [U12-032] (B 16장 / packaged_task, 인쇄 p.801)

std::async 는 실행 결과를 future 로 돌려주며, launch::async 는 바로 새 쓰레드에서 실행하고 launch::deferred 는 future::get 시 새 쓰레드 없이 실행한다. [U12-033] (B 16장 / std::async, 인쇄 p.803)

ThreadPool 만들기

쓰레드풀 사용자는 쓰레드들을 생성해 두고 수행할 일을 쓰레드풀에 추가한다. [U12-002] (B 16장 / ThreadPool, 인쇄 p.806)

원문의 jobs_ 는 std::queue<**std::function**<void()>> 이며, 가장 오래전에 추가된 작업을 먼저 얻기 위해 큐를 사용한다. [U12-034] (B 16장 / ThreadPool 만들기, 인쇄 p.808)

cv_job_q_ 와 m_job_q_ 는 생산자·소비자 패턴을 구현하고, stop_all 은 Worker 쓰레드의 종료 조건을 나타낸다. [U12-035] (B 16장 / ThreadPool 만들기, 인쇄 p.808)

WorkerThread 는 작업이 생기거나 stop_all 이 설정될 때까지 기다리고, stop_all 이 참이며 큐가 비면 종료한다. [U12-036] (B 16장 / ThreadPool 첫번째 버전, 인쇄 p.809)

EnqueueJob 은 큐에 작업을 넣은 뒤 notify_one 으로 쓰레드 하나를 깨운다. [U12-037] (B 16장 / ThreadPool 첫번째 버전, 인쇄 p.810)

원문의 확장 버전은 **packaged_task** 를 **shared_ptr** 에 보관하고 람다에 복사본을 전달해 작업 실행 시점까지 **객체**를 유지한다. [U12-038] (B 16장 / ThreadPool 확장, 인쇄 p.818)

EnqueueJob 의 반환형에는 std::result_of 로 구한 함수의 리턴 타입을 가지는 future 를 사용한다. [U12-039] (B 16장 / ThreadPool 확장, 인쇄 p.814)

동작 방식·절차

조건 변수를 쓰는 원문 생산자·소비자 처리 순서

1. wait 에 전달한 조건이 거짓이면 unique_lock 을 unlock 한 뒤 sleep 한다. [U12-017] (B 16장 / condition_variable, 인쇄 p.763)

2. notify_one 은 기다리는 쓰레드 중 하나를 깨워 조건을 다시 검사하게 한다. [U12-019] (B 16장 / condition_variable, 인쇄 p.764)

3. 원문의 생산자·소비자 예제는 생산자들을 **join** 한 뒤 notify_all 로 잠든 소비자들을 모두 깨운다. [U12-020] (B 16장 / condition_variable, 인쇄 p.764)

원문 WorkerThread 의 처리 순서

1. WorkerThread 는 작업이 생기거나 stop_all 이 설정될 때까지 기다리고, stop_all 이 참이며 큐가 비면 종료한다. [U12-036] (B 16장 / ThreadPool 첫번째 버전, 인쇄 p.809)

2. 큐 맨 앞의 job 을 꺼내고 pop 한 뒤 lock 을 unlock 한다. [U12-040] (B 16장 / ThreadPool 첫번째 버전, 인쇄 p.809)

3. 꺼낸 job 을 실행하고 while 루프에서 다음 작업을 기다린다. [U12-041] (B 16장 / ThreadPool 첫번째 버전, 인쇄 p.809)

주요 문법·표·수식

lock_guard 를 사용하는 원문 worker 발췌 (원문 발췌)

```
void worker(int& result, std::mutex& m) {
  for (int i = 0; i < 10000; i++) {
    std::lock_guard<std::mutex> lock(m);
    result += 1;
  }
}
```

[U12-042] (B 16장 / lock_guard, 인쇄 p.747)

원문의 연산별 허용 memory order

연산	허용 memory order
쓰기(store) [U12-043] (B 16장 / memory order 표, 인쇄 p.789)	memory_order_relaxed, memory_order_release, memory_order_seq_cst [U12-044] (B 16장 / memory order 표, 인쇄 p.789)
읽기(load) [U12-045] (B 16장 / memory order 표, 인쇄 p.789)	memory_order_relaxed, memory_order_consume, memory_order_acquire, memory_order_seq_cst [U12-046] (B 16장 / memory order 표, 인쇄 p.789)
읽고-수정하고-쓰기 [U12-047] (B 16장 / memory order 표, 인쇄 p.789)	memory_order_relaxed, memory_order_consume, memory_order_acquire, memory_order_release, memory_order_acq_rel, memory_order_seq_cst [U12-048] (B 16장 / memory order 표, 인쇄 p.789)

핵심 용어 표

용어	원문에 나온 정의·설명
프로세스	**프로세스**는 운영체제에서 실행되는 프로그램의 최소 단위이다. [U12-003] (B 16장 / 멀티 쓰레드 프로그램, 인쇄 p.720)
join	**join** 은 해당 쓰레드가 실행을 종료하면 리턴한다. [U12-006] (B 16장 / join, 인쇄 p.730)
lock_guard	**lock_guard** 는 **생성자**에서 뮤텍스를 lock 하고 소멸할 때 unlock 한다. [U12-012] (B 16장 / lock_guard, 인쇄 p.748)
condition_variable	조건 **변수**(**condition_variable**)는 특정 조건이 만족될 때까지 쓰레드를 기다리게 하는 데 사용한다. [U12-016] (B 16장 / condition_variable, 인쇄 p.760)
원자적 연산	**원자적 연산**은 중간에 다른 쓰레드가 끼어들 여지가 없으며, 연산을 했거나 하지 않았거나만 존재한다. [U12-021] (B 16장 / 원자성(atomicity), 인쇄 p.774-775)
promise·future	promise 가 future 에 값을 전달하기 전까지 wait 가 기다리며, get 으로 전달된 **객체**를 얻는다. [U12-027] (B 16장 / std::promise 와 std::future, 인쇄 p.795)
packaged_task	**packaged_task** 는 전달받은 함수를 실행하여 리턴값을 promise 에 설정하고, get_future 로 대응하는 future 를 얻는다. [U12-032] (B 16장 / packaged_task, 인쇄 p.801)
ThreadPool	쓰레드풀 사용자는 쓰레드들을 생성해 두고 수행할 일을 쓰레드풀에 추가한다. [U12-002] (B 16장 / ThreadPool, 인쇄 p.806)

단원 요약

요약 기준: 편집 요약

같은 **프로세스** 안의 쓰레드들은 메모리를 공유한다. [U12-004] (B 16장 / 멀티 쓰레드 프로그램, 인쇄 p.722)

lock_guard 는 **생성자**에서 뮤텍스를 lock 하고 소멸할 때 unlock 한다. [U12-012] (B 16장 / lock_guard, 인쇄 p.748)

memory_order_seq_cst 를 사용하는 메모리 연산들은 모든 쓰레드에서 동일한 연산 순서로 관찰되며, atomic 의 기본 memory_order 이다. [U12-025] (B 16장 / memory_order_seq_cst, 인쇄 p.787)

std::async 는 실행 결과를 future 로 돌려주며, launch::async 는 바로 새 쓰레드에서 실행하고 launch::deferred 는 future::get 시 새 쓰레드 없이 실행한다. [U12-033] (B 16장 / std::async, 인쇄 p.803)

13 유니폼 초기화·constexpr·decltype

B PDF 831-870

다루는 내용

유니폼 초기화는 {} 를 이용한 **생성자** 호출이며, 인자 없는 **생성자**가 함수의 정의로 오해되는 일을 막는다. [U13-001] (B 17장 / 뭘 배웠지?, 인쇄 p.836)

decltype 는 전달된 식의 타입으로 치환된다. [U13-002] (B 19장 / decltype, 인쇄 p.852)

핵심 개념

유니폼 초기화·initializer_list

A a(); 는 **객체** a 를 만드는 대신, A 를 리턴하고 인자를 받지 않는 함수 a 로 해석된다. [U13-003] (B 17장 / C++ 유니폼 초기화, 인쇄 p.826)

유니폼 초기화는 {} 를 이용한 **생성자** 호출이며, 인자 없는 **생성자**가 함수의 정의로 오해되는 일을 막는다. [U13-001] (B 17장 / 뭘 배웠지?, 인쇄 p.836)

중괄호 **초기화**는 데이터 손실이 있는 Narrowing 변환을 불허하며, 원문의 A b{3.5}; 는 double 에서 int 로 변환할 수 없어 오류가 난다. [U13-004] (B 17장 / 유니폼 초기화, 인쇄 p.829)

함수의 리턴 시 {} 를 사용하면 생성하는 **객체**의 타입을 다시 명시하지 않아도 된다. [U13-005] (B 17장 / 유니폼 초기화, 인쇄 p.829)

map 의 **초기화** 리스트는 pair<Key, Value> 원소를 받으며, pair 의 첫 원소는 키이고 두 번째 원소는 값이다. [U13-006] (B 17장 / initializer_list, 인쇄 p.832)

{} 로 **객체**를 생성할 때 **initializer_list** 를 받는 **생성자**가 있으면 오버로딩에서 최우선으로 고려한다. [U13-007] (B 17장 / initializer_list 사용시 주의할 점, 인쇄 p.832)

원문에서 vector v{10}; 은 10 을 보관한 원소 한 개를 만들고, vector v(10); 은 count **생성자**를 호출한다. [U13-008] (B 17장 / initializer_list 사용시 주의할 점, 인쇄 p.832-833)

C++ 17 기준 **auto** x = {arg1, arg2...} 는 인자들이 같은 타입이면 std::**initializer_list**<T> 로 추론된다. [U13-009] (B 17장 / initializer_list 와 auto, 인쇄 p.835)

C++ 17 기준 **auto** x {arg1, arg2, ...} 는 인자 한 개이면 그 타입으로 추론하고 여러 개이면 오류가 난다. [U13-010] (B 17장 / initializer_list 와 auto, 인쇄 p.835)

auto list = {"a", "b", "cc"}; 의 타입은 **initializer_list**<const char*> 이다. [U13-011] (B 17장 / initializer_list 와 auto, 인쇄 p.836)

컴파일 타임 상수 constexpr

constexpr 은 식이 **상수**식이라고 명시하는 키워드이며, **객체** 정의에 붙이면 그 **객체**를 **상수**식에 사용할 수 있다. [U13-012] (B 18장 / constexpr, 인쇄 p.838)

const 로 정의된 **상수**는 **컴파일** 타임에 값을 알 필요가 없지만, **constexpr 변수**의 **초기화**에는 다른 **상수**식이 와야 한다. [U13-013] (B 18장 / constexpr vs const, 인쇄 p.838-839)

constexpr 함수는 조건이 맞을 때 반환값을 **컴파일** 타임 **상수**로 만들 수 있다. [U13-014] (B 18장 / constexpr 함수, 인쇄 p.842)

원문의 Factorial(10) 은 **컴파일** 타임에 계산되어 **클래스** A 의 템플릿 인자로 들어가며 출력은 3628800 이다. [U13-015] (B 18장 / constexpr 함수, 인쇄 p.842)

원문은 C++ 14 에서 **constexpr** 함수의 제약이 완화되었다고 설명하며, goto·**리터럴 타입**이 아닌 **변수**·**초기화**되지 않은 **변수** 등의 제한을 열거한다. [U13-016] (B 18장 / constexpr 함수, 인쇄 p.843)

리터럴(Literal) 타입은 **컴파일러**가 **컴파일** 타임에 정의할 수 있는 타입이다. [U13-017] (B 18장 / 리터럴 타입?, 인쇄 p.845)

constexpr 생성자의 인자는 **리터럴 타입**이어야 하며, 해당 **클래스**는 다른 **클래스**를 가상 **상속**받을 수 없다. [U13-018] (B 18장 / constexpr 생성자, 인쇄 p.845)

if constexpr 의 조건은 bool 로 변환될 수 있는 **컴파일** 타임 **상수**식이어야 한다. [U13-019] (B 18장 / if constexpr, 인쇄 p.850)

if constexpr 이 참이면 else 문은 **컴파일**되지 않고, 거짓이면 else 부분만 **컴파일**된다. [U13-020] (B 18장 / if constexpr, 인쇄 p.850)

decltype·값 카테고리·std::declval

decltype 는 전달된 식의 타입으로 치환된다. [U13-002] (B 19장 / decltype, 인쇄 p.852)

괄호로 둘러싸이지 않은 식별자 **표현식**을 **decltype** 에 전달하면 해당 식의 타입을 얻는다. [U13-021] (B 19장 / decltype, 인쇄 p.853)

식별자 **표현식**이 아닌 경우, 식의 타입이 T 이고 값 카테고리가 xvalue 이면 T&&, lvalue 이면 T&, prvalue 이면 T 가 된다. [U13-022] (B 19장 / decltype, 인쇄 p.853)

모든 C++ 식에는 타입과 값 카테고리(value category)가 따른다. [U13-023] (B 19장 / Value Category, 인쇄 p.853)

정체를 알 수 있는 식들을 glvalue 라고 하며, 이동시킬 수 있는 식들을 rvalue 라고 한다. [U13-024] (B 19장 / Value Category, 인쇄 p.854)

이름을 가진 대부분의 **객체**는 lvalue 이며, **우측값 레퍼런스** 인자 a 도 식 a 의 값 카테고리는 lvalue 이다. [U13-025] (B 19장 / lvalue, 인쇄 p.855)

prvalue 는 **우측값 레퍼런스**와 **상수** 좌측값 레퍼런스의 **초기화**에 사용할 수 있다. [U13-026] (B 19장 / prvalue, 인쇄 p.857)

우측값 레퍼런스를 리턴하는 함수의 호출식은 xvalue 의 대표적인 경우이며 **std::move**(x) 가 예이다. [U13-027] (B 19장 / xvalue, 인쇄 p.857)

원문에서 **decltype**((a)) 는 lvalue 에 대한 타입 추론이므로 int& 가 된다. [U13-028] (B 19장 / decltype 의 쓰임새, 인쇄 p.858)

원문의 **auto** j = i; 는 const 를 없애지만, **decltype**(i) 는 const 를 보존하며 **배열** 타입도 그대로 전달한다. [U13-029] (B 19장 / decltype 의 쓰임새, 인쇄 p.858-859)

decltype 안의 식은 런타임에 실행되지 않는다. [U13-030] (B 19장 / std::declval, 인쇄 p.860)

std::declval<T>() 는 T 의 **생성자**를 직접 호출하지 않고 타입 연산에 **객체**를 나타낼 수 있으며, 런타임에서 사용하면 오류가 난다. [U13-031] (B 19장 / std::declval, 인쇄 p.863)

동작 방식·절차

원문의 decltype 판별 순서

1. 괄호로 둘러싸이지 않은 식별자 **표현식**을 **decltype** 에 전달하면 해당 식의 타입을 얻는다. [U13-021] (B 19장 / decltype, 인쇄 p.853)

2. 식별자 **표현식**이 아닌 경우, 식의 타입이 T 이고 값 카테고리가 xvalue 이면 T&&, lvalue 이면 T&, prvalue 이면 T 가 된다. [U13-022] (B 19장 / decltype, 인쇄 p.853)

원문의 if constexpr 처리

1. if constexpr 의 조건은 bool 로 변환될 수 있는 **컴파일** 타임 **상수**식이어야 한다. [U13-019] (B 18장 / if constexpr, 인쇄 p.850)

2. if constexpr 이 참이면 else 문은 **컴파일**되지 않고, 거짓이면 else 부분만 **컴파일**된다. [U13-020] (B 18장 / if constexpr, 인쇄 p.850)

주요 문법·표·수식

원문 constexpr 함수 발췌 (원문 발췌)

```
constexpr int Factorial(int n) {
  int total = 1;
  for (int i = 1; i <= n; i++) {
    total *= i;
  }
  return total;
}
```

[U13-032] (B 18장 / constexpr 함수, 인쇄 p.842)

식별자 표현식이 아닌 식의 decltype

값 카테고리	decltype 결과 (식의 타입 T)
xvalue [U13-033] (B 19장 / decltype, 인쇄 p.853)	T&& [U13-034] (B 19장 / decltype, 인쇄 p.853)
lvalue [U13-035] (B 19장 / decltype, 인쇄 p.853)	T& [U13-036] (B 19장 / decltype, 인쇄 p.853)
prvalue [U13-037] (B 19장 / decltype, 인쇄 p.853)	T [U13-038] (B 19장 / decltype, 인쇄 p.853)

핵심 용어 표

용어	원문에 나온 정의·설명
유니폼 초기화	**유니폼 초기화**는 {} 를 이용한 **생성자** 호출이며, 인자 없는 **생성자**가 함수의 정의로 오해되는 일을 막는다. [U13-001] (B 17장 / 뭘 배웠지?, 인쇄 p.836)
initializer_list	{} 로 **객체**를 생성할 때 **initializer_list** 를 받는 **생성자**가 있으면 오버로딩에서 최우선으로 고려한다. [U13-007] (B 17장 / initializer_list 사용시 주의할 점, 인쇄 p.832)
constexpr	**constexpr** 은 식이 **상수**식이라고 명시하는 키워드이며, **객체** 정의에 붙이면 그 **객체**를 **상수**식에 사용할 수 있다. [U13-012] (B 18장 / constexpr, 인쇄 p.838)
const·constexpr	const 로 정의된 **상수**는 **컴파일** 타임에 값을 알 필요가 없지만, **constexpr 변수**의 **초기화**에는 다른 **상수**식이 와야 한다. [U13-013] (B 18장 / constexpr vs const, 인쇄 p.838-839)
리터럴 타입	리터럴(Literal) 타입은 **컴파일러**가 **컴파일** 타임에 정의할 수 있는 타입이다. [U13-017] (B 18장 / 리터럴 타입?, 인쇄 p.845)
if constexpr	**if constexpr** 의 조건은 bool 로 변환될 수 있는 **컴파일** 타임 **상수**식이어야 한다. [U13-019] (B 18장 / if constexpr, 인쇄 p.850)
decltype	**decltype** 는 전달된 식의 타입으로 치환된다. [U13-002] (B 19장 / decltype, 인쇄 p.852)
glvalue·rvalue	정체를 알 수 있는 식들을 glvalue 라고 하며, 이동시킬 수 있는 식들을 rvalue 라고 한다. [U13-024] (B 19장 / Value Category, 인쇄 p.854)
std::declval	**std::declval**<T>() 는 T 의 **생성자**를 직접 호출하지 않고 타입 연산에 **객체**를 나타낼 수 있으며, 런타임에서 사용하면 오류가 난다. [U13-031] (B 19장 / std::declval, 인쇄 p.863)

단원 요약

요약 기준: 편집 요약

유니폼 초기화는 {} 를 이용한 **생성자** 호출이며, 인자 없는 **생성자**가 함수의 정의로 오해되는 일을 막는다. [U13-001] (B 17장 / 뭘 배웠지?, 인쇄 p.836)

constexpr 은 식이 **상수**식이라고 명시하는 키워드이며, **객체** 정의에 붙이면 그 **객체**를 **상수**식에 사용할 수 있다. [U13-012] (B 18장 / constexpr, 인쇄 p.838)

decltype 는 전달된 식의 타입으로 치환된다. [U13-002] (B 19장 / decltype, 인쇄 p.852)

std::declval<T>() 는 T 의 **생성자**를 직접 호출하지 않고 타입 연산에 **객체**를 나타낼 수 있으며, 런타임에서 사용하면 오류가 난다. [U13-031] (B 19장 / std::declval, 인쇄 p.863)

14 type_traits·정규표현식·난수·시간·파일시스템·유틸리티

B PDF 871-960

다루는 내용

type_traits 는 타입에 대한 여러 연산을 수행하는 메타 함수들을 제공한다. [U14-001] (B 20장 / is_void, 인쇄 p.870)

fstream 은 파일 데이터의 입출력을 담당하고, filesystem 은 파일 메타데이타에 대한 접근을 돕는다. [U14-002] (B 20장 / 파일시스템 라이브러리 소개, 인쇄 p.921)

핵심 개념

type_traits·SFINAE·enable_if

템플릿 메타 함수는 함수처럼 동작하는 템플릿 **클래스**이며 타입에 대한 연산을 수행한다. [U14-003] (B 20장 / 템플릿 메타 함수, 인쇄 p.867)

type_traits 는 타입에 대한 여러 연산을 수행하는 메타 함수들을 제공한다. [U14-001] (B 20장 / is_void, 인쇄 p.870)

is_void<void> 는 특수화된 템플릿으로 value 가 true 이고, 다른 타입은 false 이다. [U14-004] (B 20장 / is_void, 인쇄 p.870)

is_integral 은 정수 타입인지 확인하며, static_assert 는 전달한 식의 참·거짓을 **컴파일** 타임에 확인한다. [U14-005] (B 20장 / static_assert, 인쇄 p.870-871)

is_class 는 전달받은 타입이 **클래스**인지 확인하는 메타 함수이다. [U14-006] (B 20장 / is_class, 인쇄 p.872)

SFINAE 는 Substitution Failure Is Not An Error 이며, 문법에 맞지 않는 템플릿 인자 치환 결과를 **함수 오버로딩** 후보에서 제외한다. [U14-007] (B 20장 / SFINAE, 인쇄 p.878)

원문은 **SFINAE** 에서 함수의 인자와 리턴 타입을 확인하며, 함수 내부 전체를 확인하지는 않는다고 설명한다. [U14-008] (B 20장 / SFINAE, 인쇄 p.878)

enable_if 는 참으로 특수화된 버전에 type 을 정의하고, 나머지에는 type 을 정의하지 않는 방식으로 후보를 제한한다. [U14-009] (B 20장 / enable_if, 인쇄 p.882)

정규표현식 <regex>

정규표현식은 문자열에서 패턴을 찾아 규칙 확인, 검색, 치환에 사용한다. [U14-010] (B 20장 / 정규표현식 라이브러리 소개, 인쇄 p.894)

regex_match 는 문자열 전체가 **정규표현식**의 패턴에 부합하면 true 를 리턴한다. [U14-011] (B 20장 / 전체 문자열 매칭하기, 인쇄 p.896)

부분 매칭 결과는 match[i].str() 로 접근하며, regex_match 의 첫 결과는 전체 문자열이고 그 뒤는 () 안의 문자열이다. [U14-012] (B 20장 / 부분 매칭 뽑아내기, 인쇄 p.899)

match.suffix() 는 검색된 패턴 바로 뒤부터 원래 문자열 끝까지를 나타내므로 그 뒤에서 다시 검색할 수 있다. [U14-013] (B 20장 / 원하는 패턴 검색하기, 인쇄 p.901)

regex_iterator 는 생성될 때와 ++ 될 때 regex_search 를 실행하고, **역참조**하면 현재 패턴의 match_results 를 얻는다. [U14-014] (B 20장 / std::regex_iterator, 인쇄 p.903)

regex_replace 는 패턴에 맞는 문자열을 다른 문자열로 치환한다. [U14-015] (B 20장 / 원하는 패턴 치환하기, 인쇄 p.903)

치환 문자열의 $1 은 첫 번째 캡쳐 그룹을 나타내며, 중첩된 그룹의 번호는 괄호가 열리는 순서로 정해진다. [U14-016] (B 20장 / 캡쳐 그룹, 인쇄 p.904,906)

<random>·<chrono>

의사 난수(pseudo random number)는 시드(seed)를 기반으로 난수처럼 보이는 수열을 생성하는 방식이다. [U14-017] (B 20장 / 난수 생성, 인쇄 p.910)

원문은 random_device 로 시드값을 얻고, 그 이후의 난수열은 난수 엔진으로 생성하도록 설명한다. [U14-018] (B 20장 / random_device, 인쇄 p.912)

std::mt19937 은 메르센 트위스터 **알고리즘**을 사용하는 난수 생성 엔진이다. [U14-019] (B 20장 / 난수 엔진, 인쇄 p.912)

uniform_int_distribution<int> 의 **생성자**에 범위를 지정하고, 그 분포에 난수 엔진을 전달하여 샘플을 뽑는다. [U14-020] (B 20장 / 분포, 인쇄 p.913)

정규분포 예제는 std::normal_distribution<double> dist(0, 1) 로 평균 0, 표준편차 1 을 지정한다. [U14-021] (B 20장 / 정규분포, 인쇄 p.913)

clock 의 now() 는 해당 clock 에 맞는 time_point **객체**를 리턴한다. [U14-022] (B 20장 / chrono 에서 지원하는 clock 들, 인쇄 p.917)

원문의 end - start 는 duration **객체**를 얻고, count() 는 시간 차이 동안의 틱 횟수를 알려준다. [U14-023] (B 20장 / duration, 인쇄 p.917)

실제 시간 단위로 변환하기 위해 duration_cast 를 사용한다. [U14-024] (B 20장 / duration_cast, 인쇄 p.917)

날짜 출력 예제는 system_clock::now, to_time_t, localtime, put_time 의 순서로 변환한다. [U14-025] (B 20장 / 현재 시간을 날짜로, 인쇄 p.919)

〈filesystem〉

fstream 은 파일 데이터의 입출력을 담당하고, filesystem 은 파일 메타데이타에 대한 접근을 돕는다. [U14-002] (B 20장 / 파일시스템 라이브러리 소개, 인쇄 p.921)

path 는 파일과 폴더의 경로를 다루는 **객체**이다. [U14-026] (B 20장 / 뭘 배웠지?, 인쇄 p.937)

exists, is_regular_file, is_directory 는 각각 경로의 존재, 파일 여부, 디렉토리 여부를 확인한다. [U14-027] (B 20장 / 파일을 찾아보자, 인쇄 p.921)

current_path() 는 프로그램 실행 경로를 리턴하며, 상대경로는 이 경로를 기준으로 구한다. [U14-028] (B 20장 / 경로, 인쇄 p.925)

directory_iterator 로 디렉토리 안의 파일을 살피고, recursive_directory_iterator 로 안의 모든 파일·폴더를 탐색한다. [U14-029] (B 20장 / 뭘 배웠지?, 인쇄 p.937)

create_directory 는 디렉토리를 생성하고 copy 와 remove 는 파일·폴더를 복사·삭제한다. [U14-030] (B 20장 / 뭘 배웠지?, 인쇄 p.937)

optional·variant·tuple·structured binding

std::optional 은 원하는 값을 보관할 수도, 보관하지 않을 수도 있는 **클래스**이다. [U14-031] (B 20장 / std::optional, 인쇄 p.940)

optional 의 has_value() 로 값의 존재를 확인하고, value() 또는 * 로 값에 접근한다. [U14-032] (B 20장 / std::optional, 인쇄 p.942)

값이 없는 optional 은 보관할 타입의 디폴트 **객체**를 가질 필요가 없다. [U14-033] (B 20장 / std::optional, 인쇄 p.942)

std::optional<std::reference_wrapper<A>> 로 A **객체**의 레퍼런스를 보관할 수 있으며, std::ref 로 wrapper 를 만들고 get() 으로 참조한 **객체**를 얻는다. [U14-034] (B 20장 / 레퍼런스를 가지는 std::optional, 인쇄 p.945)

std::variant 는 **컴파일** 타임에 정한 여러 타입 중 한 타입의 **객체**를 보관하는 one-of **클래스**이다. [U14-035] (B 20장 / std::variant, 인쇄 p.946)

std::monostate 를 사용해 variant 가 아무것도 들고 있지 않은 **상태**를 표현할 수 있다. [U14-036] (B 20장 / std::monostate, 인쇄 p.949)

std::tuple 은 서로 다른 타입의 **객체**들을 묶으며, make_tuple 로 만들고 std::get 으로 원소에 접근한다. [U14-037] (B 20장 / std::tuple, 인쇄 p.950-951)

structured binding 은 **auto** [age, name, is_male] = student; 처럼 tuple 의 원소들을 받으며, 모든 원소를 받아야 한다. [U14-038] (B 20장 / Structured binding, 인쇄 p.953)

structured binding 에서 **auto**& 를 사용하면 복사 대신 레퍼런스를 취할 수 있으며, 구조체나 pair 에도 사용할 수 있다. [U14-039] (B 20장 / Structured binding, 인쇄 p.953-954)

동작 방식·절차

원문의 난수 생성 순서

1. 원문은 random_device 로 시드값을 얻고, 그 이후의 난수열은 난수 엔진으로 생성하도록 설명한다. [U14-018] (B 20장 / random_device, 인쇄 p.912)

2. std::mt19937 은 메르센 트위스터 **알고리즘**을 사용하는 난수 생성 엔진이다. [U14-019] (B 20장 / 난수 엔진, 인쇄 p.912)

3. uniform_int_distribution<int> 의 **생성자**에 범위를 지정하고, 그 분포에 난수 엔진을 전달하여 샘플을 뽑는다. [U14-020] (B 20장 / 분포, 인쇄 p.913)

원문의 현재 시간을 날짜로 변환하는 순서

1. system_clock::now() 로 현재 time_point 를 얻는다. [U14-040] (B 20장 / 현재 시간을 날짜로, 인쇄 p.919)

2. system_clock::to_time_t 로 time_t 로 변환한다. [U14-041] (B 20장 / 현재 시간을 날짜로, 인쇄 p.919)

3. localtime 으로 tm **객체**를 얻고 put_time 으로 출력 형식을 만든다. [U14-042] (B 20장 / 현재 시간을 날짜로, 인쇄 p.919)

주요 문법·표·수식

원문 정규분포 초기화 발췌 (원문 발췌)

```
std::random_device rd;
std::mt19937 gen(rd());
std::normal_distribution<double> dist(/* 평균= */ 0, /* 표준편차= */ 1);
```

[U14-043] (B 20장 / 정규분포, 인쇄 p.913)

정규표현식 함수 비교

함수	원문 기능
regex_match [U14-044] (B 20장 / 전체 문자열 매칭하기, 인쇄 p.896)	regex_match 는 문자열 전체가 **정규표현식**의 패턴에 부합하면 true 를 리턴한다. [U14-011] (B 20장 / 전체 문자열 매칭하기, 인쇄 p.896)
regex_iterator [U14-045] (B 20장 / std::regex_iterator, 인쇄 p.903)	regex_iterator 는 생성될 때와 ++ 될 때 regex_search 를 실행하고, **역참조**하면 현재 패턴의 match_results 를 얻는다. [U14-014] (B 20장 / std::regex_iterator, 인쇄 p.903)
regex_replace [U14-046] (B 20장 / 원하는 패턴 치환하기, 인쇄 p.903)	regex_replace 는 패턴에 맞는 문자열을 다른 문자열로 치환한다. [U14-015] (B 20장 / 원하는 패턴 치환하기, 인쇄 p.903)

핵심 용어 표

용어	원문에 나온 정의·설명
템플릿 메타 함수	**템플릿 메타 함수**는 함수처럼 동작하는 템플릿 **클래스**이며 타입에 대한 연산을 수행한다. [U14-003] (B 20장 / 템플릿 메타 함수, 인쇄 p.867)
SFINAE	**SFINAE** 는 Substitution Failure Is Not An Error 이며, 문법에 맞지 않는 템플릿 인자 치환 결과를 **함수 오버로딩** 후보에서 제외한다. [U14-007] (B 20장 / SFINAE, 인쇄 p.878)
정규표현식	**정규표현식**은 문자열에서 패턴을 찾아 규칙 확인, 검색, 치환에 사용한다. [U14-010] (B 20장 / 정규표현식 라이브러리 소개, 인쇄 p.894)
시드·의사 난수	의사 난수(pseudo random number)는 시드(seed)를 기반으로 난수처럼 보이는 수열을 생성하는 방식이다. [U14-017] (B 20장 / 난수 생성, 인쇄 p.910)
path	**path** 는 파일과 폴더의 경로를 다루는 **객체**이다. [U14-026] (B 20장 / 뭘 배웠지?, 인쇄 p.937)
std::optional	**std::optional** 은 원하는 값을 보관할 수도, 보관하지 않을 수도 있는 **클래스**이다. [U14-031] (B 20장 / std::optional, 인쇄 p.940)
std::variant	**std::variant** 는 **컴파일** 타임에 정한 여러 타입 중 한 타입의 **객체**를 보관하는 one-of **클래스**이다. [U14-035] (B 20장 / std::variant, 인쇄 p.946)
std::tuple	**std::tuple** 은 서로 다른 타입의 **객체**들을 묶으며, make_tuple 로 만들고 std::get 으로 원소에 접근한다. [U14-037] (B 20장 / std::tuple, 인쇄 p.950-951)

단원 요약

요약 기준: 편집 요약

type_traits 는 타입에 대한 여러 연산을 수행하는 메타 함수들을 제공한다. [U14-001] (B 20장 / is_void, 인쇄 p.870)

정규표현식은 문자열에서 패턴을 찾아 규칙 확인, 검색, 치환에 사용한다. [U14-010] (B 20장 / 정규표현식 라이브러리 소개, 인쇄 p.894)

fstream 은 파일 데이터의 입출력을 담당하고, filesystem 은 파일 메타데이타에 대한 접근을 돕는다. [U14-002] (B 20장 / 파일시스템 라이브러리 소개, 인쇄 p.921)

std::tuple 은 서로 다른 타입의 **객체**들을 묶으며, make_tuple 로 만들고 std::get 으로 원소에 접근한다. [U14-037] (B 20장 / std::tuple, 인쇄 p.950-951)

15 Visual Studio 디버깅·Make

L02D PDF 1-13 | B PDF 961-983

다루는 내용

중단점은 임의의 위치에서 프로그램을 정지시키는 기능이다. [U15-001] (L02D Debugging Tool 1 - Break point, 슬라이드 5)

make 는 주어진 쉘 명령어를 조건에 맞게 실행하며, 조건을 담은 파일을 Makefile 이라고 한다. [U15-002] (B 22장 / make, 인쇄 p.964)

핵심 개념

Visual Studio 디버깅

디버깅은 버그(오류)를 제거하는 것이다. [U15-003] (L02D What is debugging?, 슬라이드 3)

강의 자료는 **디버깅** 가능한 Debug 구성을 확인하며, Release 모드에서는 값을 제대로 볼 수 없다고 설명한다. [U15-004] (L02D Preface, 슬라이드 4)

중단점은 임의의 위치에서 프로그램을 정지시키는 기능이다. [U15-001] (L02D Debugging Tool 1 - Break point, 슬라이드 5)

중단할 위치의 왼쪽 끝 클릭 또는 F9 으로 **중단점**을 만들고, Debug 구성에서 Ctrl+F5 가 아닌 F5 로 빌드 후 **디버깅**을 실행한다. [U15-005] (L02D Debugging Tool 1 - Break point, 슬라이드 5)

노란 화살표는 현재 중단된 위치를 표시하며, 그 **명령문**은 아직 실행되지 않은 **상태**이다. [U15-006] (L02D Debugging Tool 1 - Break point, 슬라이드 5)

F10 은 **명령문**을 실행하고 다음 줄로 진행하는 step over 이고, F11 은 호출되는 추가 코드로 들어가는 step into 이다. [U15-007] (L02D Debugging Tool 1 - Break point, 슬라이드 6)

F5 는 다음 **중단점**까지 진행하고 Shift+F11 은 상위 stack 으로 나가는 step out 이다. [U15-008] (L02D Debugging Tool 1 - Break point, 슬라이드 6)

변수 위에 마우스를 올리거나 하단 창에서 값을 확인하며, 방금 명령줄로 값이 변한 **변수**는 빨간색으로 표시된다. [U15-009] (L02D Debugging Tool 1 - Break point, 슬라이드 7)

메모리 윈도우는 **변수**가 메모리에 저장된 모습을 살피며, 주소 부분에 &**변수**이름 을 입력한다. [U15-010] (L02D Debugging Tool 2 - Memory Window, 슬라이드 8-9)

디버깅에서 의도한 명령 실행 순서와 메모리 데이터가 맞는지 확인한다. [U15-011] (L02D How to debugging?, 슬라이드 10)

Conditional & Action breakpoint 는 특정 조건에서 **중단점**을 활성화할 수 있고 재**컴파일**이 필요 없다. [U15-012] (L02D Conditional & Action breakpoint, 슬라이드 12)

Visual Studio 디버깅 (계속)

디스어셈블리 창은 변환된 어셈블리 코드를, 레지스터 창은 현재 레지스터에 저장된 값을 보여준다. [U15-013] (L02D Disassembly & Register, 슬라이드 13)

컴파일·링킹·Makefile

컴파일은 소스코드를 컴퓨터가 이해할 어셈블리어로 변환하는 과정이다. [U15-014] (B 22장 / 컴파일 (Compile), 인쇄 p.960)

링킹은 서로 다른 파일에 흩어진 함수나 **클래스**들을 묶어 연결하는 과정이다. [U15-015] (B 22장 / 링킹 (Linking), 인쇄 p.962)

make 는 주어진 쉘 명령어를 조건에 맞게 실행하며, 조건을 담은 파일을 Makefile 이라고 한다. [U15-002] (B 22장 / make, 인쇄 p.964)

Makefile 은 target, recipe, **prerequisites** 의 세 요소로 구성된다. [U15-016] (B 22장 / make, 인쇄 p.964)

target 은 **make** 할 대상으로 전달하는 이름이며, recipe 는 그 타겟을 만들 때 실행할 명령어의 나열이다. [U15-017] (B 22장 / target·recipes, 인쇄 p.964)

recipe 의 명령어는 반드시 탭으로 한 번 들여쓴다. [U15-018] (B 22장 / 실행할 명령어(recipes), 인쇄 p.964)

prerequisites 는 타겟에 사용할 파일의 목록이며 의존 파일(dependency)이라고도 한다. [U15-019] (B 22장 / 필요조건들(prerequisites), 인쇄 p.964)

타겟이 의존 파일들의 수정 시점보다 나중에 수정되었다면 그 타겟의 명령어를 실행하지 않는다. [U15-020] (B 22장 / 필요조건들(prerequisites), 인쇄 p.964)

Makefile 변수·패턴·의존 목록

변수는 $(CC) 와 같이 $() 안에 이름을 써서 사용하며, 정의하지 않은 **변수**는 빈 문자열로 치환된다. [U15-021] (B 22장 / 변수, 인쇄 p.966-967)

원문의 = 는 참조 **변수**가 나중에 정의될 때까지 결정을 미루고, := 는 정의하는 시점의 값만 확인한다. [U15-022] (B 22장 / 변수를 정의하는 두 가지 방법, 인쇄 p.967)

.**PHONY**: clean 으로 등록하면 clean 파일의 존재와 관계없이 타겟의 명령을 실행한다. [U15-023] (B 22장 / PHONY, 인쇄 p.969)

%.o: %.cc %.h 패턴은 대응하는 파일의 규칙을 나타내며, 패턴은 target 과 prerequisite 부분에 사용한다. [U15-024] (B 22장 / 패턴 사용하기, 인쇄 p.970)

$@ 는 타겟 이름, $< 는 첫 의존 파일, $^ 는 의존 파일 목록 전체에 대응한다. [U15-025] (B 22장 / 자동 변수, 인쇄 p.971)

$? 는 타겟보다 최신인 의존 파일이며, $+ 는 중복된 파일 이름도 포함한 의존 파일 목록이다. [U15-026] (B 22장 / 자동 변수, 인쇄 p.971)

-MD 옵션으로 **컴파일**하면 의존 파일 목록을 담은 .d 파일이 생성된다. [U15-027] (B 22장 / 자동으로 prerequisite 만들기, 인쇄 p.971-972)

include main.d 는 해당 파일 내용을 포함하고, -include 는 포함할 파일이 없어도 **make** 메시지를 출력하지 않는다. [U15-028] (B 22장 / 자동으로 prerequisite 만들기, 인쇄 p.972-973)

patsubst 는 $(patsubst 패턴,치환후형태,**변수**) 꼴로 패턴을 치환한다. [U15-029] (B 22장 / 최종 정리, 인쇄 p.976)

원문의 분리된 헤더 파일 경로 예제는 **컴파일러** 옵션에 -Iinclude/ 를 추가한다. [U15-030] (B 22장 / 헤더 파일들을 따로 뽑는 경우, 인쇄 p.978)

원문은 **make** 의 -j 뒤에 사용할 쓰레드 수를 적고 **make** -j8 을 예로 든다. [U15-031] (B 22장 / 멀티코어를 활용해서 Make 속도를 올리자, 인쇄 p.978)

동작 방식·절차

강의 자료의 디버깅 진행 순서

1. 강의 자료는 **디버깅** 가능한 Debug 구성을 확인하며, Release 모드에서는 값을 제대로 볼 수 없다고 설명한다. [U15-004] (L02D Preface, 슬라이드 4)

2. 중단할 위치의 왼쪽 끝 클릭 또는 F9 으로 **중단점**을 만들고, Debug 구성에서 Ctrl+F5 가 아닌 F5 로 빌드 후 **디버깅**을 실행한다. [U15-005] (L02D Debugging Tool 1 - Break point, 슬라이드 5)

3. F10 은 **명령문**을 실행하고 다음 줄로 진행하는 step over 이고, F11 은 호출되는 추가 코드로 들어가는 step into 이다. [U15-007] (L02D Debugging Tool 1 - Break point, 슬라이드 6)

4. 변수 위에 마우스를 올리거나 하단 창에서 값을 확인하며, 방금 명령줄로 값이 변한 **변수**는 빨간색으로 표시된다. [U15-009] (L02D Debugging Tool 1 - Break point, 슬라이드 7)

5. 디버깅에서 의도한 명령 실행 순서와 메모리 데이터가 맞는지 확인한다. [U15-011] (L02D How to debugging?, 슬라이드 10)

원문 make main 의 최초 빌드 순서

1. Makefile 에서 main 타겟을 찾고 의존 파일 foo.o, bar.o, main.o 의 타겟을 찾는다. [U15-032] (B 22장 / Makefile 처리 순서, 인쇄 p.965)

2. foo.o 가 없으므로 **컴파일** 명령을 실행하고, bar.o 와 main.o 도 **컴파일**한다. [U15-033] (B 22장 / Makefile 처리 순서, 인쇄 p.965-966)

3. 마지막으로 g++ foo.o bar.o main.o -o main 을 실행한다. [U15-034] (B 22장 / Makefile 처리 순서, 인쇄 p.966)

주요 문법·표·수식

원문 Makefile 일부 발췌 (원문 발췌)

```
foo.o : foo.h foo.cc
    g++ -c foo.cc

bar.o : bar.h bar.cc
    g++ -c bar.cc
```

[U15-035] (B 22장 / Makefile, 인쇄 p.965)

원문 주의사항에 따라 recipe 앞의 들여쓰기는 탭이다. [U15-036] (B 22장 / 주의사항, 인쇄 p.965)

강의의 디버깅 진행 키

키	동작
F10 [U15-037] (L02D Break point, 슬라이드 6)	step over: 해당 **명령문** 실행 후 다음 명령줄로 진행 [U15-038] (L02D Break point, 슬라이드 6)
F11 [U15-039] (L02D Break point, 슬라이드 6)	step into: **명령문**에서 호출하는 추가 코드로 들어감 [U15-040] (L02D Break point, 슬라이드 6)
Shift+F11 [U15-041] (L02D Break point, 슬라이드 6)	step out: 상위 stack 으로 나가기 [U15-042] (L02D Break point, 슬라이드 6)

핵심 용어 표

용어	원문에 나온 정의·설명
디버깅	**디버깅**은 버그(오류)를 제거하는 것이다. [U15-003] (L02D What is debugging?, 슬라이드 3)
중단점	**중단점**은 임의의 위치에서 프로그램을 정지시키는 기능이다. [U15-001] (L02D Debugging Tool 1 - Break point, 슬라이드 5)
메모리 윈도우	**메모리 윈도우**는 **변수**가 메모리에 저장된 모습을 살피며, 주소 부분에 &**변수**이름 을 입력한다. [U15-010] (L02D Debugging Tool 2 - Memory Window, 슬라이드 8-9)
컴파일	**컴파일**은 소스코드를 컴퓨터가 이해할 어셈블리어로 변환하는 과정이다. [U15-014] (B 22장 / 컴파일(Compile), 인쇄 p.960)
링킹	**링킹**은 서로 다른 파일에 흩어진 함수나 **클래스**들을 묶어 연결하는 과정이다. [U15-015] (B 22장 / 링킹(Linking), 인쇄 p.962)
make	**make** 는 주어진 쉘 명령어를 조건에 맞게 실행하며, 조건을 담은 파일을 Makefile 이라고 한다. [U15-002] (B 22장 / make, 인쇄 p.964)
prerequisites	**prerequisites** 는 타겟에 사용할 파일의 목록이며 의존 파일(dependency)이라고도 한다. [U15-019] (B 22장 / 필요조건들(prerequisites), 인쇄 p.964)
PHONY	.**PHONY**: clean 으로 등록하면 clean 파일의 존재와 관계없이 타겟의 명령을 실행한다. [U15-023] (B 22장 / PHONY, 인쇄 p.969)

단원 요약

요약 기준: 편집 요약

디버깅에서 의도한 명령 실행 순서와 메모리 데이터가 맞는지 확인한다. [U15-011] (L02D How to debugging?, 슬라이드 10)

make 는 주어진 쉘 명령어를 조건에 맞게 실행하며, 조건을 담은 파일을 Makefile 이라고 한다. [U15-002] (B 22장 / make, 인쇄 p.964)

recipe 의 명령어는 반드시 탭으로 한 번 들여쓴다. [U15-018] (B 22장 / 실행할 명령어(recipes), 인쇄 p.964)

$? 는 타겟보다 최신인 의존 파일이며, $+ 는 중복된 파일 이름도 포함한 의존 파일 목록이다. [U15-026] (B 22장 / 자동 변수, 인쇄 p.971)

16 리스트, 스택, 큐

K PDF 10-61

다루는 내용

프로그램에 적합한 자료 구조를 사용하는 것이 얼마나 중요한지 설명한다. [U16-001] (K 1장 학습 목표, PDF p.11)

C++ 내장 자료 구조와 사용자 정의 선형 자료 구조를 다룬다. [U16-002] (K 1장 학습 목표, PDF p.11)

핵심 개념

1.1~1.2 데이터 저장과 선형 자료 구조

시간 복잡도는 특정 작업을 수행하는 데 걸리는 시간을 데이터 크기에 대한 수식으로 표현하는 방식이다. [U16-003] (K 1.2 연속된 자료 구조와 연결된 자료 구조, PDF p.12)

연속된 자료 구조는 모든 원소를 단일 메모리 청크에 저장한다. [U16-004] (K 1.2.1 연속된 자료 구조, PDF p.12)

연속된 자료 구조에서 i번째 원소의 주소는 BA + i * sizeof(type)으로 계산하며 접근 시간은 O(1)이다. [U16-005] (K 1.2.1 연속된 자료 구조, PDF p.13)

연결된 자료 구조는 노드라고 하는 여러 메모리 청크에 데이터를 저장한다. [U16-006] (K 1.2.2 연결된 자료 구조, PDF p.14)

연결 리스트의 노드는 데이터와 다음 노드를 가리키는 next **포인터**를 가지며 마지막 노드의 next는 NULL이다. [U16-007] (K 1.2.2 연결된 자료 구조, PDF p.14)

연결 리스트의 원소에 접근하려면 헤드부터 next를 따라 이동하므로 원소 접근은 O(n)이다. [U16-008] (K 1.2.2 연결된 자료 구조, PDF p.14)

C 스타일 **배열**은 메모리 할당·해제, 범위 검사, **깊은 복사**에서 제약이 있다. [U16-009] (K 1.2.4 C 스타일 배열의 제약 사항, PDF p.16-17)

1.3~1.4 std::array와 std::vector

std::array는 원소 타입과 **배열** 크기를 매개**변수**로 사용하는 **클래스** 템플릿이며 메모리를 자동으로 할당하고 해제한다. [U16-010] (K 1.3 std::array, PDF p.17)

std::vector는 고정 크기 문제를 해결하며 **초기화**할 때 데이터 크기를 제공하지 않아도 된다. [U16-011] (K 1.4.1 std::vector - 가변 크기 배열, PDF p.29)

벡터의 크기는 실제로 저장된 원소 개수이며 용량(capacity)과 다른 의미이다. [U16-012] (K 1.4.1 std::vector - 가변 크기 배열, PDF p.29)

push_back()은 맨 뒤에 원소를 추가하고 insert()는 **반복자**로 지정한 위치에 원소를 추가한다. [U16-013] (K 1.4.1 std::vector - 가변 크기 배열, PDF p.30)

push_back()은 남은 공간이 있으면 O(1), 공간이 부족하여 전체 원소를 복사·이동하면 O(n)이며 평균 **시간 복잡도**는 O(1)에 가깝다. [U16-014] (K 1.4.1 std::vector - 가변 크기 배열, PDF p.30)

pop_back()은 O(1), erase()는 뒤쪽 원소를 앞으로 이동해야 하므로 O(n)이다. [U16-015] (K 1.4.1 std::vector - 가변 크기 배열, PDF p.32)

reserve(capacity)는 벡터에서 사용할 용량을 지정하며 크기를 변경하지 않는다. [U16-016] (K 1.4.1 std::vector - 가변 크기 배열, PDF p.32)

사용자 정의 할당자는 allocate(), deallocate(), construct(), destroy() 등의 함수를 제공해야 한다. [U16-017] (K 1.4.2 std::vector 할당자, PDF p.32)

1.5~1.7 연결 리스트와 반복자

std::forward_list는 기본적인 **연결 리스트**의 성능을 유지하면서 추가 기능을 제공하며, 전체 크기 반환과 마지막 원소 직접 접근은 제공하지 않는다. [U16-018] (K 1.5 std::forward_list, PDF p.33)

std::forward_list는 push_front()와 insert_after()로 원소를 삽입하며 이들 삽입 함수의 **시간 복잡도**는 O(1)이다. [U16-019] (K 1.5.1 std::forward_list에서 원소 삽입과 삭제, PDF p.34)

unique()는 인접한 원소끼리 비교하므로 리스트 전체에서 유일한 원소만 남기려면 먼저 정렬한 후 호출한다. [U16-020] (K 1.5.2 std::forward_list의 기타 멤버 함수, PDF p.39)

반복자는 **포인터**와 비슷하지만 **STL** 컨테이너에 대해 공통의 인터페이스를 제공한다. [U16-021] (K 1.6 반복자, PDF p.39)

배열·벡터에는 임의 접근 **반복자**, std::forward_list에는 증가 연산만 가능한 순방향 **반복자**가 제공된다. [U16-022] (K 1.6 반복자, PDF p.40)

std::list는 prev와 next를 가지는 이중 **연결 리스트**이며 빠른 push_back()과 size()를 지원한다. [U16-023] (K 1.7 std::list, PDF p.48)

벡터의 메모리 재할당은 기존 **반복자**·**포인터**·참조를 무효화한다. [U16-024] (K 1.7.4 반복자 무효화, PDF p.51)

std::list와 std::forward_list의 삽입은 **반복자**의 유효성에 영향을 미치지 않으며 삭제 시에는 삭제된 원소의 **반복자**가 무효화된다. [U16-025] (K 1.7.4 반복자 무효화, PDF p.51)

1.8~1.10 덱·어댑터·벤치마킹

std::deque은 양방향 큐이며 양 끝의 삽입·삭제와 임의 접근은 O(1), 중간의 삽입·삭제는 O(n)이다. [U16-026] (K 1.8.1 덱의 구조, PDF p.53)

컨테이너 어댑터는 기존 컨테이너를 감싸 해당 자료 구조에 필요한 인터페이스만 제공한다. [U16-027] (K 1.9 컨테이너 어댑터, PDF p.57)

스택은 LIFO(Last In First Out, 후입선출) 구조이며 한쪽 끝에서만 데이터 삽입·삭제를 수행한다. [U16-028] (K 1.9 컨테이너 어댑터, PDF p.56)

std::queue는 FIFO(First In First Out, 선입선출) 구조이고 std::deque을 기본 컨테이너로 사용한다. [U16-029] (K 1.9.2 std::queue, PDF p.58)

std::priority_queue는 기본적으로 std::vector와 std::less를 사용하며 **최대 힙**으로 생성된다. [U16-030] (K 1.9.3 std::priority_queue, PDF p.58-59)

STL은 **스택**·큐·우선순위 큐 어댑터에 **반복자**를 지원하지 않는다. [U16-031] (K 1.9.4 어댑터 반복자, PDF p.59)

벤치마킹은 통계 데이터를 기반으로 더 나은 접근 방식을 결정하는 방법이다. [U16-032] (K 1.10 벤치마킹, PDF p.59)

동작 방식·절차

연결 리스트 중간에 새 원소 삽입

1. 새로운 노드를 생성한다. [U16-033] (K 1.2.2 연결된 자료 구조, PDF p.15)
2. 새 노드의 next가 다음 노드를 가리키게 한다. [U16-034] (K 1.2.2 연결된 자료 구조, PDF p.15)
3. 이전 노드의 next가 새 노드를 가리키도록 설정한다. [U16-035] (K 1.2.2 연결된 자료 구조, PDF p.15)

벤치마킹으로 구현 방법 선택

1. 실제 모델과 비슷한 작은 프로토타입을 각 컨테이너로 구현한다. [U16-036] (K 1.10 벤치마킹, PDF p.60)
2. 작업을 여러 번 반복하여 수행 시간을 측정하고 비교한다. [U16-037] (K 1.10 벤치마킹, PDF p.60)
3. 성능 테스트에서 더 나은 결과를 내는 구현 방법을 선택한다. [U16-038] (K 1.10 벤치마킹, PDF p.60)

주요 문법·표·수식

큐 사용 예제 — 원문 발췌 (원문 발췌)

```
std::queue<int> q;
q.push(1);
q.push(2);
q.push(3);
q.pop();
q.push(4);
```

[U16-039] (K 1.9.2 std::queue, PDF p.58)

연속된 자료 구조의 i번째 원소 주소: BA + i * sizeof(type); BA: 시작 주소, sizeof(type): 원소 하나에 필요한 메모리 크기 [U16-040] (K 1.2.1 연속된 자료 구조, PDF p.13)

그림 1-5 배열과 연결 리스트

연산	배열	연결 리스트
임의 접근 [U16-041] (K 1.2.3 비교, PDF p.16)	O(1) [U16-042] (K 1.2.3 비교, PDF p.16)	O(n) [U16-043] (K 1.2.3 비교, PDF p.16)
맨 뒤에 원소 삽입 [U16-044] (K 1.2.3 비교, PDF p.16)	O(1) [U16-042] (K 1.2.3 비교, PDF p.16)	O(1) [U16-042] (K 1.2.3 비교, PDF p.16)
중간에 원소 삽입 [U16-045] (K 1.2.3 비교, PDF p.16)	O(n) [U16-043] (K 1.2.3 비교, PDF p.16)	O(1) [U16-042] (K 1.2.3 비교, PDF p.16)

핵심 용어 표

용어	원문에 나온 정의·설명
시간 복잡도	**시간 복잡도**는 특정 작업을 수행하는 데 걸리는 시간을 데이터 크기에 대한 수식으로 표현하는 방식이다. [U16-003] (K 1.2 연속된 자료 구조와 연결된 자료 구조, PDF p.12)
연속된 자료 구조	**연속된 자료 구조**는 모든 원소를 단일 메모리 청크에 저장한다. [U16-004] (K 1.2.1 연속된 자료 구조, PDF p.12)
연결 리스트	**연결 리스트**의 노드는 데이터와 다음 노드를 가리키는 next **포인터**를 가지며 마지막 노드의 next는 NULL이다. [U16-007] (K 1.2.2 연결된 자료 구조, PDF p.14)
std::array	**std::array**는 원소 타입과 **배열** 크기를 매개**변수**로 사용하는 **클래스** 템플릿이며 메모리를 자동으로 할당하고 해제한다. [U16-010] (K 1.3 std::array, PDF p.17)
반복자	**반복자**는 **포인터**와 비슷하지만 **STL** 컨테이너에 대해 공통의 인터페이스를 제공한다. [U16-021] (K 1.6 반복자, PDF p.39)
컨테이너 어댑터	**컨테이너 어댑터**는 기존 컨테이너를 감싸 해당 자료 구조에 필요한 인터페이스만 제공한다. [U16-027] (K 1.9 컨테이너 어댑터, PDF p.57)
스택	**스택**은 LIFO(Last In First Out, 후입선출) 구조이며 한쪽 끝에서만 데이터 삽입·삭제를 수행한다. [U16-028] (K 1.9 컨테이너 어댑터, PDF p.56)
벤치마킹	**벤치마킹**은 통계 데이터를 기반으로 더 나은 접근 방식을 결정하는 방법이다. [U16-032] (K 1.10 벤치마킹, PDF p.59)

단원 요약

요약 기준: 원문 나가며

데이터에 적용할 연산과 사용 빈도에 따라 다양한 자료 구조를 비교한다. [U16-046] (K 1.11 나가며, PDF p.61)

컨테이너 어댑터는 의도를 파악하기 쉬운 코드를 작성하고 코딩 실수를 줄이는 데 도움이 된다. [U16-047] (K 1.11 나가며, PDF p.61)

이 장에서 설명한 구조는 논리적 관점에서 선형이다. [U16-048] (K 1.11 나가며, PDF p.61)

17 트리, 힙, 그래프

K PDF 62-105

다루는 내용

비선형 자료 구조가 필요한 경우를 판별하고 트리 구조를 구현·순회한다. [U17-001] (K 2장 학습 목표, PDF p.63)

그래프 구조를 구현하고 상황에 맞는 방법으로 **그래프**를 표현한다. [U17-002] (K 2장 학습 목표, PDF p.63)

핵심 개념

2.2~2.3 비선형 문제와 트리

데이터가 저장된 부분을 노드(node), 노드 사이를 잇는 선을 에지(edge, 간선)라고 한다. [U17-003] (K 2.2.1 계층적 문제, PDF p.65)

트리는 노드와 에지로 계층을 구성하며 중심이 되는 **루트 노드**는 보통 맨 위에 표시한다. [U17-004] (K 2.3 트리: 상하 반전된 형태, PDF p.66)

전위 순회는 현재 노드, 왼쪽 하위 노드, 오른쪽 하위 노드의 순서로 재귀적으로 방문한다. [U17-005] (K 2.3.2 트리 순회, PDF p.70)

중위 순회는 왼쪽 노드, 현재 노드, 오른쪽 노드의 순서로 방문한다. [U17-006] (K 2.3.2 트리 순회, PDF p.71)

후위 순회는 두 자식 노드를 먼저 방문한 후 현재 노드를 방문한다. [U17-007] (K 2.3.2 트리 순회, PDF p.71)

레벨 순서 순회는 맨 위 레벨부터 아래 레벨까지 왼쪽에서 오른쪽으로 방문한다. [U17-008] (K 2.3.2 트리 순회, PDF p.71)

2.4 다양한 트리 구조

이진 트리의 각 노드는 최대 두 개의 자식 노드를 가진다. [U17-009] (K 2.4 다양한 트리 구조, PDF p.73)

BST는 왼쪽 노드 ≤ 부모 노드 ≤ 오른쪽 노드의 관계를 가진다. [U17-010] (K 2.4.1 이진 검색 트리, PDF p.74)

BST의 모양은 원소 삽입 순서에 따라 결정되므로 O(log N) 검색이 항상 성립하지는 않는다. [U17-011] (K 2.4.2 트리 연산의 시간 복잡도, PDF p.78)

BST의 레벨 수를 **높이**(height)라고 하며 원소 검색의 실제 **시간 복잡도**는 O(**높이**)이다. [U17-012] (K 2.4.4 균형 트리, PDF p.85)

원소 삽입·삭제 후 트리 구성을 조정하여 편향성을 줄인 이진 검색 트리를 **높이**-균형 **BST**라고 한다. [U17-013] (K 2.4.4 균형 트리, PDF p.85)

N-항 트리는 각 노드가 N개의 자식을 가질 수 있으며 자식 노드를 벡터로 저장할 수 있다. [U17-014] (K 2.4.5 N-항 트리, PDF p.86)

2.5 힙과 불변성

힙은 최대 원소 접근 O(1), 원소 삽입과 최대 원소 삭제 O(log N)을 만족해야 한다. [U17-015] (K 2.5 힙, PDF p.88)

힙은 완전 **이진 트리**를 사용하며 루트부터 레벨 순서로 **배열**이나 벡터에 저장할 수 있다. [U17-016] (K 2.5 힙, PDF p.88)

부모 노드가 i번째 **배열** 원소이면 자식 노드는 2*i + 1, 2*i + 2이며 자식 i의 부모는 (i - 1)/2이다. [U17-017] (K 2.5 힙, PDF p.89)

최대 힙은 최대 원소를 루트에 두고 부모 노드가 두 자식 노드보다 항상 커야 한다는 불변성을 유지한다. [U17-018] (K 2.5 힙, PDF p.89)

모든 비교 연산을 반대로 설정하면 최소 원소에 빠르게 접근하는 최소 힙을 만들 수 있다. [U17-019] (K 2.5 힙, PDF p.89)

힙 생성 **알고리즘**은 아래쪽 서브 트리부터 위로 올라가며 불변성을 맞추고 O(N) 시간에 힙을 **초기화**한다. [U17-020] (K 2.5.1 힙 연산, PDF p.91)

std::make_heap()은 **배열**이나 벡터의 **반복자**를 인자로 받아 힙을 구성한다. [U17-021] (K 2.5.1 힙 연산, PDF p.91)

2.6 그래프와 표현 방법

트리에서는 한 노드에서 다른 노드로 이동하는 경로가 하나만 존재하여 순환적 종속성을 표현할 수 없다. [U17-022] (K 2.6 그래프, PDF p.96)

그래프는 노드 데이터와 노드 사이의 에지 데이터를 함께 저장한다. [U17-023] (K 2.6 그래프, PDF p.96)

에지에 가중치나 더 많은 정보를 부여한 **그래프**를 가중 **그래프**라고 한다. [U17-024] (K 2.6 그래프, PDF p.96)

무방향 **그래프**의 에지는 양방향이며 방향 **그래프**에서 양방향을 표현하려면 서로 반대 방향의 두 에지를 사용한다. [U17-025] (K 2.6 그래프, PDF p.96)

인접 행렬은 N개 노드를 N×N **배열**로 표현하고 **배열** 원소에 에지 가중치를 저장하며 에지가 없으면 -1을 설정한다. [U17-026] (K 2.6.1 인접 행렬로 그래프 표현하기, PDF p.97)

인접 리스트는 각 노드에 직접 연결된 노드의 ID만 저장한다. [U17-027] (K 2.6.3 인접 리스트로 그래프 표현하기, PDF p.101)

동작 방식·절차

레벨 순서 순회

1. **루트 노드**를 방문하고 자식 노드를 큐에 추가한다. [U17-028] (K 2.3.3 연습 문제 8, PDF p.73)

2. 현재 레벨 방문이 끝나면 큐에서 다음 레벨 노드를 꺼내 방문한다. [U17-029] (K 2.3.3 연습 문제 8, PDF p.73)

3. 큐가 완전히 빌 때까지 반복한다. [U17-030] (K 2.3.3 연습 문제 8, PDF p.73)

최대 힙에 원소 삽입

1. **배열**의 맨 마지막 위치에 새 원소를 추가하여 완전 **이진 트리**를 유지한다. [U17-031] (K 2.5.1 힙 연산, PDF p.89)

2. 새 원소의 부모 노드가 더 작으면 서로 교환한다. [U17-032] (K 2.5.1 힙 연산, PDF p.89)

3. 전체 트리가 불변 조건을 만족하도록 교환을 반복한다. [U17-033] (K 2.5.1 힙 연산, PDF p.90)

최대 힙에서 원소 삭제

1. 루트와 마지막 노드를 교환한 후 마지막 노드를 삭제한다. [U17-034] (K 2.5.1 힙 연산, PDF p.90)

2. 루트와 두 자식 중 더 큰 노드를 비교·교환한다. [U17-035] (K 2.5.1 힙 연산, PDF p.90-91)

3. 불변성이 깨진 서브 트리에서도 재귀적으로 교환을 반복한다. [U17-036] (K 2.5.1 힙 연산, PDF p.91)

주요 문법·표·수식

N-항 트리 노드 — 원문 발췌 (원문 발췌)

```
struct nTree
{
    int data;
    std::vector<nTree*> children;
};
```

[U17-037] (K 2.4.5 N-항 트리, PDF p.86)

중위 순회 — 원문 발췌 (원문 발췌)

```
static void inOrder(node* start)
{
    if (!start)
        return;
    inOrder(start->first);
    std::cout << start->position << ", ";
    inOrder(start->second);
}
```

[U17-038] (K 2.3.2 트리 순회, PDF p.71)

BST 검색의 재귀식과 조건: T(N) = T(N/2) + 1; 원문은 이어서 검색 범위가 절반으로 줄어드는 가정이 항상 성립하지 않는다고 밝힌다. [U17-039] (K 2.4.2 트리 연산의 시간 복잡도, PDF p.77-78)

배열에 저장한 힙의 인덱스: 자식: 2*i + 1, 2*i + 2 / 부모: (i - 1)/2; 부모 또는 자식이 i번째 인덱스일 때 [U17-040] (K 2.5 힙, PDF p.89)

트리 순회 순서

방법	순서
전위 순회 [U17-041] (K 2.3.2, PDF p.70)	현재 → 왼쪽 → 오른쪽 [U17-042] (K 2.3.2, PDF p.70)
중위 순회 [U17-043] (K 2.3.2, PDF p.71)	왼쪽 → 현재 → 오른쪽 [U17-044] (K 2.3.2, PDF p.71)
후위 순회 [U17-045] (K 2.3.2, PDF p.71)	왼쪽 → 오른쪽 → 현재 [U17-046] (K 2.3.2, PDF p.71)

핵심 용어 표

용어	원문에 나온 정의·설명
노드·에지	데이터가 저장된 부분을 노드(node), 노드 사이를 잇는 선을 에지(edge, 간선)라고 한다. [U17-003] (K 2.2.1 계층적 문제, PDF p.65)
루트 노드	트리는 노드와 에지로 계층을 구성하며 중심이 되는 **루트 노드**는 보통 맨 위에 표시한다. [U17-004] (K 2.3 트리: 상하 반전된 형태, PDF p.66)
이진 트리	**이진 트리**의 각 노드는 최대 두 개의 자식 노드를 가진다. [U17-009] (K 2.4 다양한 트리 구조, PDF p.73)
BST	**BST**는 왼쪽 노드 ≤ 부모 노드 ≤ 오른쪽 노드의 관계를 가진다. [U17-010] (K 2.4.1 이진 검색 트리, PDF p.74)
높이	**BST**의 레벨 수를 **높이**(height)라고 하며 원소 검색의 실제 **시간 복잡도**는 O(**높이**)이다. [U17-012] (K 2.4.4 균형 트리, PDF p.85)
N-항 트리	**N-항 트리**는 각 노드가 N개의 자식을 가질 수 있으며 자식 노드를 벡터로 저장할 수 있다. [U17-014] (K 2.4.5 N-항 트리, PDF p.86)
최대 힙	**최대 힙**은 최대 원소를 루트에 두고 부모 노드가 두 자식 노드보다 항상 커야 한다는 불변성을 유지한다. [U17-018] (K 2.5 힙, PDF p.89)
인접 리스트	**인접 리스트**는 각 노드에 직접 연결된 노드의 ID만 저장한다. [U17-027] (K 2.6.3 인접 리스트로 그래프 표현하기, PDF p.101)

단원 요약

요약 기준: 원문 나가며

이 장은 트리와 **그래프**라는 두 자료 구조를 중점적으로 다뤘다. [U17-047] (K 2.7 나가며, PDF p.105)

다양한 상황에서 사용할 수 있는 여러 종류의 트리를 설명했다. [U17-048] (K 2.7 나가며, PDF p.105)

프로그램에서 이들 자료 구조를 표현하는 여러 방법을 검토했다. [U17-049] (K 2.7 나가며, PDF p.105)

18 해시 테이블과 블룸 필터

K PDF 106-145

다루는 내용

대용량 데이터의 **룩업** 문제를 이해하고 결정적·비결정적 솔루션을 구분한다. [U18-001] (K 3장 학습 목표, PDF p.107)

상황에 맞는 효율적인 **룩업**과 C++ **STL**에서 제공되는 솔루션을 구현한다. [U18-002] (K 3장 학습 목표, PDF p.107)

핵심 개념

3.1~3.2 룩업과 해싱

룩업(**lookup**, 조회)은 특정 원소의 존재를 확인하거나 특정 키에 해당하는 값을 찾는 작업이다. [U18-003] (K 3.1 들어가며, PDF p.107)

해싱은 데이터를 가급적 고유한 숫자 값으로 표현하고 같은 숫자를 이용하여 유무를 확인하거나 원본 데이터를 추출하는 작업이다. [U18-004] (K 3.2 해시 테이블, PDF p.108)

해시 함수는 데이터로부터 고유한 숫자 값을 계산하는 함수이며 해시 값은 그 함수가 반환한 숫자 값이다. [U18-005] (K 3.2 해시 테이블·3.2.1 해싱, PDF p.108-109)

모듈로 함수 x % n은 0부터 n - 1 사이의 정수를 반환한다. [U18-006] (K 3.2.1 해싱, PDF p.109)

충돌은 서로 다른 키에 대해 해시 함수가 같은 해시 값을 반환하는 현상이다. [U18-007] (K 3.2.1 해싱, PDF p.109)

3.3.1~3.3.3 충돌 처리와 부하율

체이닝은 해시 테이블의 특정 위치에 **연결 리스트**를 두어 **충돌**한 키들을 함께 저장한다. [U18-008] (K 3.3.1 체이닝, PDF p.114)

체이닝에서 삽입은 O(1)이지만 모든 키가 같은 해시 값을 가지면 **룩업**은 O(N)이 된다. [U18-009] (K 3.3.2 연습 문제 14, PDF p.118)

부하율은 해시 테이블의 각 리스트에 저장되는 키의 평균 개수이다. [U18-010] (K 3.3.2 연습 문제 14, PDF p.118)

부하율은 해시 테이블 성능을 결정하는 유일한 지표가 아니며 해시 값이 골고루 분포되어야 한다. [U18-011] (K 3.3.2 연습 문제 14, PDF p.119)

열린 주소 지정은 모든 원소를 해시 테이블 내부에 저장하고 **충돌** 시 다른 빈 위치를 탐색한다. [U18-012] (K 3.3.3 열린 주소 지정, PDF p.119)

선형 탐색은 **충돌**한 위치에서 다음 셀로 하나씩 이동하여 빈 셀을 찾는다. [U18-013] (K 3.3.3 열린 주소 지정, PDF p.120)

이차함수 탐색은 선형 방정식 대신 이차 방정식을 사용하여 이동 폭을 증가시킨다. [U18-014] (K 3.3.3 열린 주소 지정, PDF p.121)

3.3.4 뻐꾸기 해싱

뻐꾸기 해싱은 크기가 같고 해시 함수가 서로 다른 두 해시 테이블을 사용한다. [U18-015] (K 3.3.4 뻐꾸기 해싱, PDF p.122)

각 원소는 두 위치 중 하나에 저장되며 삽입 후 다른 위치로 이동할 수 있다. [U18-016] (K 3.3.4 뻐꾸기 해싱, PDF p.122)

룩업은 저장 가능한 두 위치만 확인하므로 **시간 복잡도**가 항상 O(1)이다. [U18-017] (K 3.3.4 뻐꾸기 해싱, PDF p.122)

삽입 중 순환이 발생하면 새 해시 함수로 재**해싱**해야 하며 여러 번 필요할 수 있다. [U18-018] (K 3.3.4 뻐꾸기 해싱, PDF p.123)

원문은 높은 성능을 보장하려면 **부하율**을 0.5보다 작게 설정해야 한다고 설명한다. [U18-019] (K 3.3.4 뻐꾸기 해싱, PDF p.123)

3.4 C++ 해시 테이블

std::unordered_set은 키만, std::unordered_map은 키와 값을 함께 저장한다. [U18-020] (K 3.4 C++ 해시 테이블, PDF p.131)

기본 최대 **부하율**은 1이며 max_load_factor(float)로 최대 한계치를 변경할 수 있다. [U18-021] (K 3.4 C++ 해시 테이블, PDF p.131)

std::unordered_map의 [] 연산자는 참조를 반환하며 키가 없으면 기본값을 추가하여 반환한다. [U18-022] (K 3.4.1 연습 문제 16, PDF p.135)

std::unordered_set과 std::unordered_map은 중복 키를 허용하지 않으며 중복 저장에는 unordered_multiset 또는 unordered_multimap을 사용한다. [U18-023] (K 3.4.1 연습 문제 16, PDF p.135)

3.5 블룸 필터

블룸 필터는 공간 효율이 높지만 거짓-긍정(false positive)이 나올 수 있으며 거짓-부정(false negative)은 없다. [U18-024] (K 3.5 블룸 필터, PDF p.138)

블룸 필터의 긍정 답변은 실제로 있을 수도 없을 수도 있음을 뜻하며 부정 답변은 확실히 없음을 뜻한다. [U18-025] (K 3.5 블룸 필터, PDF p.138)

블룸 필터는 실제 값 대신 부울 타입 **배열**을 사용하고 여러 해시 함수에 대응하는 비트를 설정한다. [U18-026] (K 3.5 블룸 필터, PDF p.138)

원소가 많아질수록 거짓-긍정이 발생할 가능성이 증가한다. [U18-027] (K 3.5 블룸 필터, PDF p.138)

부울 **배열**이 모두 1이면 포화 **상태**가 되어 **룩업**은 항상 true를 반환하고 삽입은 필터 **상태**를 바꾸지 못한다. [U18-028] (K 3.5 블룸 필터, PDF p.138)

동작 방식·절차

뻐꾸기 해싱 삽입

1. 첫 번째 테이블에서 새 원소를 저장할 위치가 비었는지 검사한다. [U18-029] (K 3.3.4 뻐꾸기 해싱, PDF p.122)

2. 이미 원소가 있으면 새 원소를 저장하고 기존 원소를 두 번째 테이블로 옮긴다. [U18-030] (K 3.3.4 뻐꾸기 해싱, PDF p.122)

3. 빈 셀이 나타날 때까지 다른 테이블로 옮기는 과정을 반복하고 순환 시 재**해싱**한다. [U18-031] (K 3.3.4 뻐꾸기 해싱, PDF p.122-123)

블룸 필터 삽입·룩업

1. 삽입 시 모든 해시 함수 값을 계산하고 대응 비트를 1로 설정한다. [U18-032] (K 3.5 블룸 필터, PDF p.138)

2. 룩업 시 모든 해시 함수 값을 계산하여 해당 비트를 검사한다. [U18-033] (K 3.5 블룸 필터, PDF p.138)

3. 모든 비트가 1이면 true, 1이 아닌 비트가 하나라도 있으면 false를 반환한다. [U18-034] (K 3.5 블룸 필터, PDF p.138)

주요 문법·표·수식

블룸 필터의 해시 함수 — 원문 발췌 (원문 발췌)

```
int hash(int num, int key)
{
    switch (num)
    {
    case 0: return key % nBits;
    case 1: return (key / 7) % nBits;
    case 2: return (key / 11) % nBits;
    }
    return 0;
}
```

[U18-035] (K 3.5.1 연습 문제 17, PDF p.140)

부하율: **부하율** = 전체 키 개수 / 해시 테이블 크기; 각 리스트에 저장되는 키의 평균 개수 [U18-036] (K 3.3.2 연습 문제 14, PDF p.118)

STL 해시 컨테이너의 저장 대상

컨테이너	저장 대상
std::unordered_set [U18-037] (K 3.4, PDF p.131)	키 [U18-038] (K 3.4, PDF p.131)
std::unordered_map [U18-039] (K 3.4, PDF p.131)	키와 값 [U18-040] (K 3.4, PDF p.131)
unordered_multiset / unordered_multimap [U18-041] (K 3.4.1, PDF p.135)	중복 저장 허용 [U18-042] (K 3.4.1, PDF p.135)

핵심 용어 표

용어	원문에 나온 정의·설명
룩업	**룩업**(**lookup**, 조회)은 특정 원소의 존재를 확인하거나 특정 키에 해당하는 값을 찾는 작업이다. [U18-003] (K 3.1 들어가며, PDF p.107)
해싱	**해싱**은 데이터를 가급적 고유한 숫자 값으로 표현하고 같은 숫자를 이용하여 유무를 확인하거나 원본 데이터를 추출하는 작업이다. [U18-004] (K 3.2 해시 테이블, PDF p.108)
해시 함수·해시 값	해시 함수는 데이터로부터 고유한 숫자 값을 계산하는 함수이며 해시 값은 그 함수가 반환한 숫자 값이다. [U18-005] (K 3.2 해시 테이블·3.2.1 해싱, PDF p.108-109)
충돌	**충돌**은 서로 다른 키에 대해 해시 함수가 같은 해시 값을 반환하는 현상이다. [U18-007] (K 3.2.1 해싱, PDF p.109)
체이닝	**체이닝**은 해시 테이블의 특정 위치에 **연결 리스트**를 두어 **충돌**한 키들을 함께 저장한다. [U18-008] (K 3.3.1 체이닝, PDF p.114)
부하율	**부하율**은 해시 테이블의 각 리스트에 저장되는 키의 평균 개수이다. [U18-010] (K 3.3.2 연습 문제 14, PDF p.118)
열린 주소 지정	**열린 주소 지정**은 모든 원소를 해시 테이블 내부에 저장하고 **충돌** 시 다른 빈 위치를 탐색한다. [U18-012] (K 3.3.3 열린 주소 지정, PDF p.119)
뻐꾸기 해싱	**뻐꾸기 해싱**은 크기가 같고 해시 함수가 서로 다른 두 해시 테이블을 사용한다. [U18-015] (K 3.3.4 뻐꾸기 해싱, PDF p.122)
블룸 필터	**블룸 필터**는 공간 효율이 높지만 거짓-긍정(false positive)이 나올 수 있으며 거짓-부정(false negative)은 없다. [U18-024] (K 3.5 블룸 필터, PDF p.138)

단원 요약

요약 기준: 원문 나가며

룩업 문제에서는 필요에 따라 결정적 솔루션과 확률적 솔루션을 선택할 수 있다. [U18-043] (K 3.6 나가며, PDF p.144)

C++의 **해싱** 관련 컨테이너를 이용하면 해시 테이블을 직접 작성하지 않아도 된다. [U18-044] (K 3.6 나가며, PDF p.144)

컨테이너에서 find()를 많이 사용한다면 **룩업** 기반 솔루션을 고려한다. [U18-045] (K 3.6 나가며, PDF p.144)

19 분할 정복

K PDF 146-193

다루는 내용

분할 정복 설계 패러다임과 **병합 정렬·퀵 정렬·선형 시간 선택**을 구현한다. [U19-001] (K 4장 학습 목표, PDF p.147)

맵**리듀스** 모델과 멀티스레드를 사용하는 C++ 맵**리듀스** 라이브러리를 다룬다. [U19-002] (K 4장 학습 목표, PDF p.147)

핵심 개념

4.1~4.3 알고리즘과 분할 정복

알고리즘은 입력을 받아 일련의 변환을 수행하여 결과를 출력한다. [U19-003] (K 4.1 들어가며, PDF p.147)

분할 정복은 문제를 작은 부분 문제로 나누고 각각의 해답을 구한 뒤 합쳐 전체 해답을 얻는다. [U19-004] (K 4.1 들어가며, PDF p.149)

선형 검색은 전체 원소를 방문하여 대상을 찾으며 정렬 여부와 관계없이 동작하고 **시간 복잡도**는 O(N)이다. [U19-005] (K 4.2 이진 검색, PDF p.150)

이진 검색은 시퀀스가 정렬되어 있다는 사실을 이용하여 가운데 원소와 대상을 비교하고 검색 범위를 수정한다. [U19-006] (K 4.2 이진 검색, PDF p.150)

분할 정복은 분할(divide), 정복(conquer), 결합(combine)의 세 단계로 이루어진다. [U19-007] (K 4.3 분할 정복 이해하기, PDF p.157)

이진 검색은 작은 범위에서 구한 해답이 전체 해답과 같으므로 일반적인 결합 과정이 필요하지 않다. [U19-008] (K 4.3 분할 정복 이해하기, PDF p.158)

4.4 정렬·선택 알고리즘

정렬 **알고리즘**은 여러 데이터 타입, 많은 데이터, 빠른 동작이라는 요구 사항을 고려한다. [U19-009] (K 4.4 분할 정복을 이용한 정렬 알고리즘, PDF p.158)

외부 정렬은 전체 데이터가 메인 메모리에 상주하지 않은 **상태**에서 수행되는 정렬이다. [U19-010] (K 4.4 분할 정복을 이용한 정렬 알고리즘, PDF p.159)

병합 정렬은 집합을 작은 부분집합으로 나누어 각각을 정렬한 뒤 순서를 유지하며 합친다. [U19-011] (K 4.4.1 병합 정렬, PDF p.159)

병합 정렬의 분할은 각 부분 **배열**이 하나의 원소만 가질 때 멈춘다. [U19-012] (K 4.4.1 병합 정렬, PDF p.160)

퀵 정렬의 목표는 평균 실행 시간을 줄이는 것이며 핵심 연산은 분할이다. [U19-013] (K 4.4.3 퀵 정렬, PDF p.163)

퀵 정렬의 분할은 피벗 P를 기준으로 작거나 같은 원소의 L과 큰 원소의 R을 만들고 L, P, R 순서로 재구성한다. [U19-014] (K 4.4.3 퀵 정렬, PDF p.163)

분할 후 피벗은 **배열** 전체를 정렬했을 때 실제로 있어야 하는 위치로 이동한다. [U19-015] (K 4.4.3 퀵 정렬, PDF p.164)

퀵 정렬은 두 개 이상의 원소가 있는 **배열**을 분할하고 L과 R에 재귀적으로 같은 작업을 적용한다. [U19-016] (K 4.4.3 퀵 정렬, PDF p.164)

그림 4-9에서 **병합 정렬**의 최선·평균·최악 **시간 복잡도**는 모두 O(nlogn)이고, **퀵 정렬**은 최선·평균 O(nlogn), 최악 $O(n^2)$이다. [U19-017] (K 4.4.4 그림 4-9, PDF p.168)

선형 시간 선택은 정렬되지 않은 집합에서 i번째로 작은 원소를 O(n)에 찾는 문제를 다룬다. [U19-018] (K 4.4.6 선형 시간 선택, PDF p.170)

원문의 **선형 시간 선택**은 다섯 원소씩 나눈 부분 벡터의 중앙값을 모으고 그 중앙값을 피벗으로 사용한다. [U19-019] (K 4.4.6 선형 시간 선택, PDF p.170-171)

4.4 정렬·선택 알고리즘 (계속)

선형 시간 선택은 분할된 벡터 중 하나에만 재귀 호출하며 벡터 크기는 매번 적어도 30% 줄어든다. [U19-020] (K 4.4.6~4.4.7 선형 시간 선택, PDF p.177)

4.5 표준 라이브러리 함수

std::stable_sort()는 순위가 같은 원소의 원본 순서를 유지하며 정렬한다. [U19-021] (K 4.5 그림 4-13, PDF p.178)

std::partial_sort()는 컨테이너의 일부 구간에 대해 정렬한다. [U19-022] (K 4.5 그림 4-13, PDF p.178)

std::transform()은 모든 원소에 함수를 적용하고 std::accumulate()는 누적 연산을 수행한다. [U19-023] (K 4.5 그림 4-13, PDF p.178)

std::reduce()는 지정한 범위의 원소에 **리듀스** 연산을 수행하고 결과를 반환한다. [U19-024] (K 4.5 그림 4-13, PDF p.178)

4.6 맵리듀스

맵은 컨테이너 C의 모든 원소에 함수 f(x)를 적용하는 연산이다. [U19-025] (K 4.6.1 맵과 리듀스 추상화, PDF p.180)

리듀스는 모든 원소 x에 함수 f(acc,x)를 적용하여 하나의 값으로 축약하는 연산이다. [U19-026] (K 4.6.1 맵과 리듀스 추상화, PDF p.180)

맵**리듀스** 모델은 맵 단계에서 중간 키·값 쌍을 만들고 **리듀스** 단계에서 결합하여 최종 결과를 생성한다. [U19-027] (K 4.6.3 맵리듀스 프레임워크를 이용한 부분 통합, PDF p.183)

워드카운트는 여러 텍스트 파일에서 각각의 단어가 나타나는 횟수를 구하는 문제이다. [U19-028] (K 4.6.5 실습 문제 10, PDF p.188)

동작 방식·절차

분할 정복의 세 단계

1. 동일한 방식으로 해결할 수 있는 여러 부분 문제로 분할한다. [U19-029] (K 4.3 분할 정복 이해하기, PDF p.157)

2. 각 부분 문제의 해답을 구한다. [U19-030] (K 4.3 분할 정복 이해하기, PDF p.157)

3. 각 해답을 결합하여 전체 문제의 해답을 구한다. [U19-031] (K 4.3 분할 정복 이해하기, PDF p.157)

퀵 정렬의 분할·재귀

1. 퀵 정렬의 분할은 피벗 P를 기준으로 작거나 같은 원소의 L과 큰 원소의 R을 만들고 L, P, R 순서로 재구성한다. [U19-014] (K 4.4.3 퀵 정렬, PDF p.163)

2. 분할 후 피벗은 **배열** 전체를 정렬했을 때 실제로 있어야 하는 위치로 이동한다. [U19-015] (K 4.4.3 퀵 정렬, PDF p.164)

3. 퀵 정렬은 두 개 이상의 원소가 있는 **배열**을 분할하고 L과 R에 재귀적으로 같은 작업을 적용한다. [U19-016] (K 4.4.3 퀵 정렬, PDF p.164)

선형 시간 선택

1. 입력 벡터를 다섯 개 이하의 원소를 갖는 부분 벡터들로 나누고 각각 정렬한다. [U19-032] (K 4.4.6 선형 시간 선택, PDF p.170)

2. 각 부분 벡터의 중앙값들을 모아 집합 M을 만들고 M의 중앙값 q를 찾는다. [U19-033] (K 4.4.6 선형 시간 선택, PDF p.171)

3. q를 피벗으로 전체 벡터를 L과 R로 나눈다. [U19-034] (K 4.4.6 선형 시간 선택, PDF p.171)

4. 피벗의 순위와 i를 비교하여 반환하거나 L 또는 R에서 반복한다. [U19-035] (K 4.4.6 선형 시간 선택, PDF p.172)

주요 문법·표·수식

중앙값 위치 찾기 — 원문 발췌 (원문 발췌)

```
template<typename T>
auto find_median(typename std::vector<T>::iterator begin,
    typename std::vector<T>::iterator last)
{
    // 정렬
    quick_sort<T>(begin, last);
    // 가운데(median) 원소 반복자를 반환
    return begin + (std::distance(begin, last) / 2);
}
```

[U19-036] (K 4.4.7 연습 문제 21, PDF p.172)

선형 시간 선택의 복잡도: O(n); 원문의 다섯 원소 묶음과 중앙값 피벗을 사용하는 **알고리즘** [U19-037] (K 4.4.6~4.4.7 선형 시간 선택, PDF p.177)

그림 4-9 병합 정렬과 퀵 정렬의 점근적 시간 복잡도

알고리즘	최선의 경우	평균	최악의 경우
병합 정렬 [U19-038] (K 4.4.4 그림 4-9, PDF p.168)	O(nlogn) [U19-039] (K 4.4.4 그림 4-9, PDF p.168)	O(nlogn) [U19-039] (K 4.4.4 그림 4-9, PDF p.168)	O(nlogn) [U19-039] (K 4.4.4 그림 4-9, PDF p.168)
퀵 정렬 [U19-040] (K 4.4.4 그림 4-9, PDF p.168)	O(nlogn) [U19-039] (K 4.4.4 그림 4-9, PDF p.168)	O(nlogn) [U19-039] (K 4.4.4 그림 4-9, PDF p.168)	$O(n^2)$ [U19-041] (K 4.4.4 그림 4-9, PDF p.168)

핵심 용어 표

용어	원문에 나온 정의·설명
알고리즘	**알고리즘**은 입력을 받아 일련의 변환을 수행하여 결과를 출력한다. [U19-003] (K 4.1 들어가며, PDF p.147)
분할 정복	**분할 정복**은 문제를 작은 부분 문제로 나누고 각각의 해답을 구한 뒤 합쳐 전체 해답을 얻는다. [U19-004] (K 4.1 들어가며, PDF p.149)
외부 정렬	**외부 정렬**은 전체 데이터가 메인 메모리에 상주하지 않은 **상태**에서 수행되는 정렬이다. [U19-010] (K 4.4 분할 정복을 이용한 정렬 알고리즘, PDF p.159)
병합 정렬	**병합 정렬**은 집합을 작은 부분집합으로 나누어 각각을 정렬한 뒤 순서를 유지하며 합친다. [U19-011] (K 4.4.1 병합 정렬, PDF p.159)
퀵 정렬	**퀵 정렬**의 목표는 평균 실행 시간을 줄이는 것이며 핵심 연산은 분할이다. [U19-013] (K 4.4.3 퀵 정렬, PDF p.163)
선형 시간 선택	**선형 시간 선택**은 정렬되지 않은 집합에서 i번째로 작은 원소를 O(n)에 찾는 문제를 다룬다. [U19-018] (K 4.4.6 선형 시간 선택, PDF p.170)
맵	맵은 컨테이너 C의 모든 원소에 함수 f(x)를 적용하는 연산이다. [U19-025] (K 4.6.1 맵과 리듀스 추상화, PDF p.180)
리듀스	**리듀스**는 모든 원소 x에 함수 f(acc,x)를 적용하여 하나의 값으로 축약하는 연산이다. [U19-026] (K 4.6.1 맵과 리듀스 추상화, PDF p.180)
워드카운트	**워드카운트**는 여러 텍스트 파일에서 각각의 단어가 나타나는 횟수를 구하는 문제이다. [U19-028] (K 4.6.5 실습 문제 10, PDF p.188)

단원 요약

요약 기준: 원문 나가며

분할 정복을 **알고리즘** 설계 패러다임과 소프트웨어 확장 도구의 두 관점으로 살펴봤다. [U19-042] (K 4.7 나가며, PDF p.192)

분할 연산은 부분 정렬과 **선형 시간 선택** 같은 문제의 기초가 된다. [U19-043] (K 4.7 나가며, PDF p.192)

알고리즘 구현은 데이터 타입에 종속적이지 않아야 하며 템플릿과 표준 라이브러리 함수를 사용할 수 있다. [U19-044] (K 4.7 나가며, PDF p.192)

20 그리디 알고리즘

K PDF 194-231

다루는 내용

그리디 접근 방식과 **최적 부분 구조·그리디 선택** 속성을 판별한다. [U20-001] (K 5장 학습 목표, PDF p.195)

분할 가능 배낭·그래프 컬러링·디스조인트-셋을 이용한 크루스칼 MST를 구현한다. [U20-002] (K 5장 학습 목표, PDF p.195)

핵심 개념

5.1~5.2 그리디 접근과 최단 작업 우선

그리디 알고리즘은 매 단계에서 가장 좋아 보이는 해답을 선택하여 지역적 최적 해법으로 전역적 최적 해법을 구성한다. [U20-003] (K 5.1 들어가며, PDF p.195)

최단 작업 우선 스케줄링은 평균 대기 시간을 최소화하기 위해 일 처리가 빨리 끝나는 사람부터 배치한다. [U20-004] (K 5.2.1 최단 작업 우선 스케줄링, PDF p.197)

i번째 사람의 대기 시간은 첫 번째부터 i - 1번째 사람까지의 처리 시간 합이다. [U20-005] (K 5.2.1 최단 작업 우선 스케줄링, PDF p.197)

원문의 대기열은 순서 조정 후 평균 대기 시간이 15.25에서 8.875로 줄어든다. [U20-006] (K 5.2.2 연습문제 24, PDF p.198)

5.3.1~5.3.5 배낭·스케줄링·필요 속성

0-1 **배낭 문제**는 물건들의 무게 합이 T를 넘지 않으면서 가격 합이 최대가 되는 선택을 구한다. [U20-007] (K 5.3.1 0-1 배낭 문제, PDF p.200)

분할 가능 배낭 문제는 물건을 분할하여 일부만 배낭에 담을 수 있다. [U20-008] (K 5.3.2 분할 가능 배낭 문제, PDF p.201)

분할 가능 배낭은 단위 무게당 가격이 높은 순서로 선택한다. [U20-009] (K 5.3.2 분할 가능 배낭 문제, PDF p.201)

작업 스케줄링 실습은 종료 시간 순으로 정렬하여 가장 빨리 끝나는 작업을 고르고 겹치는 작업을 제거한다. [U20-010] (K 5.3.4 실습 문제 11, PDF p.206)

최적 부분 구조는 문제의 최적 솔루션이 부분 문제들의 최적 솔루션으로 구성되는 속성이다. [U20-011] (K 5.3.5 그리디 알고리즘의 요구 조건, PDF p.207)

그리디 선택은 지역적 최적 솔루션을 반복 선택하여 전체 최적 솔루션을 구할 수 있는 속성이다. [U20-012] (K 5.3.5 그리디 알고리즘의 요구 조건, PDF p.207)

원문은 **최적 부분 구조**와 **그리디 선택**을 모두 가진 문제에 그리디 방식으로 최적 솔루션을 구할 수 있다고 설명한다. [U20-013] (K 5.3.5 그리디 알고리즘의 요구 조건, PDF p.207)

5.3.6~5.3.8 MST와 디스조인트-셋

최소 신장 트리는 **그래프**의 모든 정점을 연결하며 에지 가중치 합이 최소인 트리이다. [U20-014] (K 5.3.6 최소 신장 트리 문제, PDF p.208)

크루스칼 **알고리즘**은 매번 최소 가중치 에지를 선택하고 사이클이 생기지 않을 때 솔루션에 추가한다. [U20-015] (K 5.3.6 최소 신장 트리 문제, PDF p.209)

디스조인트-셋 또는 유니온-파인드는 트리 형태의 원소로 구성된 포레스트이다. [U20-016] (K 5.3.7 디스조인트-셋 자료 구조, PDF p.211)

디스조인트-셋의 원소는 숫자 ID, 랭크와 부모 **포인터**를 가진다. [U20-017] (K 5.3.7 디스조인트-셋 자료 구조, PDF p.211)

make-set(x)는 랭크 0이고 부모가 자기 자신인 새 원소를 추가한다. [U20-018] (K 5.3.7 디스조인트-셋 자료 구조, PDF p.211)

find(x)는 부모 **포인터**를 따라 이동하여 트리의 루트를 반환한다. [U20-019] (K 5.3.7 디스조인트-셋 자료 구조, PDF p.211)

union(x, y)는 두 원소의 루트가 같으면 아무 작업도 하지 않고 다르면 높은 랭크 루트를 낮은 랭크 루트의 부모로 설정한다. [U20-020] (K 5.3.7 디스조인트-셋 자료 구조, PDF p.211)

크루스칼에서 에지 양 끝의 루트가 같으면 사이클이 발생하므로 그 에지를 MST에 추가하지 않는다. [U20-021] (K 5.3.7 디스조인트-셋 자료 구조, PDF p.213)

5.4 그래프 컬러링

그래프 컬러링은 인접한 정점끼리 같은 색을 갖지 않도록 모든 정점에 색상을 지정하는 문제이다. [U20-022] (K 5.4 그래프 컬러링, PDF p.222)

그래프 컬러링에서는 에지 가중치를 사용하지 않는다. [U20-023] (K 5.4 그래프 컬러링, PDF p.222)

원문은 최소 색상 수를 찾는 **그래프 컬러링**을 NP-완전 문제로 소개한다. [U20-024] (K 5.4 그래프 컬러링, PDF p.222)

차수(degree)는 정점에 연결된 에지의 개수이다. [U20-025] (K 5.4.2 실습 문제 12, PDF p.227)

웰시-포웰 **알고리즘**은 **차수**가 높은 정점부터 차례대로 **그래프 컬러링**을 수행한다. [U20-026] (K 5.4.2 실습 문제 12, PDF p.227)

동작 방식·절차

작업 스케줄링

1. 작업 리스트를 종료 시간으로 정렬한다. [U20-027] (K 5.3.4 실습 문제 11, PDF p.206)

2. 가장 빠른 종료 시간을 갖는 작업을 선택한다. [U20-028] (K 5.3.4 실습 문제 11, PDF p.206)

3. 선택된 작업과 겹치는 작업을 제거하고 남은 작업에 대해 선택을 반복한다. [U20-029] (K 5.3.4 실습 문제 11, PDF p.206)

크루스칼 MST와 사이클 검사

1. **그래프**의 모든 에지를 최소 힙에 넣고 모든 정점으로 **디스조인트-셋**을 **초기화**한다. [U20-030] (K 5.3.6~5.3.7, PDF p.208,213)

2. 최소 가중치 에지를 하나 꺼낸다. [U20-031] (K 5.3.6 최소 신장 트리 문제, PDF p.208)

3. 양 끝 정점의 루트가 같으면 버리고 다르면 트리를 병합하고 에지를 MST에 추가한다. [U20-032] (K 5.3.7 디스조인트-셋 자료 구조, PDF p.213)

웰시-포웰 알고리즘

1. 모든 정점을 **차수** 내림차순으로 정렬한다. [U20-033] (K 5.4.2 실습 문제 12, PDF p.227)

2. 색이 없는 첫 정점에 이웃이 사용하지 않은 색상 C를 지정한다. [U20-034] (K 5.4.2 실습 문제 12, PDF p.227)

3. 색이 없는 다른 정점도 이웃이 C를 갖지 않으면 C를 지정한다. [U20-035] (K 5.4.2 실습 문제 12, PDF p.227)

4. 색이 없는 정점이 남아 있으면 반복하고 없으면 종료한다. [U20-036] (K 5.4.2 실습 문제 12, PDF p.227)

주요 문법·표·수식

대기 시간 계산 — 원문 발췌 (원문 발췌)

```
template<typename T>
auto compute_waiting_times(std::vector<T>& service_times)
{
    std::vector<T> W(service_times.size());
    W[0] = 0;
    for (auto i = 1; i < service_times.size(); i++)
        W[i] = W[i - 1] + service_times[i - 1];
    return W;
}
```

[U20-037] (K 5.2.2 연습 문제 24, PDF p.198)

원문 대기 시간 계산식: W[0] = 0; W[i] = W[i - 1] + service_times[i - 1];; service_times: 일 처리 시간 벡터, W: 대기 시간 벡터 [U20-038] (K 5.2.2 연습 문제 24, PDF p.198)

배낭 문제의 물건 선택

문제	물건 분할
0-1 배낭 [U20-039] (K 5.3.1, PDF p.200)	물건들을 선택하여 담음 [U20-040] (K 5.3.1, PDF p.200)
분할 가능 배낭 [U20-041] (K 5.3.2, PDF p.201)	물건의 일부분만 담을 수 있음 [U20-042] (K 5.3.2, PDF p.201)

핵심 용어 표

용어	원문에 나온 정의·설명
그리디 알고리즘	**그리디 알고리즘**은 매 단계에서 가장 좋아 보이는 해답을 선택하여 지역적 최적 해법으로 전역적 최적 해법을 구성한다. [U20-003] (K 5.1 들어가며, PDF p.195)
최단 작업 우선 스케줄링	**최단 작업 우선 스케줄링**은 평균 대기 시간을 최소화하기 위해 일 처리가 빨리 끝나는 사람부터 배치한다. [U20-004] (K 5.2.1 최단 작업 우선 스케줄링, PDF p.197)
분할 가능 배낭	**분할 가능 배낭** 문제는 물건을 분할하여 일부만 배낭에 담을 수 있다. [U20-008] (K 5.3.2 분할 가능 배낭 문제, PDF p.201)
최적 부분 구조	**최적 부분 구조**는 문제의 최적 솔루션이 부분 문제들의 최적 솔루션으로 구성되는 속성이다. [U20-011] (K 5.3.5 그리디 알고리즘의 요구 조건, PDF p.207)
그리디 선택	**그리디 선택**은 지역적 최적 솔루션을 반복 선택하여 전체 최적 솔루션을 구할 수 있는 속성이다. [U20-012] (K 5.3.5 그리디 알고리즘의 요구 조건, PDF p.207)
최소 신장 트리	**최소 신장 트리**는 **그래프**의 모든 정점을 연결하며 에지 가중치 합이 최소인 트리이다. [U20-014] (K 5.3.6 최소 신장 트리 문제, PDF p.208)
디스조인트-셋	**디스조인트-셋** 또는 유니온-파인드는 트리 형태의 원소로 구성된 포레스트이다. [U20-016] (K 5.3.7 디스조인트-셋 자료 구조, PDF p.211)
그래프 컬러링	**그래프 컬러링**은 인접한 정점끼리 같은 색을 갖지 않도록 모든 정점에 색상을 지정하는 문제이다. [U20-022] (K 5.4 그래프 컬러링, PDF p.222)
차수	**차수**(degree)는 정점에 연결된 에지의 개수이다. [U20-025] (K 5.4.2 실습 문제 12, PDF p.227)

단원 요약

요약 기준: 원문 나가며

그리디 방식은 부분적인 최선의 선택이 전체 최선으로 이어지는 문제에 적용할 수 있다. [U20-043] (K 5.5 나가며, PDF p.230)

최적 부분 구조와 **그리디 선택**을 크루스칼 MST로 살펴보며 **디스조인트-셋**을 설명했다. [U20-044] (K 5.5 나가며, PDF p.230)

그래프 컬러링 솔루션으로 웰시-포웰 **알고리즘**을 다뤘다. [U20-045] (K 5.5 나가며, PDF p.230)

21 그래프 알고리즘 I

K PDF 232-269

다루는 내용

그래프에서 적절한 탐색 방법을 선택하여 구현하고, 프림 **알고리즘**으로 **최소 신장 트리**를 구한다. [U21-001] (K 6장 학습 목표, PDF p.233)

프림과 크루스칼 **알고리즘**을 상황에 따라 선택하고, 다익스트라 **알고리즘**으로 두 정점 사이의 최단 경로를 찾는다. [U21-002] (K 6장 학습 목표, PDF p.233)

핵심 개념

6.1 들어가며

그래프는 정점(vertex)의 집합과 정점들을 서로 잇는 에지(edge)의 집합으로 구성되며, G = 〈V, E〉에서 V는 정점 집합, E는 에지 집합이다. [U21-003] (K 6.1, PDF p.233)

에지가 특정 정점에서 다른 정점으로 향하는 방향이 있으면 방향 **그래프**(directed graph), 특정 방향이 없으면 무방향 **그래프**(undirected graph)이다. [U21-004] (K 6.1, PDF p.233)

에지에 가중치가 있으면 가중 **그래프**(weighted graph), 가중치가 없으면 비가중 **그래프**(unweighted graph)이다. [U21-005] (K 6.1, PDF p.233)

그래프에는 두 정점 사이의 여러 에지, 한 에지의 여러 가중치, 자기 자신으로 연결되는 셀프 에지(self edge)가 있을 수 있다. [U21-006] (K 6.1, PDF p.234)

6.2.1~6.2.2 너비 우선 탐색

그래프 순회 문제(graph traversal problem)는 특정 정점에서 시작하여 나머지 모든 정점을 방문하는 문제이며, 특정 정점을 찾는 용도로도 쓰여 **그래프** 탐색 문제라고도 부른다. [U21-007] (K 6.2, PDF p.235)

너비 우선 탐색(**BFS**, Breadth-First Search)은 시작 정점을 경계(frontier)에 추가한 뒤, 현재 경계에 인접한 정점을 반복적으로 탐색한다. [U21-008] (K 6.2.1, PDF p.236)

BFS는 시작 정점에서 멀리 있는 정점보다 가까운 정점을 먼저 방문하며, 같은 거리에 있는 정점들의 방문 순서는 임의로 정할 수 있다. [U21-009] (K 6.2.1, PDF p.236)

BFS는 자식 정점을 손자 정점보다 먼저 방문하며, 보통 경계를 별도 자료 구조로 만들지 않고 정점 ID를 큐에 저장한다. [U21-010] (K 6.2.1, PDF p.237)

6.2.3~6.2.4 깊이 우선 탐색

깊이 우선 탐색(**DFS**, Depth-First Search)은 시작 정점에서 특정 경로를 따라 가능한 멀리 있는 정점을 재귀적으로 먼저 방문한다. [U21-011] (K 6.2.3, PDF p.243)

DFS는 더 방문할 정점이 없어지면 다른 경로를 찾아 다시 멀어지는 방향으로 탐색하며, 이러한 방법을 **백트래킹**(backtracking)이라고 한다. [U21-012] (K 6.2.3, PDF p.243)

BFS는 선입선출(FIFO) 큐를 사용하고, **DFS**는 후입선출(LIFO) **스택**을 사용하여 인접한 정점들을 재귀적으로 이동하며 방문한다. [U21-013] (K 6.2.3, PDF p.245)

BFS는 시작 정점에서 가까운 정점을, **DFS**는 대체로 멀리 있는 정점을 찾을 때 적합하다. [U21-014] (K 6.2.4, PDF p.249)

본문은 **BFS**의 검색 트리를 짧고 넓으며 많은 메모리를 필요로 하는 것으로, **DFS**의 검색 트리를 길고 좁으며 상대적으로 적은 메모리를 필요로 하는 것으로 비교한다. [U21-015] (K 6.2.4, PDF p.249)

6.2.5 이분 그래프 판별하기

이분 그래프(bipartite graph)는 정점을 두 집합으로 나눌 수 있고 모든 에지가 서로 다른 집합의 정점끼리 연결되는 **그래프**이다. [U21-016] (K 6.2.5, PDF p.249)

DFS로 이동하며 두 색을 번갈아 칠할 수 있으면 **이분 그래프**이며, 같은 색을 할당한 인접한 두 정점을 만나면 **이분 그래프**가 아니다. [U21-017] (K 6.2.5, PDF p.252)

6.3 프림의 최소 신장 트리 알고리즘

최소 신장 트리(MST) 문제는 주어진 **그래프**의 모든 정점을 연결하고, 연결된 에지의 가중치 합이 최소인 트리 T를 구하는 문제이다. [U21-018] (K 6.3, PDF p.253)

프림 **알고리즘**(Prim's algorithm)은 시작 정점으로 경계를 구성한 뒤 경계를 관통하는 에지 중 가중치가 가장 작은 에지를 선택하여 연결된 정점을 방문한다. [U21-019] (K 6.3, PDF p.253)

프림 **알고리즘**을 구현하려면 각 정점에 경계로부터의 거리(distance)를 기록해야 한다. [U21-020] (K 6.3, PDF p.253)

프림 **알고리즘**은 인접한 정점의 거리 값이 현재 정점과 연결하는 에지 가중치보다 크면 그 가중치로 바꾸며, 이 과정을 현재 정점에 안착(settle)했다고 한다. [U21-021] (K 6.3, PDF p.254)

6.4 다익스트라 최단 경로 알고리즘

단일 시작(single source) 최단 경로 문제(shortest **path** problem)는 시작 정점에서 목적 정점으로 이어지는 최소 비용 경로를 구하는 문제이다. [U21-022] (K 6.4, PDF p.259)

다익스트라 **알고리즘**(Dijkstra's algorithm)은 음수 가중치가 없는 **그래프**에서 동작하며 프림의 MST **알고리즘**을 약간 변경한 형태이다. [U21-023] (K 6.4, PDF p.260)

프림은 경계로부터의 최소 거리를 사용하지만, 다익스트라는 시작 정점으로부터 각 정점까지의 전체 거리를 사용한다. [U21-024] (K 6.4, PDF p.260)

다익스트라는 목적 정점이 나타나면 종료하지만, 프림은 모든 정점을 방문해야 종료한다. [U21-025] (K 6.4, PDF p.260)

다익스트라는 인접 정점의 거리 값이 현재 정점의 거리 값과 연결 에지 가중치의 합보다 크면 그 합으로 거리 값을 바꾼다. [U21-026] (K 6.4, PDF p.261)

동작 방식·절차

프림 알고리즘의 동작 순서

1. 모든 정점의 거리 값을 무한대로 **초기화**하고 시작 정점의 거리만 0으로 설정한 뒤, 모든 거리 값을 최소 힙 H에 추가한다. [U21-027] (K 6.3, PDF p.253)

2. 최소 힙에서 정점 U를 꺼내 MST에 추가하고, U를 포함하도록 경계를 새로 설정한다. [U21-028] (K 6.3, PDF p.254)

3. U에 인접한 각 정점 V의 거리 값이 (U, V)의 에지 가중치보다 크면 그 가중치로 설정한다. [U21-029] (K 6.3, PDF p.254)

4. 방문하지 않은 정점이 남아 있으면 최소 힙에서 정점을 꺼내는 단계로 돌아간다. [U21-030] (K 6.3, PDF p.254)

다익스트라 알고리즘의 동작 순서

1. 모든 정점의 거리 값을 무한대로 **초기화**하되 시작 정점만 0으로 설정하고, 모든 거리 값을 최소 힙 H에 추가한다. [U21-031] (K 6.4, PDF p.260)

2. 최소 힙에서 정점 U를 꺼내고, U가 목적 정점이면 **알고리즘**을 종료한다. [U21-032] (K 6.4, PDF p.260)

3. 다익스트라는 인접 정점의 거리 값이 현재 정점의 거리 값과 연결 에지 가중치의 합보다 크면 그 합으로 거리 값을 바꾼다. [U21-026] (K 6.4, PDF p.261)

4. 방문하지 않은 정점이 남아 있으면 최소 힙에서 정점을 꺼내는 단계로 돌아가며, 목적 정점을 꺼내면 종료한다. [U21-033] (K 6.4, PDF p.261)

DFS를 이용한 이분 그래프 판별

1. 그래프를 이동하면서 두 색을 번갈아 정점에 칠한다. [U21-034] (K 6.2.5, PDF p.252)

2. 모든 정점에 이 방식으로 색을 칠할 수 있으면 **이분 그래프**로 간주한다. [U21-035] (K 6.2.5, PDF p.252)

3. DFS 과정에서 같은 색의 인접한 두 정점을 만나면 **이분 그래프**가 아닌 것으로 판별한다. [U21-036] (K 6.2.5, PDF p.252)

주요 문법·표·수식

연습 문제 28: Edge 구조체의 원문 발췌 (원문 발췌)

```
template <typename T>
struct Edge
{
    unsigned src;
    unsigned dst;
    T weight;
};
```

[U21-037] (K 6.2.2, PDF p.238)

BFS와 **DFS**의 **시간 복잡도**: O(V + E); V는 정점의 개수, E는 에지의 개수이다. [U21-038] (K 6.2.2·6.2.4, PDF p.243,249)

프림 **알고리즘**의 **시간 복잡도**: O(E log V), O(E + V log V); 앞의 식은 이진 최소 힙과 **인접 리스트**를 사용한 경우, 뒤의 식은 피보나치 최소 힙을 사용한 경우이다. [U21-039] (K 6.3.1, PDF p.259)

다익스트라 **알고리즘**의 **시간 복잡도**: O(E + V log V); 피보나치 최소 힙을 사용한 경우이다. [U21-040] (K 6.4.1, PDF p.266)

크루스칼 알고리즘과 프림 알고리즘 비교

비교 기준	크루스칼 알고리즘	프림 알고리즘
MST 구성 방식 [U21-041] (K 6.3.1, PDF p.259)	최소 가중치 에지를 차례대로 추가한다. [U21-042] (K 6.3.1, PDF p.259)	아무 정점부터 시작한다. [U21-043] (K 6.3.1, PDF p.259)
가장 널리 알려진 **시간 복잡도** [U21-044] (K 6.3.1, PDF p.259)	O(E log V) [U21-045] (K 6.3.1, PDF p.259)	O(E + V log V) — 피보나치 최소 힙 사용 [U21-046] (K 6.3.1, PDF p.259)
주로 사용하는 **그래프** [U21-047] (K 6.3.1, PDF p.259)	적은 수의 에지로 구성된 희소 **그래프**(sparse graph) [U21-048] (K 6.3.1, PDF p.259)	많은 수의 에지로 구성된 밀집 **그래프**(dense graph) [U21-049] (K 6.3.1, PDF p.259)

핵심 용어 표

용어	원문에 나온 정의·설명
그래프	**그래프**는 정점(vertex)의 집합과 정점들을 서로 잇는 에지(edge)의 집합으로 구성되며, G = ⟨V, E⟩에서 V는 정점 집합, E는 에지 집합이다. [U21-003] (K 6.1, PDF p.233)
방향·무방향 그래프	에지가 특정 정점에서 다른 정점으로 향하는 방향이 있으면 방향 **그래프**(directed graph), 특정 방향이 없으면 무방향 **그래프**(undirected graph)이다. [U21-004] (K 6.1, PDF p.233)
가중·비가중 그래프	에지에 가중치가 있으면 가중 **그래프**(weighted graph), 가중치가 없으면 비가중 **그래프**(unweighted graph)이다. [U21-005] (K 6.1, PDF p.233)
그래프 순회 문제	**그래프 순회 문제**(graph traversal problem)는 특정 정점에서 시작하여 나머지 모든 정점을 방문하는 문제이며, 특정 정점을 찾는 용도로도 쓰여 **그래프** 탐색 문제라고도 부른다. [U21-007] (K 6.2, PDF p.235)
BFS	너비 우선 탐색(**BFS**, Breadth-First Search)은 시작 정점을 경계(frontier)에 추가한 뒤, 현재 경계에 인접한 정점을 반복적으로 탐색한다. [U21-008] (K 6.2.1, PDF p.236)
DFS	깊이 우선 탐색(**DFS**, Depth-First Search)은 시작 정점에서 특정 경로를 따라 가능한 멀리 있는 정점을 재귀적으로 먼저 방문한다. [U21-011] (K 6.2.3, PDF p.243)
백트래킹	**DFS**는 더 방문할 정점이 없어지면 다른 경로를 찾아 다시 멀어지는 방향으로 탐색하며, 이러한 방법을 **백트래킹**(backtracking)이라고 한다. [U21-012] (K 6.2.3, PDF p.243)
이분 그래프	**이분 그래프**(bipartite graph)는 정점을 두 집합으로 나눌 수 있고 모든 에지가 서로 다른 집합의 정점끼리 연결되는 **그래프**이다. [U21-016] (K 6.2.5, PDF p.249)
최소 신장 트리(MST)	**최소 신장 트리**(MST) 문제는 주어진 **그래프**의 모든 정점을 연결하고, 연결된 에지의 가중치 합이 최소인 트리 T를 구하는 문제이다. [U21-018] (K 6.3, PDF p.253)

용어	원문에 나온 정의·설명
단일 시작 최단 경로 문제	단일 시작(single source) 최단 경로 문제(shortest **path** problem)는 시작 정점에서 목적 정점으로 이어지는 최소 비용 경로를 구하는 문제이다. [U21-022] (K 6.4, PDF p.259)

단원 요약

요약 기준: 원문 나가며

그래프 순회 문제의 해결 방법으로 너비 우선 탐색(**BFS**)과 깊이 우선 탐색(**DFS**)을 소개했다. [U21-050] (K 6.5, PDF p.268)

MST 문제를 해결하는 프림 **알고리즘**을 설명하고 크루스칼 **알고리즘**과 비교했다. [U21-051] (K 6.5, PDF p.268)

단일 시작 최단 경로 문제와 다익스트라 **알고리즘**을 설명하며, 6.4 본문은 음수 가중치가 없는 **그래프**에서 동작한다고 명시한다. [U21-052] (K 6.5, PDF p.268; K 6.4, PDF p.260)

22 그래프 알고리즘 II

K PDF 270-313

다루는 내용

다익스트라 **알고리즘**의 문제를 이해하고, **벨만-포드 알고리즘**과 **존슨 알고리즘**으로 최단 경로를 찾으며 **음수 가중치 사이클**을 판단한다. [U22-001] (K 7장 학습 목표, PDF p.271)

방향·무방향 그래프의 연결성 차이를 설명하고, **코사라주 알고리즘**으로 **강한 연결 요소**를 찾는다. [U22-002] (K 7장 학습 목표, PDF p.271)

핵심 개념

7.2~7.3 최단 경로와 벨만-포드 알고리즘

다익스트라는 방문하지 않은 정점 중 가장 가까운 정점을 선택한 뒤 다시 검토하지 않으므로, 음수 에지를 고려하기 전에 다른 정점을 방문해 최적 경로를 찾지 못할 수 있다. [U22-003] (K 7.2, PDF p.272)

벨만-포드 알고리즘(Bellman-Ford algorithm)은 음수 가중치가 있는 **그래프**에서 사용할 수 있으며, 모든 에지에 대한 선택을 (V - 1)번 반복해 점진적으로 최단 거리를 찾는다. [U22-004] (K 7.3, PDF p.273)

연습 문제 32의 **UNKNOWN**은 무한대를 표현하는 정수 **상수**로, 에지 가중치의 합보다 충분히 큰 값을 사용하며 코드에서는 INT_MAX로 정의한다. [U22-005] (K 7.3.1, PDF p.274)

BellmanFord()는 에지 리스트 edges, 정점 개수 V, 출발 정점 번호 start를 받고 거리 값 **배열**을 반환한다. [U22-006] (K 7.3.1, PDF p.274)

각 에지의 시작 정점 거리 값이 **UNKNOWN**이 아닌 경우, 시작 정점 거리와 에지 가중치의 합이 목적 정점 거리보다 작으면 그 합으로 목적 정점 거리를 교체한다. [U22-007] (K 7.3.1, PDF p.275)

7.4 벨만-포드 알고리즘과 음수 가중치 사이클

음수 가중치 사이클(negative weight cycle)은 에지 가중치의 합이 음수가 되는 사이클이다. [U22-008] (K 7.4, PDF p.277)

음수 가중치 사이클이 있으면 그 사이클이 계산에 반복 사용되어 최종 결과가 왜곡될 수 있다. [U22-009] (K 7.4, PDF p.277)

벨만-포드의 (V - 1)번 반복 후 전체 에지를 다시 검사하여 더 짧은 경로가 발견되면 **음수 가중치 사이클**이 있다고 판단한다. [U22-010] (K 7.4·7.4.1, PDF p.277-278)

연습 문제 33은 **음수 가중치 사이클**이 발견되면 "**음수 가중치 사이클** 발견!"을 출력하고 빈 벡터를 반환하며 종료한다. [U22-011] (K 7.4.1, PDF p.278)

실습 문제 15 '욕심쟁이 로봇'은 도착 시 남는 에너지를 최대화하는 경로를 선택하고, 충전소를 반복 방문하여 과충전될 가능성을 예측하는 문제이다. [U22-012] (K 7.4.2, PDF p.281)

음수 에지 가중치가 없는 **그래프**에서는 그리디 접근을 사용하는 다익스트라 **알고리즘**이 벨만-포드보다 효율적이다. [U22-013] (K 7.4.2, PDF p.284)

7.5 존슨 알고리즘

존슨 알고리즘(Johnson's algorithm)은 벨만-포드와 다익스트라를 결합하여 모든 정점 사이의 최단 경로를 구하며, 전체 에지 가중치를 음수가 아닌 형태로 변환한다. [U22-014] (K 7.5, PDF p.285)

새 더미(dummy) 정점을 추가하고 모든 정점에 가중치가 0인 에지를 연결한 뒤, 벨만-포드로 더미 정점에서 각 정점까지의 최단 거리를 기록한다. [U22-015] (K 7.5, PDF p.285)

가중치 변환 수식을 연속 적용하면 각 줄의 세 번째 항이 다음 줄의 가운데 항에 의해 상쇄되며, 이를 **망원**(telescoping) 속성이라고 한다. [U22-016] (K 7.5, PDF p.286)

가중치 변환은 원래의 최단 경로 순서를 유지하면서 음이 아닌 가중치의 **그래프**를 만들며, 각 정점에서 다익스트라를 적용해 모든 정점 쌍의 최단 거리를 구한다. [U22-017] (K 7.5, PDF p.287)

연습 문제 34는 변환한 가중치로 구한 거리에 변환 수식을 역으로 적용하여 실제 최단 거리 값을 다시 계산한다. [U22-018] (K 7.5.1, PDF p.292)

실습 문제 16에서 정점의 평균 최단 거리는 그 정점에서 출발하는 최단 경로 거리 합을 도달 가능한 정점 개수로 나눈 값이며, **그래프**의 평균 거리는 모든 정점의 평균 최단 거리의 평균이다. [U22-019] (K 7.5.2, PDF p.295)

7.6 강한 연결 요소

방향 **그래프**를 같은 구역의 정점끼리 서로 이동 가능한 경로를 갖도록 나눌 때, 각 구역이 **강한 연결 요소**를 나타낸다. [U22-020] (K 7.6, PDF p.297)

무방향 **그래프**의 **연결 요소**(connected component)는 모든 정점이 서로 연결된 부분 **그래프** 중 최대 크기의 부분 **그래프** 집합이며, 그 안의 두 정점은 상호 접근이 가능하다. [U22-021] (K 7.6.1, PDF p.297)

무방향 **그래프**의 **연결 요소**는 다른 부분 **그래프**와 완전히 단절되지만, **강한 연결 요소**(strongly connected component)는 다른 부분 **그래프**와 완전히 단절될 필요가 없다. [U22-022] (K 7.6.1, PDF p.298)

7.7 코사라주 알고리즘

코사라주 알고리즘(Kosaraju's algorithm)은 **강한 연결 요소**를 찾기 위해 입력 **그래프**에 **DFS**를 한 번, **전치**한 **그래프**에 **DFS**를 한 번 수행한다. [U22-023] (K 7.7, PDF p.300)

그래프의 **전치**(transpose)는 시작 정점과 목표 정점을 서로 뒤바꿔 각 에지의 방향을 반대로 하는 것이다. [U22-024] (K 7.7, PDF p.300)

첫 번째 **DFS**는 방문을 기록하고 미방문 인접 정점을 재귀적으로 탐색한 뒤, 인접 정점을 모두 탐색한 현재 정점을 **스택**에 추가한다. [U22-025] (K 7.7, PDF p.300)

전치 그래프에서는 **스택**에서 꺼낸 정점을 아직 방문하지 않았다면 **DFS**를 시작하며, 각각의 **DFS**에서 만나는 정점들이 **강한 연결 요소**를 구성한다. [U22-026] (K 7.7, PDF p.300)

본문은 코사라주 구현에서 인접 행렬을 권장하지 않으며, 각 정점의 이웃을 찾는 데 상당한 반복이 필요하기 때문이라고 설명한다. [U22-027] (K 7.7, PDF p.301)

실습 문제 17의 미로에서 점수를 무한히 낮출 경로의 존재 여부는, 미로를 가중 **그래프**로 표현하고 **음수 가중치 사이클**이 있는지 판단하는 것으로 해석한다. [U22-028] (K 7.7.2, PDF p.310)

7.8 적절한 방법 선택하기

본문은 모든 정점 사이의 최단 거리에는 **존슨 알고리즘**을, 하나의 시작 정점에서 나머지 정점까지의 최단 거리에는 존슨보다 벨만-포드를 사용하는 것이 효율적이라고 비교한다. [U22-029] (K 7.8, PDF p.311)

그래프 구현 방식은 데이터 특성과 문제 세부 사항에 따라 선택하며, 성능 외에 코드의 가독성과 간결성을 고려할 수도 있다. [U22-030] (K 7.8, PDF p.311)

구현에 앞서 기본 목적과 필요한 기능, 문제 정보를 나타내는 가장 기본적인 구성 요소를 숙지해야 한다. [U22-031] (K 7.8, PDF p.312)

동작 방식·절차

벨만-포드와 음수 가중치 사이클 검사

1. distance **배열**을 **UNKNOWN**으로 **초기화**하고 출발 정점의 값만 0으로 설정한다. [U22-032] (K 7.3.1, PDF p.274)

2. 전체 에지를 검토하며 거리 값을 갱신하는 반복을 (V - 1)번 수행한다. [U22-033] (K 7.3.1, PDF p.275)

3. 각 에지의 시작 정점 거리 값이 **UNKNOWN**이 아닌 경우, 시작 정점 거리와 에지 가중치의 합이 목적 정점 거리보다 작으면 그 합으로 목적 정점 거리를 교체한다. [U22-007] (K 7.3.1, PDF p.275)

4. 벨만-포드의 (V - 1)번 반복 후 전체 에지를 다시 검사하여 더 짧은 경로가 발견되면 **음수 가중치 사이클**이 있다고 판단한다. [U22-010] (K 7.4·7.4.1, PDF p.277-278)

존슨 알고리즘의 처리 순서

1. 새 더미(dummy) 정점을 추가하고 모든 정점에 가중치가 0인 에지를 연결한 뒤, 벨만-포드로 더미 정점에서 각 정점까지의 최단 거리를 기록한다. [U22-015] (K 7.5, PDF p.285)

2. BellmanFord()의 결과 h가 비어 있으면 종료하고, 그렇지 않으면 모든 에지에 가중치 변환 수식을 적용한다. [U22-034] (K 7.5.1, PDF p.291)

3. 각 정점에서 다익스트라를 호출하여 변환된 **그래프**의 모든 정점 간 최단 거리를 shortest에 저장한다. [U22-035] (K 7.5.1, PDF p.291-292)

4. 연습 문제 34는 변환한 가중치로 구한 거리에 변환 수식을 역으로 적용하여 실제 최단 거리 값을 다시 계산한다. [U22-018] (K 7.5.1, PDF p.292)

코사라주 알고리즘의 처리 순서

1. 첫 번째 **DFS**는 방문을 기록하고 미방문 인접 정점을 재귀적으로 탐색한 뒤, 인접 정점을 모두 탐색한 현재 정점을 **스택**에 추가한다. [U22-025] (K 7.7, PDF p.300)

2. 원본 **그래프**의 **DFS**가 끝나면 **그래프**를 **전치**한다. [U22-036] (K 7.7, PDF p.300)

3. 전치 그래프에서는 **스택**에서 꺼낸 정점을 아직 방문하지 않았다면 **DFS**를 시작하며, 각각의 **DFS**에서 만나는 정점들이 **강한 연결 요소**를 구성한다. [U22-026] (K 7.7, PDF p.300)

주요 문법·표·수식

연습 문제 33: 음수 가중치 사이클 검사 원문 발췌 (원문 발췌)

```
for (auto& e : edges)
{
    if (distance[e.src] == UNKNOWN)
        continue;

    if (distance[e.dst] > distance[e.src] + e.weight)
    {
        cout << "음수 가중치 사이클 발견!" << endl;
        return {};
    }
}
```

[U22-037] (K 7.4.1, PDF p.278)

연습 문제 34: 에지 가중치 재설정 원문 발췌 (원문 발췌)

```
// 에지 가중치 재설정
for (auto& e : edges)
{
    e.weight += (h[e.src] - h[e.dst]);
}
```

[U22-038] (K 7.5.1, PDF p.291)

존슨 알고리즘의 에지 가중치 변환: W(uv) = w(uv) + d[s, u] - d[s, v]; w(uv)는 초기 에지 가중치, d[s, u]와 d[s, v]는 더미 정점 s로부터의 최단 거리, W(uv)는 변환된 가중치이다. [U22-039] (K 7.5, PDF p.286)

코사라주 알고리즘의 **시간 복잡도**: O(V + E); 입력 **그래프**를 **인접 리스트**로 표현한 경우이다. [U22-040] (K 7.7, PDF p.301)

7.8에서 제시한 최단 거리 알고리즘 선택

구하려는 값	본문의 선택
모든 정점 사이의 최단 거리 [U22-041] (K 7.8, PDF p.311)	**존슨 알고리즘** [U22-042] (K 7.8, PDF p.311)
하나의 시작 정점에서 나머지 정점까지의 최단 거리 [U22-043] (K 7.8, PDF p.311)	존슨보다 벨만-포드가 효율적 [U22-044] (K 7.8, PDF p.311)

핵심 용어 표

용어	원문에 나온 정의·설명
벨만-포드 알고리즘	**벨만-포드 알고리즘**(Bellman-Ford algorithm)은 음수 가중치가 있는 **그래프**에서 사용할 수 있으며, 모든 에지에 대한 선택을 (V - 1)번 반복해 점진적으로 최단 거리를 찾는다. [U22-004] (K 7.3, PDF p.273)
UNKNOWN	연습 문제 32의 **UNKNOWN**은 무한대를 표현하는 정수 **상수**로, 에지 가중치의 합보다 충분히 큰 값을 사용하며 코드에서는 INT_MAX로 정의한다. [U22-005] (K 7.3.1, PDF p.274)
음수 가중치 사이클	**음수 가중치 사이클**(negative weight cycle)은 에지 가중치의 합이 음수가 되는 사이클이다. [U22-008] (K 7.4, PDF p.277)
존슨 알고리즘	**존슨 알고리즘**(Johnson's algorithm)은 벨만-포드와 다익스트라를 결합하여 모든 정점 사이의 최단 경로를 구하며, 전체 에지 가중치를 음수가 아닌 형태로 변환한다. [U22-014] (K 7.5, PDF p.285)
망원(telescoping) 속성	가중치 변환 수식을 연속 적용하면 각 줄의 세 번째 항이 다음 줄의 가운데 항에 의해 상쇄되며, 이를 **망원**(telescoping) 속성이라고 한다. [U22-016] (K 7.5, PDF p.286)
강한 연결 요소	방향 **그래프**를 같은 구역의 정점끼리 서로 이동 가능한 경로를 갖도록 나눌 때, 각 구역이 **강한 연결 요소**를 나타낸다. [U22-020] (K 7.6, PDF p.297)
연결 요소	무방향 **그래프**의 **연결 요소**(connected component)는 모든 정점이 서로 연결된 부분 **그래프** 중 최대 크기의 부분 **그래프** 집합이며, 그 안의 두 정점은 상호 접근이 가능하다. [U22-021] (K 7.6.1, PDF p.297)
전치(transpose)	**그래프**의 **전치**(transpose)는 시작 정점과 목표 정점을 서로 뒤바꿔 각 에지의 방향을 반대로 하는 것이다. [U22-024] (K 7.7, PDF p.300)
코사라주 알고리즘	**코사라주 알고리즘**(Kosaraju's algorithm)은 **강한 연결 요소**를 찾기 위해 입력 **그래프**에 **DFS**를 한 번, **전치**한 **그래프**에 **DFS**를 한 번 수행한다. [U22-023] (K 7.7, PDF p.300)

단원 요약

요약 기준: 원문 7.9 나가며

그래프 기반 솔루션으로 복잡한 데이터를 비교적 쉽게 조작하고 다루는 방법을 살펴봤다. [U22-045] (K 7.9, PDF p.312)

6장의 **그래프** 구조와 순회 기본 원리를 확장하여 고급 문제를 해결했다. [U22-046] (K 7.9, PDF p.312)

기본 개념을 제대로 이해하는 것은 더 심도 있는 **그래프** 구현을 탐구하기 위해 중요하다. [U22-047] (K 7.9, PDF p.312)

23 동적 계획법 I

K PDF 314-373

다루는 내용

주어진 문제에 동적 계획법을 적용할 수 있는지 분석하고 **메모이제이션**과 **타뷸레이션**을 비교한다. [U23-001] (K 8장 학습 목표, PDF p.315)

직관적인 전수 조사부터 점진적으로 최적화 **알고리즘**을 구현하며 동적 계획법 해법을 개발한다. [U23-002] (K 8장 학습 목표, PDF p.315)

핵심 개념

동적 계획법의 두 특성·기저 조건·상태

중복되는 부분 문제는 같은 부분 문제가 반복적으로 나타나는 특성이다. [U23-003] (K 8.2 동적 계획법이란?, PDF p.318)

최적 부분 구조는 전체 문제의 최적해를 부분 문제의 최적해 조합으로 표현할 수 있는 특성이다. [U23-004] (K 8.2 동적 계획법이란?, PDF p.318)

책은 동적 계획법을 적용할 문제에 **중복되는 부분 문제**와 **최적 부분 구조**의 두 속성이 필요하다고 설명한다. [U23-005] (K 8.2 동적 계획법이란?, PDF p.318)

기저 조건(base case)은 더 이상의 재귀 없이 해를 구할 수 있는 지점이다. [U23-006] (K 8.2 동적 계획법이란?, PDF p.316)

상태(state)를 정할 때는 각 부분 문제를 다른 부분 문제와 다르다고 판단할 수 있는 기준을 찾아야 한다. [U23-007] (K 8.5.2 기저 조건과 상태 정의하기, PDF p.325)

메모이제이션과 캐시

메모이제이션(memoization)은 하향식 방식에서 부분 문제의 해를 캐시에 넣어 사용하는 기술이다. [U23-008] (K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)

함수가 호출되면 먼저 현재 **상태**가 캐시에 있는지 확인하고 이미 저장되어 있으면 그 값을 반환한다. [U23-009] (K 8.3 메모이제이션: 하향식 접근 방법, PDF p.318)

캐시 키는 서로 다른 **상태**를 **충돌** 없이 표현해야 하고, 저장값을 다시 참조할 수 있어야 한다. [U23-010] (K 8.5.7 메모이제이션·캐시 사용하기, PDF p.338)

하향식 접근은 재귀 호출이 너무 많으면 호출 **스택** 메모리를 초과하는 **스택** 오버플로가 발생할 수 있다. [U23-011] (K 8.3 메모이제이션: 하향식 접근 방법, PDF p.320)

타뷸레이션과 상향식 구성

타뷸레이션(tabulation)은 **기저 조건** 해답부터 시작하여 부분 문제의 해답을 표에 저장하고 재사용하는 방식이다. [U23-012] (K 8.4 타뷸레이션: 상향식 접근 방법, PDF p.320)

타뷸레이션은 앞부분 **상태**부터 채운 값을 나중 **상태**에서 재사용한다. [U23-013] (K 8.6.5 하향식에서 상향식으로 바꾸기, PDF p.367)

부분집합의 합 **상태**에는 현재까지의 합과 현재 처리하는 집합 인덱스가 필요하다. [U23-014] (K 8.5.9 타뷸레이션, PDF p.343)

부분집합의 합: 전수 조사에서 최적화로

부분집합의 합 문제는 음수가 아닌 정수 집합 S와 정수 x에 대해 원소 합이 x인 S의 부분집합이 존재하는지를 묻는다. [U23-015] (K 8.5 부분집합의 합 문제, PDF p.322)

책은 전수 조사, **백트래킹**, **메모이제이션**, **타뷸레이션**의 순서로 접근 방법을 발전시킨다. [U23-016] (K 8.5.2 기저 조건과 상태 정의하기, PDF p.325)

전수 조사는 단순한 문제 이해와 동적 계획법의 결과를 비교할 정답을 마련하는 데 도움이 된다. [U23-017] (K 8.5.3 전수 조사, PDF p.325)

백트래킹은 유효하지 않은 부분 문제의 경우를 제거하여 **알고리즘**을 개선한다. [U23-018] (K 8.5.5 최적화 적용하기 - 백트래킹, PDF p.330)

부분집합 합은 실제 구성 원소를 모두 보관하지 않고 sum과 i 두 정수로 중간 **상태**를 표현할 수 있다. [U23-019] (K 8.5.6 백트래킹을 사용하여 부분집합의 합 문제 풀기, PDF p.333)

부분집합 합만 캐시 키로 쓰면 같은 합에 도달한 서로 다른 경우를 구분하지 못하여 잘못된 결과가 생길 수 있다. [U23-020] (K 8.5.7 메모이제이션·캐시 사용하기, PDF p.338)

최장 공통 부분 시퀀스

최장 공통 부분 시퀀스(LCS, Longest Common Subsequence)는 두 데이터 시퀀스에 공통으로 나타나는 가장 긴 부분 시퀀스를 찾는 문제이다. [U23-021] (K 8.6.1 최장 공통 부분 시퀀스 문제, PDF p.355)

두 문자열 중 하나라도 빈 문자열이면 LCS 값은 0이다. [U23-022] (K 8.6.3 최적 부분 구조 찾기, PDF p.364)

두 부분 문자열의 마지막 문자가 같으면 각각 그 문자를 뺀 부분 문자열의 LCS 길이에 1을 더한다. [U23-023] (K 8.6.3 최적 부분 구조 찾기, PDF p.364)

마지막 문자가 다르면 A에서 한 문자를 뺀 경우와 B에서 한 문자를 뺀 경우의 LCS 길이 중 최댓값을 택한다. [U23-024] (K 8.6.3 최적 부분 구조 찾기, PDF p.364)

LCS **메모이제이션**은 두 문자열 길이를 각 차원 크기로 하는 2차원 벡터에 계산 결과를 저장한다. [U23-025] (K 8.6.3 최적 부분 구조 찾기, PDF p.364)

실습 문제: 여행 경로·멜로디 순열

여행 경로 실습은 각 도시에서 이동 가능한 최대 도시 수가 주어졌을 때 목적지까지 도달하는 방법의 가짓수를 구한다. [U23-026] (K 8.5.11 실습 문제 18: 여행 경로, PDF p.349)

멜로디 순열 실습은 주어진 멜로디에 음이름 집합의 순열이 나타나는 횟수를 센다. [U23-027] (K 8.7 실습 문제 21: 멜로디 순열, PDF p.370)

멜로디 실습의 유효 순열에서 음이름은 순서와 상관없이 여러 번 반복되어 사용될 수 있다. [U23-028] (K 8.7 실습 문제 21: 멜로디 순열, PDF p.370)

동작 방식·절차

부분집합의 합 문제에서 점진적으로 접근하는 순서

1. 중복되는 부분 문제와 **최적 부분 구조**가 있는지 확인한다. [U23-029] (K 8.2 동적 계획법이란?, PDF p.318)

2. 기저 조건과 **상태**를 정하고 전수 조사·**백트래킹**으로 문제 구조를 분석한다. [U23-030] (K 8.5.2 기저 조건과 상태 정의하기, PDF p.325)

3. 하향식 해법에서 부분 문제의 해답을 캐시에 저장하는 **메모이제이션**을 적용한다. [U23-031] (K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)

4. 기저 조건에서 시작해 표를 채우는 **타뷸레이션**으로 구성한다. [U23-032] (K 8.4 타뷸레이션: 상향식 접근 방법, PDF p.320)

부분집합 합의 원문 상향식 로직

1. 부분집합의 합과 인덱스를 0으로 설정하여 시작한다. [U23-033] (K 8.5.9 타뷸레이션, PDF p.344)

2. 인덱스 i까지 합 x를 만들 수 있으면 i+1까지도 합 x를 만들 수 있다. [U23-034] (K 8.5.9 타뷸레이션, PDF p.344)

3. 같은 **상태**에서 현재 원소를 추가하면 i+1까지 합 x + set[i]도 만들 수 있다. [U23-035] (K 8.5.9 타뷸레이션, PDF p.344)

주요 문법·표·수식

메모이제이션 피보나치 함수 - 원문 발췌 (원문 발췌)

```
int Fibonacci(int n)
{
    if (n < 2)
        return n;
    if (memo[n] != UNKNOWN)
        return memo[n];
    int result = Fibonacci(n - 1) + Fibonacci(n - 2);
    memo[n] = result;
    return result;
}
```

[U23-036] (K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)

원문은 **UNKNOWN** = -1, MAX_SIZE = 100인 **상수**와 memo(MAX_SIZE, **UNKNOWN**) 벡터를 함수 앞에 정의한다. [U23-037] (K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)

피보나치 수열의 **기저 조건**과 재귀 관계: F(0) = 0
F(1) = 1
F(n) = F(n-1) + F(n-2); 세 번째 이후 원소는 바로 앞 두 원소의 합이다. [U23-038] (K 8.2 동적 계획법이란?, PDF p.316)

LCS 테이블을 채우는 원문 로직: i = 0 또는 j = 0: LCS(i, j) = 0
마지막 문자가 같음: LCS(i, j) = LCS(i - 1, j - 1) + 1
그렇지 않음: LCS(i - 1, j)와 LCS(i, j - 1) 중 최댓값; i와 j는 두 부분 문자열의 길이에 해당하는 **상태**이다. [U23-039] (K 8.6.5 하향식에서 상향식으로 바꾸기, PDF p.367)

계산 결과를 재사용하는 두 방법

방법	원문 설명
메모이제이션 [U23-040] (K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)	**메모이제이션**(memoization)은 하향식 방식에서 부분 문제의 해를 캐시에 넣어 사용하는 기술이다. [U23-008] (K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)
타뷸레이션 [U23-041] (K 8.4 타뷸레이션: 상향식 접근 방법, PDF p.320)	**타뷸레이션**(tabulation)은 **기저 조건** 해답부터 시작하여 부분 문제의 해답을 표에 저장하고 재사용하는 방식이다. [U23-012] (K 8.4 타뷸레이션: 상향식 접근 방법, PDF p.320)

핵심 용어 표

용어	원문에 나온 정의·설명
중복되는 부분 문제	**중복되는 부분 문제**는 같은 부분 문제가 반복적으로 나타나는 특성이다. [U23-003] (K 8.2 동적 계획법이란?, PDF p.318)
최적 부분 구조	**최적 부분 구조**는 전체 문제의 최적해를 부분 문제의 최적해 조합으로 표현할 수 있는 특성이다. [U23-004] (K 8.2 동적 계획법이란?, PDF p.318)
기저 조건	**기저 조건**(base case)은 더 이상의 재귀 없이 해를 구할 수 있는 지점이다. [U23-006] (K 8.2 동적 계획법이란?, PDF p.316)
상태	**상태**(state)를 정할 때는 각 부분 문제를 다른 부분 문제와 다르다고 판단할 수 있는 기준을 찾아야 한다. [U23-007] (K 8.5.2 기저 조건과 상태 정의하기, PDF p.325)
메모이제이션	**메모이제이션**(memoization)은 하향식 방식에서 부분 문제의 해를 캐시에 넣어 사용하는 기술이다. [U23-008] (K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)
타뷸레이션	**타뷸레이션**(tabulation)은 **기저 조건** 해답부터 시작하여 부분 문제의 해답을 표에 저장하고 재사용하는 방식이다. [U23-012] (K 8.4 타뷸레이션: 상향식 접근 방법, PDF p.320)
부분집합의 합 문제	**부분집합의 합 문제**는 음수가 아닌 정수 집합 S와 정수 x에 대해 원소 합이 x인 S의 부분집합이 존재하는지를 묻는다. [U23-015] (K 8.5 부분집합의 합 문제, PDF p.322)
최장 공통 부분 시퀀스	**최장 공통 부분 시퀀스**(LCS, Longest Common Subsequence)는 두 데이터 시퀀스에 공통으로 나타나는 가장 긴 부분 시퀀스를 찾는 문제이다. [U23-021] (K 8.6.1 최장 공통 부분 시퀀스 문제, PDF p.355)

단원 요약

요약 기준: 원문 8.8 나가며

이 장은 두 가지 전형적인 동적 계획법 문제를 분석하고 해결 방법을 구현했다. [U23-042] (K 8.8 나가며, PDF p.373)

상태·기저 조건·재귀 관계로 복잡한 문제를 더 단순한 구성 요소로 분해했다. [U23-043] (K 8.8 나가며, PDF p.373)

책은 점진적 접근이 유일한 해법은 아니며 다양한 문제를 지속해서 연습해야 한다고 정리한다. [U23-044] (K 8.8 나가며, PDF p.373)

24 동적 계획법 II

K PDF 374-425

다루는 내용

0-1 **배낭 문제**와 **무한 개수 배낭 문제**를 구현하고 **상태 공간 축소**를 적용한다. [U24-001] (K 9장 학습 목표, PDF p.375)

가중 **그래프**의 **모든 쌍 최단 경로** 문제를 동적 계획법의 최적화 접근으로 해결한다. [U24-002] (K 9장 학습 목표, PDF p.375)

핵심 개념

P·NP와 해답 검증·생성의 구분

P 문제는 다항 시간(polynomial time) 내에 해답을 구할 수 있는 문제이다. [U24-003] (K 9.2 P와 NP, PDF p.377)

9.2절은 정점에 이미 할당된 색상을 검증하는 문제와 조건에 맞는 색상을 배정할 수 있는지 찾는 문제를 비교한다. [U24-004] (K 9.2 P와 NP, PDF p.377)

원문은 주어진 솔루션의 검증이 쉽다는 점과 실제 솔루션을 만들어내는 어려움을 구분하며 P = NP인가를 미해결 문제로 소개한다. [U24-005] (K 9.2 P와 NP, PDF p.378)

부분집합 합의 유효성 검증은 원소를 모두 더하여 목표치와 같은지 확인하므로 부분집합 크기에 대해 선형이다. [U24-006] (K 9.3 부분집합의 합 문제 다시 보기, PDF p.380)

원문은 부분집합 합의 동적 계획법을 **의사 다항 시간**(pseudo-polynomial time)으로 분류한다. [U24-007] (K 9.3 부분집합의 합 문제 다시 보기, PDF p.381)

부분집합 합에서 배낭 문제로

배낭 문제는 제한 용량을 넘지 않으면서 넣은 물건들의 가격 합을 최대화하는 조합을 찾는 문제이다. [U24-008] (K 9.4 배낭 문제, PDF p.381)

0-1 배낭에서는 물건을 추가해도 용량을 넘지 않으면 가격을 더하는 선택과, 기존 가격 합을 유지하는 선택을 비교한다. [U24-009] (K 9.4.1 0-1 배낭 문제, PDF p.383)

원문의 0-1 배낭 DP는 이전 행의 같은 무게 값과 현재 물건을 넣는 경우의 가격 합 중 최댓값을 저장한다. [U24-010] (K 9.4.2 연습 문제 41: 0-1 배낭 문제, PDF p.386)

무한 개수 배낭 문제(unbounded knapsack problem)는 각 물건을 개수 제한 없이 배낭에 담을 수 있는 문제이다. [U24-011] (K 9.4.3 무한 개수 배낭 문제, PDF p.386)

무한 개수 배낭은 **상태** 전환 조건이 무게에만 의존하여 1차원 DP 테이블로 표현할 수 있다. [U24-012] (K 9.4.4 상태 공간 축소, PDF p.394)

상태 공간 축소·대칭·필요한 행

상태 공간 축소(state space reduction)는 **상태**를 표현하는 공간의 크기를 최소화하도록 동적 계획법 **알고리즘**을 재구성하는 작업이다. [U24-013] (K 9.4.4 상태 공간 축소, PDF p.389)

파스칼의 삼각형은 각 행에 필요한 만큼만 공간을 만들고 행의 좌우 대칭을 활용하여 중복 계산을 줄일 수 있다. [U24-014] (K 9.4.4 상태 공간 축소, PDF p.391)

파스칼의 삼각형에서 특정 원소만 필요하면 현재 행 계산에 이전 행만 필요하므로 전체 DP 테이블을 저장할 필요가 없다. [U24-015] (K 9.4.4 상태 공간 축소, PDF p.392)

최단 경로의 동적 계획법 해석

단일 시작 최단 경로 문제(single-source shortest **path** problem)는 한 정점에서 나머지 모든 정점까지의 최단 경로를 찾는다. [U24-016] (K 9.5.2 동적 계획법으로 최단 경로 문제 다루기, PDF p.405)

모든 쌍 최단 경로 문제(all-pairs shortest **path** problem)는 모든 정점 사이의 최단 경로를 계산한다. [U24-017] (K 9.5.2 동적 계획법으로 최단 경로 문제 다루기, PDF p.405)

존슨 알고리즘은 벨만-포드의 동적 계획법 접근과 다익스트라의 그리디 접근을 결합한다. [U24-018] (K 9.5.2 동적 계획법으로 최단 경로 문제 다루기, PDF p.405)

최단 경로 **메모이제이션**은 현재 정점 node와 반복수 depth의 쌍으로 **상태**를 표현한다. [U24-019] (K 9.5.2 동적 계획법으로 최단 경로 문제 다루기, PDF p.405-406)

플로이드-워셜 알고리즘

플로이드-워셜은 모든 정점을 중간 정점으로 사용하면서 모든 정점 쌍 사이의 거리를 테이블에 저장한다. [U24-020] (K 9.5.5 플로이드-워셜 알고리즘, PDF p.413)

현재 start-end 거리보다 start-mid와 mid-end 거리의 합이 짧으면 그 합으로 갱신한다. [U24-021] (K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)

중첩 반복문의 순서는 가장 바깥쪽 mid, 그다음 start, 가장 안쪽 end이다. [U24-022] (K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)

테이블의 distance[i][i]가 0보다 작으면 **음수 가중치 사이클**이 있는 것으로 판단한다. [U24-023] (K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)

원문은 밀집 **그래프**에서 플로이드-워셜을 고려하며, 음수 에지 가중치는 허용하되 음수 사이클에 별도 처리가 필요하다고 설명한다. [U24-024] (K 9.5.5 플로이드-워셜 알고리즘, PDF p.414)

실습 문제: 최대 이익·도로 건설

최대 이익 실습은 capacity와 budget의 두 제약을 고려하기 위해 3차원 DP 테이블을 사용하도록 안내한다. [U24-025] (K 9.4.6 실습 문제 22: 최대 이익, PDF p.401)

도로 건설 실습은 지반 강도에 따라 비용이 달라지는 지역에서 저렴한 도로 시스템을 설계하는 문제이다. [U24-026] (K 9.5.7 실습 문제 23: 도로 건설, PDF p.419)

동작 방식·절차

벨만-포드 알고리즘을 다시 보는 원문 순서

1. 각 정점 거리를 **UNKNOWN**으로 **초기화**하고 시작 정점은 0으로 둔다. [U24-027] (K 9.5.1 벨만-포드 알고리즘 다시 보기, PDF p.402)

2. V-1번 반복하며 모든 에지를 검사하고 시작 정점의 거리가 알려진 경우 인접 정점까지의 후보 거리를 비교한다. [U24-028] (K 9.5.1 벨만-포드 알고리즘 다시 보기, PDF p.402)

3. 더 짧은 후보 거리로 갱신하며, 마지막에 한 번 더 검사하여 음수 사이클 존재 여부를 확인한다. [U24-029] (K 9.5.1 벨만-포드 알고리즘 다시 보기, PDF p.402)

플로이드-워셜 구현 순서

1. distance 테이블을 가중치 행렬로 **초기화**하고 자기 자신까지의 거리는 0으로 설정한다. [U24-030] (K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.415)

2. mid, start, end의 중첩 반복문에서 더 짧은 거리를 발견할 때 테이블을 갱신한다. [U24-031] (K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)

3. 대각 원소가 음수인지 확인하여 **음수 가중치 사이클**을 검사하고 distance를 반환한다. [U24-032] (K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)

주요 문법·표·수식

플로이드-워셜 거리 갱신 - 원문 발췌 (원문 발췌)

```
for (int mid = 0; mid < V; mid++) {
    for (int start = 0; start < V; start++) {
        for (int end = 0; end < V; end++) {
            if (distance[start][mid] + distance[mid][end] < distance[start][end]) {
                distance[start][end] = distance[start][mid] + distance[mid][end];
            }
        }
    }
}
```

[U24-033] (K 9.5.6 연습 문제 44: 플로이드-워셜 알고리즘 구현하기, PDF p.416)

파스칼의 삼각형: 원문 **기저 조건**·순환식: m = 1, m = n → triangle(n, m) = 1
triangle(n, m) = triangle(n-1, m-1) + triangle(n-1, m); m ≤ n인 두 정수 m, n에 대한 원문 표기. [U24-034] (K 9.4.4 상태 공간 축소, PDF p.389)

플로이드-워셜 **알고리즘** 복잡도: 시간 O(V³) · 메모리 O(V²); V는 **그래프** 정점 수이다. [U24-035] (K 9.5.4 모든 쌍 최단 경로, PDF p.413)

배낭 문제의 선택과 상태

문제	상태 전환의 차이
0-1 배낭 [U24-036] (K 9.4.4 상태 공간 축소, PDF p.394)	i번째 물건 포함 여부를 결정하기 위해 (i - 1)개 물건의 **상태**를 고려한다. [U24-037] (K 9.4.4 상태 공간 축소, PDF p.394)
무한 개수 배낭 [U24-038] (K 9.4.4 상태 공간 축소, PDF p.394)	같은 물건을 중복 포함할 수 있고, **상태** 전환은 무게에만 의존하므로 1차원 DP로 표현할 수 있다. [U24-039] (K 9.4.4 상태 공간 축소, PDF p.394)

핵심 용어 표

용어	원문에 나온 정의·설명
P 문제	**P 문제**는 다항 시간(polynomial time) 내에 해답을 구할 수 있는 문제이다. [U24-003] (K 9.2 P와 NP, PDF p.377)
의사 다항 시간	원문은 부분집합 합의 동적 계획법을 **의사 다항 시간**(pseudo-polynomial time)으로 분류한다. [U24-007] (K 9.3 부분집합의 합 문제 다시 보기, PDF p.381)
배낭 문제	**배낭 문제**는 제한 용량을 넘지 않으면서 넣은 물건들의 가격 합을 최대화하는 조합을 찾는 문제이다. [U24-008] (K 9.4 배낭 문제, PDF p.381)
무한 개수 배낭 문제	**무한 개수 배낭 문제**(unbounded knapsack problem)는 각 물건을 개수 제한 없이 배낭에 담을 수 있는 문제이다. [U24-011] (K 9.4.3 무한 개수 배낭 문제, PDF p.386)
상태 공간 축소	**상태 공간 축소**(state space reduction)는 **상태**를 표현하는 공간의 크기를 최소화하도록 동적 계획법 **알고리즘**을 재구성하는 작업이다. [U24-013] (K 9.4.4 상태 공간 축소, PDF p.389)
단일 시작 최단 경로	**단일 시작 최단 경로 문제**(single-source shortest **path** problem)는 한 정점에서 나머지 모든 정점까지의 최단 경로를 찾는다. [U24-016] (K 9.5.2 동적 계획법으로 최단 경로 문제 다루기, PDF p.405)
모든 쌍 최단 경로	**모든 쌍 최단 경로** 문제(all-pairs shortest **path** problem)는 모든 정점 사이의 최단 경로를 계산한다. [U24-017] (K 9.5.2 동적 계획법으로 최단 경로 문제 다루기, PDF p.405)

단원 요약

요약 기준: 원문 9.6 나가며

익숙한 문제를 동적 계획법 관점에서 바라보는 것은 적절한 해법을 작성하는 핵심 아이디어를 제공한다. [U24-040] (K 9.6 나가며, PDF p.424)

배운 기법을 언제 사용할지는 하나로 정해져 있지 않으므로 여러 맥락에 적용해보는 것이 좋다. [U24-041] (K 9.6 나가며, PDF p.424)

책은 자료 구조와 여러 기법을 적절하게 사용하는 감각을 얻고 더 깊이 탐구하기를 권한다. [U24-042] (K 9.6 나가며, PDF p.424)

25 실습 문제 풀이

K PDF 426-529

첨부 수록 범위 확인

K PDF444-446에서 인쇄 쪽수는 455쪽에서 459쪽으로 이어집니다. PDF445 하단 쪽수는 왼쪽이 잘려 전체 숫자를 확인할 수 없습니다. A.2.2 제목·도입 일부는 원문에서 확인 불가이며, 수록된 최소 힙 병합 절차만 정리했습니다. (K 부록 A.2.2, PDF p.444-446)

다루는 내용

각 장의 실습 문제를 풀기 위해 수행해야 할 세부 단계를 코드와 함께 확인한다. [U25-001] (K 부록 실습 문제 풀이, PDF p.426)

리스트·트리·해시·**분할 정복**·그리디·**그래프**·동적 계획법의 실습 풀이를 다룬다. [U25-002] (K 부록 A.1~A.9, PDF p.427,441,447,454,463,469,476,494,517)

핵심 개념

A.1 리스트·스택·큐 실습

음악 재생 목록을 저장하고 필요한 기능을 제공하기 위해 수정된 형태의 이중 **연결 리스트**를 구현한다. [U25-003] (K A.1.1 음악 재생 목록 구현하기, PDF p.427)

한 벌의 카드를 만들고 잘 섞은 후 네 명의 플레이어에게 나눠주며, game 구조체에 이 기능을 구현한다. [U25-004] (K A.1.2 카드 게임 시뮬레이션, PDF p.435)

한 라운드 경기에서는 두 플레이어의 카드를 비교하여 서로 같으면 제거하는 함수를 따로 만든다. [U25-005] (K A.1.2 카드 게임 시뮬레이션, PDF p.435)

Printer **클래스**는 요청된 순서대로 프린트하기 위해 std::queue를 사용하고, 최대 인쇄 요청 개수를 템플릿 인자로 받는다. [U25-006] (K A.1.3 사무실 공유 프린터의 인쇄 대기 목록 시뮬레이션, PDF p.439)

startPrinting()은 대기열이 빌 때까지 jobs.front()를 출력하고 jobs.pop()으로 제거한다. [U25-007] (K A.1.3 사무실 공유 프린터의 인쇄 대기 목록 시뮬레이션, PDF p.439)

A.2 트리·힙·그래프 실습

파일 시스템 표현에는 **N-항 트리**를 사용하고, **생성자**에서 루트 디렉터리를 만든 뒤 현재 디렉터리로 설정한다. [U25-008] (K A.2.1 파일 시스템 자료 구조 만들기, PDF p.441)

n_ary_node는 디렉터리/파일 정보를 저장하며 name, is_dir, children을 가진다. [U25-009] (K A.2.1 파일 시스템 자료 구조 만들기, PDF p.441)

show_path()는 파일이 인자로 전달되면 파일 이름을, 디렉터리가 전달되면 하위 디렉터리와 파일을 모두 출력한다. [U25-010] (K A.2.1 파일 시스템 자료 구조 만들기, PDF p.444)

병합 풀이의 힙 노드는 데이터, 입력 리스트 인덱스, 해당 리스트에서의 데이터 인덱스를 저장한다. [U25-011] (K A.2 트리, 힙, 그래프: 병합 풀이 수록 부분, PDF p.445)

A.3 해시 테이블과 블룸 필터 실습

URLService는 짧은 URL을 키로, 원본 URL을 데이터 값으로 사용하여 맵에 함께 저장한다. [U25-012] (K A.3.1 긴 URL을 짧은 URL로 매핑하기, PDF p.447)

lookup()은 짧은 URL을 이용하여 긴 원본 URL을 찾는 **룩업** 함수이다. [U25-013] (K A.3.1 긴 URL을 짧은 URL로 매핑하기, PDF p.447)

이메일 주소 중복 검사 풀이에서는 **BloomFilter 클래스**를 추가하고 MD5() 함수로 해시 값을 계산한다. [U25-014] (K A.3.2 이메일 주소 중복 검사, PDF p.451-452)

이메일 주소를 추가할 때 hash(key)를 호출하고, 해시 값 버퍼를 반복하면서 해당 비트를 true로 설정한다. [U25-015] (K A.3.2 이메일 주소 중복 검사, PDF p.452)

A.4 분할 정복 실습

needs_vaccination()은 목록에 있는 학생이 예방 접종을 받지 않았으면 true, 이미 받았으면 false를 반환하고, 목록에 없으면 true를 반환한다. [U25-016] (K A.4.1 예방 접종, PDF p.456)

부분 **퀵 정렬**은 **퀵 정렬 알고리즘**을 변형하며, 마지막 피벗 위치가 k보다 작으면 오른쪽 부분 벡터도 정렬한다. [U25-017] (K A.4.2 부분 정렬, PDF p.458-459)

워드카운트의 맵 함수는 각 입력 파일에서 단어를 추출하고, key를 단어로, value를 1로 설정한 쌍을 생성한다. [U25-018] (K A.4.3 맵리듀스를 이용하여 워드카운트 구현하기, PDF p.462)

리듀스 작업에서는 std::accumulate()로 key에 해당하는 단어가 나타나는 총 횟수를 구한다. [U25-019] (K A.4.3 맵리듀스를 이용하여 워드카운트 구현하기, PDF p.463)

A.5 그리디 알고리즘 실습

작업 스케줄링은 작업 종료 시간을 기준으로 정렬한 뒤 현재 작업과 시간이 겹치는 작업을 제거한다. [U25-020] (K A.5.1 작업 스케줄링 문제, PDF p.464-465)

웰시-포웰 구현은 정점의 **차수** 기준으로 내림차순 정렬하고, 인접 정점에 현재 색상이 없으면 해당 색상을 지정한다. [U25-021] (K A.5.2 웰시-포웰 알고리즘, PDF p.467)

A.6 그래프 알고리즘 I 실습

이분 그래프 판별 실습은 **DFS**를 이용하여 주어진 **그래프**가 **이분 그래프**인지 확인한다. [U25-022] (K A.6.1 이분 그래프 판별하기, PDF p.469)

뉴욕 최단 경로 실습은 USA-road-d.NY.gr 파일을 읽어 **그래프 객체**를 생성하고 다익스트라 **알고리즘**을 구현한다. [U25-023] (K A.6.2 뉴욕에서 최단 경로 찾기, PDF p.472,474)

A.7 그래프 알고리즘 II 실습

욕심쟁이 로봇에서 에너지를 최대로 유지하는 것은 이동 시 에너지 손실을 최소로 하는 것과 같으므로, 실제 구현에서는 전력 값의 부호를 바꾼다. [U25-024] (K A.7.1 욕심쟁이 로봇, PDF p.478)

로봇 풀이의 BellmanFord()는 연습 문제 33과 같으며, 호출할 때 정점 개수를 전체 셀 개수인 N * N으로 지정한다. [U25-025] (K A.7.1 욕심쟁이 로봇, PDF p.479)

무작위 **그래프** 풀이에서는 모든 정점에 들어오거나 나가는 에지가 적어도 하나 존재하는지 인접 행렬의 대각 원소로 확인한다. [U25-026] (K A.7.2 무작위 그래프 통계, PDF p.482)

BellmanFord()가 반환한 거리 벡터가 비어 있지 않으면 **존슨 알고리즘**을 계속 수행하여 에지 가중치를 재설정하고 각각의 정점에서 다익스트라 **알고리즘**을 수행한다. [U25-027] (K A.7.2 무작위 그래프 통계, PDF p.485)

미로-순간이동 게임의 미로·방·순간이동장치·점수는 **그래프**·정점·에지·가중치로 바꾸어 생각하며, 점수를 무한히 낮출 수 있는 상황은 **음수 가중치 사이클**이 있는 경우이다. [U25-028] (K A.7.3 미로-순간이동 게임, PDF p.489)

다른 구역으로 이동하지 못하는 방을 찾으려면 **강한 연결 요소** 중에서 다른 **연결 요소**로 나가는 에지가 없는 것들을 찾고, 코사라주 구현에 필요한 로직을 추가한다. [U25-029] (K A.7.3 미로-순간이동 게임, PDF p.492)

A.8 동적 계획법 I 실습

여행 경로 풀이의 **기저 조건**은 목적지에 도착했는지를 판단하는 것이며, i번째 도시의 이동 거리 제한이 x이면 (i + 1)번째부터 (i + x)번째 도시 사이로 이동할 수 있다. [U25-030] (K A.8.1 여행 경로, PDF p.494)

여행 경로의 추가 점수 풀이에서는 **구간 합 배열**(prefix sum array)을 사용하여 안쪽 for 반복문으로 계산하던 구간 합을 **상수** 시간에 계산한다. [U25-031] (K A.8.1 여행 경로, PDF p.499)

LCS_Memoization()은 두 문자열의 부분 문자열을 재귀적으로 탐색하며 각 단계에서 memo 테이블에 결과를 저장한다. [U25-032] (K A.8.2 메모이제이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.501)

LCS_Tabulation()은 문자열 A와 B를 인자로 받고 **최장 공통 부분 시퀀스** 문자열을 반환한다. [U25-033] (K A.8.3 타뷸레이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.505)

최장 공통 부분 시퀀스의 길이는 DP[A.size()][B.size()]에서 알 수 있고, 실제 시퀀스는 DP 테이블에 **백트래킹**을 수행하여 구한다. [U25-034] (K A.8.3 타뷸레이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.507)

멜로디 순열 풀이의 **상태**는 지금까지 고려한 멜로디의 길이와, 현재 음을 추가하거나 추가하지 않은 부분집합의 두 차원으로 표현한다. [U25-035] (K A.8.4 멜로디 순열, PDF p.511)

멜로디 순열 풀이에서는 각각의 음을 2의 승수로 간주하고 C++ 비트 단위 연산으로 **상태**에 해당하는 부분집합을 표현한다. [U25-036] (K A.8.4 멜로디 순열, PDF p.512)

A.9 동적 계획법 II 실습

최대 이익 실습에서는 선택한 상품 가격 합이 budget을, 수량이 capacity를 넘지 않으면서 이익 합이 최대가 되도록 한다. [U25-037] (K A.9.1 최대 이익, PDF p.518)

최대 이익의 상향식 **상태**는 현재 인덱스, 상품 구매 비용, 상품 개수로 정의되며, 구현은 0-1 **배낭 문제** 풀이에서 차원이 하나 추가된 형태이다. [U25-038] (K A.9.1 최대 이익, PDF p.519-520)

도로 건설에서 모든 집이 단지 입구까지 가장 낮은 비용으로 이동하도록 플로이드-워셜 **알고리즘**을 사용하고, 경로 재구성을 위한 next 행렬을 둔다. [U25-039] (K A.9.2 도로 건설, PDF p.525)

플로이드-워셜의 최단 거리가 갱신될 때마다 next[start][end]도 next[start][mid] 값으로 갱신한다. [U25-040] (K A.9.2 도로 건설, PDF p.526)

동작 방식·절차

수록된 최소 힙 병합 풀이의 순서

1. 각 리스트의 가장 작은 원소를 추출하여 최소 힙을 구성한다. [U25-041] (K A.2 트리, 힙, 그래프: 병합 풀이 수록 부분, PDF p.445)

2. 힙에서 가장 작은 원소를 꺼내어 출력 벡터에 추가한다. [U25-042] (K A.2 트리, 힙, 그래프: 병합 풀이 수록 부분, PDF p.445)

3. 꺼낸 원소가 있던 리스트에 다음 원소가 있으면 힙에 넣고, 힙이 완전히 빌 때까지 반복한다. [U25-043] (K A.2 트리, 힙, 그래프: 병합 풀이 수록 부분, PDF p.445)

도로 건설 풀이의 기본 단계

1. 입력을 처리하고 **그래프**를 구성한다(인접 정점 찾기, 가중치 설정하기 등). [U25-044] (K A.9.2 도로 건설, PDF p.520)

2. 그래프 정점 사이의 최단 거리를 찾는다. [U25-045] (K A.9.2 도로 건설, PDF p.521)

3. 경로를 재구성한다. [U25-046] (K A.9.2 도로 건설, PDF p.521)

4. 지도를 다시 그린다. [U25-047] (K A.9.2 도로 건설, PDF p.521)

주요 문법·표·수식

파일 시스템 노드 — 원문 발췌 (원문 발췌)

```
struct n_ary_node
{
    std::string name;
    bool is_dir;
    std::vector<n_ary_node*> children;
};
```

[U25-048] (K A.2.1 파일 시스템 자료 구조 만들기, PDF p.441)

n_ary_node는 디렉터리/파일 정보를 저장하며 name, is_dir, children을 가진다. [U25-009] (K A.2.1 파일 시스템 자료 구조 만들기, PDF p.441)

인쇄 대기열 처리 — 원문 발췌 (원문 발췌)

```
void startPrinting()
{
    while (not jobs.empty())
    {
        std::cout << "인쇄 중: " << jobs.front() << std::endl;
        jobs.pop();
    }
}
```

[U25-049] (K A.1.3 사무실 공유 프린터의 인쇄 대기 목록 시뮬레이션, PDF p.439)

startPrinting()은 대기열이 빌 때까지 jobs.front()를 출력하고 jobs.pop()으로 제거한다. [U25-007] (K A.1.3 사무실 공유 프린터의 인쇄 대기 목록 시뮬레이션, PDF p.439)

구간 합 배열: sum[i + 1] = sum[i] + distance[i]
[L, R] 구간의 원소 합: sum[R+1] - sum[L]; 합을 구할 전체 데이터 개수보다 하나 큰 **배열** sum을 선언하고 모든 원소를 0으로 **초기화**한다. [U25-050] (K A.8.1 여행 경로, PDF p.499)

멜로디 음 부분집합 수: $2^{12} = 4096$; 각 음을 2의 승수로 간주하여 비트 단위 연산으로 부분집합을 표현한다. [U25-051] (K A.8.4 멜로디 순열, PDF p.512)

부록 LCS 구현의 인터페이스

함수	원문에서 설명한 반환값·처리
LCS_Memoization() [U25-052] (K A.8.2 메모이제이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.501)	int형 LCS 길이; 2차원 정수 벡터 참조형 memo에 결과를 저장한다. [U25-053] (K A.8.2 메모이제이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.501)
LCS_Tabulation() [U25-054] (K A.8.3 타뷸레이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.505)	string형 **최장 공통 부분 시퀀스** 문자열을 반환한다. [U25-055] (K A.8.3 타뷸레이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.505)

핵심 용어 표

용어	원문에 나온 정의·설명
n_ary_node	**n_ary_node**는 디렉터리/파일 정보를 저장하며 name, is_dir, children을 가진다. [U25-009] (K A.2.1 파일 시스템 자료 구조 만들기, PDF p.441)
URLService	**URLService**는 짧은 URL을 키로, 원본 URL을 데이터 값으로 사용하여 맵에 함께 저장한다. [U25-012] (K A.3.1 긴 URL을 짧은 URL로 매핑하기, PDF p.447)
lookup()	**lookup**()은 짧은 URL을 이용하여 긴 원본 URL을 찾는 **룩업** 함수이다. [U25-013] (K A.3.1 긴 URL을 짧은 URL로 매핑하기, PDF p.447)
BloomFilter	이메일 주소 중복 검사 풀이에서는 **BloomFilter 클래스**를 추가하고 MD5() 함수로 해시 값을 계산한다. [U25-014] (K A.3.2 이메일 주소 중복 검사, PDF p.451-452)
구간 합 배열(prefix sum array)	여행 경로의 추가 점수 풀이에서는 **구간 합 배열**(prefix sum array)을 사용하여 안쪽 for 반복문으로 계산하던 구간 합을 **상수** 시간에 계산한다. [U25-031] (K A.8.1 여행 경로, PDF p.499)
LCS_Memoization()	**LCS_Memoization**()은 두 문자열의 부분 문자열을 재귀적으로 탐색하며 각 단계에서 memo 테이블에 결과를 저장한다. [U25-032] (K A.8.2 메모이제이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.501)
LCS_Tabulation()	**LCS_Tabulation**()은 문자열 A와 B를 인자로 받고 **최장 공통 부분 시퀀스** 문자열을 반환한다. [U25-033] (K A.8.3 타뷸레이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.505)

단원 요약

요약 기준: 편집 요약: 부록에는 별도 나가며 절이 없어 각 실습 풀이를 재압축함.

Printer **클래스**는 요청된 순서대로 프린트하기 위해 std::queue를 사용하고, 최대 인쇄 요청 개수를 템플릿 인자로 받는다. [U25-006] (K A.1.3 사무실 공유 프린터의 인쇄 대기 목록 시뮬레이션, PDF p.439)

URLService는 짧은 URL을 키로, 원본 URL을 데이터 값으로 사용하여 맵에 함께 저장한다. [U25-012] (K A.3.1 긴 URL을 짧은 URL로 매핑하기, PDF p.447)

미로-순간이동 게임의 미로·방·순간이동장치·점수는 **그래프**·정점·에지·가중치로 바꾸어 생각하며, 점수를 무한히 낮출 수 있는 상황은 **음수 가중치 사이클**이 있는 경우이다. [U25-028] (K A.7.3 미로-순간이동 게임, PDF p.489)

최대 이익의 상향식 **상태**는 현재 인덱스, 상품 구매 비용, 상품 개수로 정의되며, 구현은 0-1 **배낭 문제** 풀이에서 차원이 하나 추가된 형태이다. [U25-038] (K A.9.1 최대 이익, PDF p.519-520)

원문 기반 복습 질문과 정답

01 개발환경·변수·기본 문법

01-1. 컴파일러와 링커가 각각 변환하는 것은 무엇인가? (L01 Terminologies, 슬라이드 5)

01-2. 변수의 초기화란 무엇인가? (L01 Definition and Initialization, 슬라이드 42)

01-3. 배열의 첫 인덱스와 마지막 인덱스는 무엇인가? (L02 Arrays, 슬라이드 6)

01-4. continue와 break는 반복문에서 각각 어떻게 동작하는가? (L02 Looping Statement, 슬라이드 39)

정답과 원문 근거

1. **컴파일러**는 소스 코드를 오브젝트 코드로, **링커**는 오브젝트 코드를 실행파일(exe)로 변환한다. [U01-056] (L01 Terminologies, 슬라이드 5)

2. **변수**를 정의하면서 초기값을 설정하는 것을 **초기화**라고 한다. [U01-018] (L01 Definition and Initialization, 슬라이드 42)

3. 첫 인덱스는 0이고 마지막 인덱스는 size-1이다. [U01-057] (L02 Arrays, 슬라이드 6)

4. continue는 다음 iteration으로 넘어가고, break는 루프 밖으로 빠져나간다. [U01-058] (L02 Looping Statement, 슬라이드 39)

02 함수·포인터·참조·메모리

02-1. 함수의 반환 타입만 다르게 하여 오버로딩할 수 있는가? (L03 Function Overloading, 슬라이드 34)

02-2. 참조자를 한 번 초기화한 뒤 다른 변수의 참조자로 바꿀 수 있는가? (L04 Reference, 슬라이드 42)

02-3. 댕글링 레퍼런스(Dangling reference)란 무엇인가? (B 3장·지역변수의 레퍼런스를 리턴?, 인쇄 p.42)

02-4. new []로 할당한 배열은 무엇으로 해제하는가? (B 4장·new 로 배열 할당하기, 인쇄 p.51)

정답과 원문 근거

1. 반환 타입만 다른 오버로딩은 불가능하다. [U02-055] (L03 Function Overloading, 슬라이드 34)

2. 참조자는 **변수**의 별명이며 선언과 동시에 **초기화**되어야 하고, 한 번 **초기화**되면 다른 **변수**의 **참조자**가 될 수 없다. [U02-021] (L04 Reference, 슬라이드 42)

3. 댕글링 레퍼런스(Dangling reference)는 레퍼런스는 있지만 원래 참조하던 것이 사라진 레퍼런스이다. [U02-024] (B 3장·지역변수의 레퍼런스를 리턴?, 인쇄 p.42)

4. new []로 할당한 **배열**은 delete []로 해제한다. [U02-056] (B 4장·new 로 배열 할당하기, 인쇄 p.51)

03 클래스·생성·복사·const

03-1. 클래스와 객체는 각각 무엇인가? (L05A Classes and Objects, 슬라이드 14-15)

03-2. 클래스 안의 레퍼런스 변수나 상수는 어떻게 초기화해야 하는가? (B 5장·const, const, const!·생성자의 초기화 리스트, 인쇄 p.113)

03-3. 얕은 복사와 깊은 복사는 포인터가 가리키는 데이터를 어떻게 다루는가? (L05B Shallow vs Deep Copy, 슬라이드 14,20)

03-4. explicit과 mutable은 각각 어떤 동작을 지정하는가? (B 5장·클래스의 explicit 과 mutable 키워드, 인쇄 p.183)

정답과 원문 근거

1. 클래스는 **객체** 생성의 틀이자 사용자 정의 자료형이고, **객체**는 **클래스**로부터 생성된 실체이다. [U03-054] (L05A Classes and Objects, 슬라이드 14-15)

2. 생성자의 **초기화** 리스트를 사용하여 **초기화**해야 한다. [U03-055] (B 5장·const, const, const!·생성자의 초기화 리스트, 인쇄 p.113)

3. 얕은 복사는 주소값을 복사하고, **깊은 복사**는 새 힙 공간에 데이터를 복사한다. [U03-056] (L05B Shallow vs Deep Copy, 슬라이드 14,20)

4. explicit은 해당 **생성자**를 이용한 암시적 변환을 막고, **mutable**은 const 함수 안에서 해당 멤버**변수**를 바꿀 수 있게 한다. [U03-057] (B 5장·클래스의 explicit 과 mutable 키워드, 인쇄 p.183)

04 상속·다형성

04-1. 유도 클래스 객체의 생성자와 소멸자는 어떤 순서로 호출되는가? (L06 (Summary) Inheritance, 슬라이드 53)

04-2. 정적 바인딩과 동적 바인딩은 호출 함수를 언제 결정하는가? (L07 Polymorphism, 슬라이드 13)

04-3. 순수가상함수를 오버라이드하지 않은 유도 클래스는 객체를 만들 수 있는가? (L07 Pure Virtual Functions and Abstract Classes, 슬라이드 33,35)

04-4. 다이아몬드 상속에서 Human의 중복 포함을 원문은 어떻게 해결하는가? (B 7장·상속에 관련된 잡다한 내용들·가상 상속, 인쇄 p.337)

정답과 원문 근거

1. 생성은 기본-유도, 소멸은 유도-기본 순서이다. [U04-057] (L06 (Summary) Inheritance, 슬라이드 53)

2. **정적 바인딩**은 **컴파일** 시, **동적 바인딩**은 런타임에 결정한다. [U04-058] (L07 Polymorphism, 슬라이드 13)

3. 그 **유도 클래스**도 **추상 클래스**로 간주되므로 **객체**를 생성할 수 없다. [U04-059] (L07 Pure Virtual Functions and Abstract Classes, 슬라이드 33,35)

4. HandsomeHuman과 SmartHuman이 Human을 virtual로 **상속**하도록 하여 Me에 Human을 한 번만 포함시킨다. [U04-060] (B 7장·상속에 관련된 잡다한 내용들·가상 상속, 인쇄 p.337)

05 연산자 오버로딩·N차원 배열 프로젝트

05-1. 연산자 오버로딩으로 연산 우선순위를 바꾸거나 새로운 연산자를 정의할 수 있는가? (L08 Operator Overloading, 슬라이드 11)

05-2. 전위와 후위 증가 연산자는 어떻게 구분하여 선언하는가? (B 6장·전위/후위 증감 연산자, 인쇄 p.236)

05-3. 스트림 삽입 연산자가 ostream 참조자를 반환하는 이유는 무엇인가? (L08 Stream Insertion and Extraction Overloading, 슬라이드 31)

05-4. N차원 Array의 반복자는 무엇을 위해 추가하며 end는 어디를 뜻하는가? (B 6장·N 차원 배열 만들기·반복자, 인쇄 p.260-261)

정답과 원문 근거

1. 연산 우선순위를 바꿀 수 없고 새로운 연산자를 정의할 수도 없다. [U05-051] (L08 Operator Overloading, 슬라이드 11)

2. 전위는 operator++(), 후위는 의미 없는 int 매개**변수**를 둔 operator++(int) 형태로 구분한다. [U05-052] (B 6장·전위/후위 증감 연산자, 인쇄 p.236)

3. cout << p1 << p2 같은 chain insertion을 가능하게 하기 위해서이다. [U05-053] (L08 Stream Insertion and Extraction Overloading, 슬라이드 31)

4. 반복자는 모든 원소를 순차 접근하기 위해 추가하며, end는 마지막 원소 바로 다음을 뜻한다. [U05-054] (B 6장·N 차원 배열 만들기·반복자, 인쇄 p.260-261)

06 표준 입출력·파일·엑셀 프로젝트

06-1. ios_base와 ios가 각각 처리하는 내용은 무엇인가? (B C++ 입출력 라이브러리, 인쇄 p.341)

06-2. fail 상태의 입력을 복구할 때 clear와 ignore는 각각 무엇을 하는가? (B istream 클래스, 인쇄 p.348)

06-3. 파일 전체를 읽을 때 seekg와 tellg는 어떤 역할을 하는가? (B 파일 전체 읽기, 인쇄 p.362)

06-4. 후위 표기법에서 연산자를 만나면 스택을 어떻게 처리하는가? (B 엑셀 만들기 프로젝트 2부 / 중위 표기법과 후위 표기법, 인쇄 p.398)

정답과 원문 근거

1. ios_base는 스트림의 입출력 형식 관련 데이터를 처리하고, ios는 **스트림 버퍼**를 **초기화**하며 입출력 작업의 **상태**를 처리한다. [U06-003] (B C++ 입출력 라이브러리, 인쇄 p.341)

2. clear()에 인자를 주지 않으면 플래그를 goodbit으로 **초기화**하고, ignore는 최대 첫 번째 인자만큼, 두 번째 인자가 나올 때까지 그 문자를 포함하여 버퍼에서 무시한다. [U06-009] (B istream 클래스, 인쇄 p.348)

3. seekg는 파일 위치지정자를 옮기며, tellg는 시작지점으로부터 위치지정자의 위치를 반환한다. [U06-019] (B 파일 전체 읽기, 인쇄 p.362)

4. 두 피연산자를 pop하여 연산하고 결과를 다시 push한다. [U06-052] (B 엑셀 만들기 프로젝트 2부 / 중위 표기법과 후위 표기법, 인쇄 p.398)

07 템플릿·가변 인자·메타프로그래밍

07-1. 템플릿 인스턴스화란 무엇인가? (B C++ 템플릿(template), 인쇄 p.418)

07-2. 파라미터 팩과 sizeof...이 각각 나타내는 것은 무엇인가? (B 가변 길이 템플릿 / sizeof..., 인쇄 p.446,453)

07-3. Factorial의 재귀 구조는 어떻게 끝나는가? (B 템플릿 메타프로그래밍 1부, 인쇄 p.464-465)

07-4. 의존 타입에 typename을 붙이는 이유는 무엇인가? (B 템플릿 메타프로그래밍 2부 / 지난 강의 생각해보기 문제, 인쇄 p.477-478)

정답과 원문 근거

1. 클래스 템플릿에 인자를 전달하여 실제 코드를 생성하는 것을 **클래스 템플릿 인스턴스화**(class template instantiation)라고 한다. [U07-007] (B C++ 템플릿(template), 인쇄 p.418)

2. 파라미터 팩은 여러 인자를 나타내고 sizeof...은 그 인자의 개수를 반환한다. [U07-044] (B 가변 길이 템플릿 / sizeof..., 인쇄 p.446,453)

3. Factorial은 재귀적인 템플릿 구조를 사용하고 Factorial<1>의 특수화로 재귀 구조를 끝낸다. [U07-025] (B 템플릿 메타프로그래밍 1부, 인쇄 p.464-465)

4. 의존 타입(dependent type)은 템플릿 인자에 따라 달라질 수 있는 타입이며, 원문은 의존하는 식별자가 타입임을 알리기 위해 typename을 붙인다. [U07-028] (B 템플릿 메타프로그래밍 2부 / 지난 강의 생각해보기 문제, 인쇄 p.477-478)

08 STL 컨테이너·알고리즘·문자열

08-1. begin과 end가 가리키는 위치는 무엇인가? (B 반복자(iterator), 인쇄 p.500)

08-2. map의 []로 없는 키를 참조하면 어떤 일이 생기는가? (B 맵(map), 인쇄 p.538,540,542)

08-3. remove 다음에 erase가 필요한 이유는 무엇인가? (B 원소 제거(remove, remove_if), 인쇄 p.568)

08-4. string_view가 참조하는 문자열의 수명을 확인해야 하는 이유는 무엇인가? (B string_view / 소유하지 않고 읽기만 한다!, 인쇄 p.609)

정답과 원문 근거

1. begin()은 첫 원소를, end()는 마지막 원소 한 칸 뒤를 가리키며, begin()==end()이면 빈 컨테이너를 나타낸다. [U08-009] (B 반복자(iterator), 인쇄 p.500)

2. map의 []는 없는 키를 참조하면 값의 디폴트 **생성자**를 호출하여 원소를 추가하고, 기존 키이면 대응 값을 바꿀 수 있다. [U08-022] (B 맵(map), 인쇄 p.538,540,542)

3. remove는 원소를 이동할 뿐 실제 삭제하지 않으므로, 컨테이너에서 지우려면 erase를 호출한다. [U08-031] (B 원소 제거(remove, remove_if), 인쇄 p.568)

4. string_view는 문자열을 소유하지 않고 참조해서 읽으며, 가리키는 문자열이 소멸한 뒤 사용하면 Undefined behavior가 발생한다. [U08-042] (B string_view / 소유하지 않고 읽기만 한다!, 인쇄 p.609)

09 예외 처리

09-1. throw 이후 같은 함수의 문장들이 실행되는가? (B 예외 발생시키기 - throw, 인쇄 p.616)

09-2. 스택 풀기란 무엇인가? (B 스택 풀기(stack unwinding), 인쇄 p.621)

09-3. Child와 Parent 예외를 별도로 받으려면 catch 순서를 어떻게 두는가? (B 여러 종류의 예외 받기, 인쇄 p.624-625)

09-4. noexcept 함수에서 예외가 발생하면 어떻게 되는가? (B 예외를 발생시키지 않는 함수 - noexcept, 인쇄 p.628)

정답과 원문 근거

1. throw를 실행하면 그 위치에서 즉시 함수를 빠져나가 **예외** 처리 부분으로 이동하므로 아래 문장들은 실행하지 않는다. [U09-009] (B 예외 발생시키기 - throw, 인쇄 p.616)

2. 스택 풀기(stack unwinding)는 **catch**로 이동하면서 **스택**상에 정의된 **객체**들을 소멸시키는 과정이다. [U09-014] (B 스택 풀기(stack unwinding), 인쇄 p.621)

3. 원문은 파생 **클래스 예외**를 받는 **catch**를 기반 **클래스 catch**보다 앞에 두어 각각 처리한다. [U09-019] (B 여러 종류의 예외 받기, 인쇄 p.624-625)

4. noexcept 함수가 실제 **예외**를 발생시키면 **예외**가 제대로 처리되지 않고 프로그램이 종료된다. [U09-023] (B 예외를 발생시키지 않는 함수 - noexcept, 인쇄 p.628)

10 우측값·이동·완벽한 전달

10-1. 이름이 있는 우측값 레퍼런스 매개변수 str 자체는 어떤 값인가? (B 우측값 레퍼런스, 인쇄 p.641)

10-2. 이동 생성자에서 원본 포인터를 nullptr로 바꾸는 이유는 무엇인가? (B 우측값 레퍼런스, 인쇄 p.642)

10-3. std::move 자체가 이동을 수행하는가? (B move 함수(move semantics), 인쇄 p.655-657)

10-4. 완벽한 전달에서 std::forward를 사용하는 이유는 무엇인가? (B 보편적 레퍼런스(Universal reference), 인쇄 p.669-670)

정답과 원문 근거

1. 우측값 레퍼런스 매개**변수** str은 이름이 있으므로 **표현식** 자체는 좌측값이다. [U10-010] (B 우측값 레퍼런스, 인쇄 p.641)

2. 이동한 원본의 string_content를 **nullptr**로 바꾸어 원본 **소멸자**가 새 **객체**도 가리키는 메모리를 해제하지 않게 한다. [U10-012] (B 우측값 레퍼런스, 인쇄 p.642)

3. std::move는 <utility>에서 제공되며, 이동 자체를 수행하지 않고 받은 **객체**를 우측값으로 변환한다. [U10-016] (B move 함수(move semantics), 인쇄 p.655-657)

4. std::forward<T>(u)는 원래 우측값인 인자는 우측값으로, 좌측값인 인자는 좌측값으로 전달하여 원문과 같은 g 오버로드가 호출되게 한다. [U10-028] (B 보편적 레퍼런스(Universal reference), 인쇄 p.669-670)

11 스마트 포인터·함수 객체

11-1. RAII는 자원 관리를 무엇을 통해 수행하는가? (B Resource Acquisition Is Initialization - RAII, 인쇄 p.674)

11-2. unique_ptr의 소유권을 옮긴 뒤 원래 포인터는 어떤 상태인가? (B unique_ptr 소유권 이전하기, 인쇄 p.679-680)

11-3. weak_ptr의 lock은 객체 소멸 여부에 따라 무엇을 반환하는가? (B weak_ptr, 인쇄 p.702)

11-4. bind에 복사 대신 참조를 전달하려면 무엇을 사용하는가? (B std::bind, 인쇄 p.717-718)

정답과 원문 근거

1. RAII(Resource Acquisition Is Initialization)는 **자원** 관리를 **스택**에 할당한 **객체**를 통해 수행하는 방식이다. [U11-006] (B Resource Acquisition Is Initialization - RAII, 인쇄 p.674)

2. unique_ptr는 이동 생성이 가능하며, **std::move**로 소유권을 옮기면 원래 **포인터**는 아무것도 가리키지 않는다. [U11-012] (B unique_ptr 소유권 이전하기, 인쇄 p.679-680)

3. weak_ptr의 lock은 **객체**가 살아 있으면 그 **객체**의 **shared_ptr**, 이미 해제되었으면 빈 **shared_ptr**를 반환한다. [U11-025] (B weak_ptr, 인쇄 p.702)

4. bind에 전달한 값은 복사되므로 참조 전달이 필요하면 std::ref를 사용하고, const 레퍼런스에는 std::cref를 사용한다. [U11-032] (B std::bind, 인쇄 p.717-718)

12 쓰레드·동기화·atomic·비동기·ThreadPool

12-1. join 은 언제 리턴하는가? (B 16장 / join, 인쇄 p.730)

12-2. lock_guard 는 언제 lock/unlock 하는가? (B 16장 / lock_guard, 인쇄 p.748)

12-3. future::get 을 두 번 호출하면 안 되는 이유는? (B 16장 / std::promise 와 std::future, 인쇄 p.795)

12-4. WorkerThread 는 어떤 조건에서 종료하는가? (B 16장 / ThreadPool 첫번째 버전, 인쇄 p.809)

정답과 원문 근거

1. join 은 해당 쓰레드가 실행을 종료하면 리턴한다. [U12-006] (B 16장 / join, 인쇄 p.730)

2. lock_guard 는 **생성자**에서 뮤텍스를 lock 하고 소멸할 때 unlock 한다. [U12-012] (B 16장 / lock_guard, 인쇄 p.748)

3. future 의 get 은 안의 데이터를 이동하므로 두 번 호출하면 안 된다. [U12-028] (B 16장 / std::promise 와 std::future, 인쇄 p.795)

4. WorkerThread 는 작업이 생기거나 stop_all 이 설정될 때까지 기다리고, stop_all 이 참이며 큐가 비면 종료한다. [U12-036] (B 16장 / ThreadPool 첫번째 버전, 인쇄 p.809)

13 유니폼 초기화·constexpr·decltype

13-1. A a(); 는 어떻게 해석되는가? (B 17장 / C++ 유니폼 초기화, 인쇄 p.826)

13-2. const 와 constexpr 의 차이는? (B 18장 / constexpr vs const, 인쇄 p.838-839)

13-3. decltype((a)) 가 int& 가 되는 이유는? (B 19장 / decltype 의 쓰임새, 인쇄 p.858)

13-4. std::declval 을 런타임에서 사용할 수 있는가? (B 19장 / std::declval, 인쇄 p.863)

정답과 원문 근거

1. A a(); 는 **객체** a 를 만드는 대신, A 를 리턴하고 인자를 받지 않는 함수 a 로 해석된다. [U13-003] (B 17장 / C++ 유니폼 초기화, 인쇄 p.826)

2. const 로 정의된 **상수**는 **컴파일** 타임에 값을 알 필요가 없지만, **constexpr 변수**의 **초기화**에는 다른 **상수**식이 와야 한다. [U13-013] (B 18장 / constexpr vs const, 인쇄 p.838-839)

3. 원문에서 **decltype**((a)) 는 lvalue 에 대한 타입 추론이므로 int& 가 된다. [U13-028] (B 19장 / decltype 의 쓰임새, 인쇄 p.858)

4. std::declval<T>() 는 T 의 **생성자**를 직접 호출하지 않고 타입 연산에 **객체**를 나타낼 수 있으며, 런타임에서 사용하면 오류가 난다. [U13-031] (B 19장 / std::declval, 인쇄 p.863)

14 type_traits·정규표현식·난수·시간·파일시스템·유틸리티

14-1. SFINAE 는 치환 오류를 어떻게 처리하는가? (B 20장 / SFINAE, 인쇄 p.878)

14-2. regex_match 의 매칭 범위는? (B 20장 / 전체 문자열 매칭하기, 인쇄 p.896)

14-3. filesystem 과 fstream 은 무엇을 담당하는가? (B 20장 / 파일시스템 라이브러리 소개, 인쇄 p.921)

14-4. structured binding 은 원소 일부만 받을 수 있는가? (B 20장 / Structured binding, 인쇄 p.953)

정답과 원문 근거

1. SFINAE 는 Substitution Failure Is Not An Error 이며, 문법에 맞지 않는 템플릿 인자 치환 결과를 **함수 오버로딩** 후보에서 제외한다. [U14-007] (B 20장 / SFINAE, 인쇄 p.878)

2. regex_match 는 문자열 전체가 **정규표현식**의 패턴에 부합하면 true 를 리턴한다. [U14-011] (B 20장 / 전체 문자열 매칭하기, 인쇄 p.896)

3. fstream 은 파일 데이터의 입출력을 담당하고, filesystem 은 파일 메타데이타에 대한 접근을 돕는다. [U14-002] (B 20장 / 파일시스템 라이브러리 소개, 인쇄 p.921)

4. structured binding 은 **auto** [age, name, is_male] = student; 처럼 tuple 의 원소들을 받으며, 모든 원소를 받아야 한다. [U14-038] (B 20장 / Structured binding, 인쇄 p.953)

15 Visual Studio 디버깅·Make

15-1. F10 과 F11 은 어떤 차이가 있는가? (L02D Debugging Tool 1 - Break point, 슬라이드 6)

15-2. 중단점의 노란 화살표 명령문은 실행되었는가? (L02D Debugging Tool 1 - Break point, 슬라이드 5)

15-3. recipe 의 들여쓰기 조건은? (B 22장 / 실행할 명령어(recipes), 인쇄 p.964)

15-4. .PHONY: clean 은 왜 등록하는가? (B 22장 / PHONY, 인쇄 p.969)

정답과 원문 근거

1. F10 은 **명령문**을 실행하고 다음 줄로 진행하는 step over 이고, F11 은 호출되는 추가 코드로 들어가는 step into 이다. [U15-007] (L02D Debugging Tool 1 - Break point, 슬라이드 6)

2. 노란 화살표는 현재 중단된 위치를 표시하며, 그 **명령문**은 아직 실행되지 않은 **상태**이다. [U15-006] (L02D Debugging Tool 1 - Break point, 슬라이드 5)

3. recipe 의 명령어는 반드시 탭으로 한 번 들여쓴다. [U15-018] (B 22장 / 실행할 명령어(recipes), 인쇄 p.964)

4. .**PHONY**: clean 으로 등록하면 clean 파일의 존재와 관계없이 타겟의 명령을 실행한다. [U15-023] (B 22장 / PHONY, 인쇄 p.969)

16 리스트, 스택, 큐

16-1. 연결 리스트에서 임의 원소 접근이 O(n)인 이유는 무엇인가? (K 1.2.2 연결된 자료 구조, PDF p.14)

16-2. reserve(capacity)는 벡터 크기를 바꾸는가? (K 1.4.1 std::vector - 가변 크기 배열, PDF p.32)

16-3. 벡터에서 메모리를 재할당하면 기존 반복자는 어떻게 되는가? (K 1.7.4 반복자 무효화, PDF p.51)

16-4. std::queue와 스택이 사용하는 처리 순서는 각각 무엇인가? (K 1.9 컨테이너 어댑터·1.9.2 std::queue, PDF p.56,58)

정답과 원문 근거

1. 연결 리스트의 원소에 접근하려면 헤드부터 next를 따라 이동하므로 원소 접근은 O(n)이다. [U16-008] (K 1.2.2 연결된 자료 구조, PDF p.14)

2. reserve(capacity)는 벡터에서 사용할 용량을 지정하며 크기를 변경하지 않는다. [U16-016] (K 1.4.1 std::vector - 가변 크기 배열, PDF p.32)

3. 벡터의 메모리 재할당은 기존 **반복자**·**포인터**·참조를 무효화한다. [U16-024] (K 1.7.4 반복자 무효화, PDF p.51)

4. std::queue는 FIFO, **스택**은 LIFO이다. [U16-049] (K 1.9 컨테이너 어댑터·1.9.2 std::queue, PDF p.56,58)

17 트리, 힙, 그래프

17-1. 전위·중위·후위 순회의 방문 순서는 무엇인가? (K 2.3.2 트리 순회, PDF p.70-71)

17-2. BST 검색이 항상 O(log N)인가? (K 2.4.2 트리 연산의 시간 복잡도, PDF p.78)

17-3. 힙 생성 알고리즘의 시간 복잡도는 무엇인가? (K 2.5.1 힙 연산, PDF p.91)

17-4. 인접 행렬과 달리 인접 리스트가 저장하는 것은 무엇인가? (K 2.6.3 인접 리스트로 그래프 표현하기, PDF p.101)

정답과 원문 근거

1. 전위: 현재·왼쪽·오른쪽, 중위: 왼쪽·현재·오른쪽, 후위: 왼쪽·오른쪽·현재이다. [U17-050] (K 2.3.2 트리 순회, PDF p.70-71)

2. BST의 모양은 원소 삽입 순서에 따라 결정되므로 O(log N) 검색이 항상 성립하지는 않는다. [U17-011] (K 2.4.2 트리 연산의 시간 복잡도, PDF p.78)

3. 힙 생성 **알고리즘**은 아래쪽 서브 트리부터 위로 올라가며 불변성을 맞추고 O(N) 시간에 힙을 **초기화**한다. [U17-020] (K 2.5.1 힙 연산, PDF p.91)

4. 인접 리스트는 각 노드에 직접 연결된 노드의 ID만 저장한다. [U17-027] (K 2.6.3 인접 리스트로 그래프 표현하기, PDF p.101)

18 해시 테이블과 블룸 필터

18-1. 서로 다른 키의 해시 값이 같아지는 현상은 무엇인가? (K 3.2.1 해싱, PDF p.109)

18-2. 부하율만으로 해시 테이블의 성능을 판단할 수 있는가? (K 3.3.2 연습 문제 14, PDF p.119)

18-3. std::unordered_map에서 없는 키에 []를 적용하면 어떻게 되는가? (K 3.4.1 연습 문제 16, PDF p.135)

18-4. 블룸 필터가 없다고 답한 원소는 실제로 없다고 보장되는가? (K 3.5 블룸 필터, PDF p.138)

정답과 원문 근거

1. 충돌은 서로 다른 키에 대해 해시 함수가 같은 해시 값을 반환하는 현상이다. [U18-007] (K 3.2.1 해싱, PDF p.109)

2. 부하율은 해시 테이블 성능을 결정하는 유일한 지표가 아니며 해시 값이 골고루 분포되어야 한다. [U18-011] (K 3.3.2 연습 문제 14, PDF p.119)

3. std::unordered_map의 [] 연산자는 참조를 반환하며 키가 없으면 기본값을 추가하여 반환한다. [U18-022] (K 3.4.1 연습 문제 16, PDF p.135)

4. 블룸 필터의 긍정 답변은 실제로 있을 수도 없을 수도 있음을 뜻하며 부정 답변은 확실히 없음을 뜻한다. [U18-025] (K 3.5 블룸 필터, PDF p.138)

19 분할 정복

19-1. 분할 정복의 세 단계는 무엇인가? (K 4.3 분할 정복 이해하기, PDF p.157)

19-2. 이진 검색에는 왜 결합 과정이 필요하지 않은가? (K 4.3 분할 정복 이해하기, PDF p.158)

19-3. 퀵 정렬에서 분할 후 피벗의 위치는 어떤 의미인가? (K 4.4.3 퀵 정렬, PDF p.164)

19-4. 맵과 리듀스의 차이는 무엇인가? (K 4.6.1 맵과 리듀스 추상화, PDF p.180)

정답과 원문 근거

1. 분할 정복은 분할(divide), 정복(conquer), 결합(combine)의 세 단계로 이루어진다. [U19-007] (K 4.3 분할 정복 이해하기, PDF p.157)

2. 이진 검색은 작은 범위에서 구한 해답이 전체 해답과 같으므로 일반적인 결합 과정이 필요하지 않다. [U19-008] (K 4.3 분할 정복 이해하기, PDF p.158)

3. 분할 후 피벗은 **배열** 전체를 정렬했을 때 실제로 있어야 하는 위치로 이동한다. [U19-015] (K 4.4.3 퀵 정렬, PDF p.164)

4. 맵은 각 원소에 f(x)를 적용하고 **리듀스**는 f(acc,x)를 적용하여 하나의 값으로 축약한다. [U19-045] (K 4.6.1 맵과 리듀스 추상화, PDF p.180)

20 그리디 알고리즘

20-1. 그리디 최적 솔루션에 필요한 두 속성은 무엇인가? (K 5.3.5 그리디 알고리즘의 요구 조건, PDF p.207)

20-2. 분할 가능 배낭 문제에서는 어떤 기준으로 물건을 고르는가? (K 5.3.2 분할 가능 배낭 문제, PDF p.201)

20-3. 크루스칼에서 양 끝 정점의 루트가 같으면 에지를 추가하는가? (K 5.3.7 디스조인트-셋 자료 구조, PDF p.213)

20-4. 웰시-포웰은 어떤 순서로 정점을 처리하는가? (K 5.4.2 실습 문제 12, PDF p.227)

정답과 원문 근거

1. 원문은 **최적 부분 구조**와 **그리디 선택**을 모두 가진 문제에 그리디 방식으로 최적 솔루션을 구할 수 있다고 설명한다. [U20-013] (K 5.3.5 그리디 알고리즘의 요구 조건, PDF p.207)

2. 분할 가능 배낭은 단위 무게당 가격이 높은 순서로 선택한다. [U20-009] (K 5.3.2 분할 가능 배낭 문제, PDF p.201)

3. 크루스칼에서 에지 양 끝의 루트가 같으면 사이클이 발생하므로 그 에지를 MST에 추가하지 않는다. [U20-021] (K 5.3.7 디스조인트-셋 자료 구조, PDF p.213)

4. 웰시-포웰 **알고리즘**은 **차수**가 높은 정점부터 차례대로 **그래프 컬러링**을 수행한다. [U20-026] (K 5.4.2 실습 문제 12, PDF p.227)

21 그래프 알고리즘 I

21-1. BFS와 DFS는 방문하지 않은 정점을 저장할 때 각각 어떤 자료 구조를 사용하는가? (K 6.2.3, PDF p.245)

21-2. DFS로 이분 그래프를 판별할 때 실패하는 조건은 무엇인가? (K 6.2.5, PDF p.252)

21-3. 프림 알고리즘은 경계를 관통하는 에지 중 무엇을 선택하는가? (K 6.3, PDF p.253)

21-4. 다익스트라와 프림이 사용하는 거리 값은 어떻게 다른가? (K 6.4, PDF p.260)

정답과 원문 근거

1. BFS는 선입선출(FIFO) 큐를 사용하고, **DFS**는 후입선출(LIFO) **스택**을 사용하여 인접한 정점들을 재귀적으로 이동하며 방문한다. [U21-013] (K 6.2.3, PDF p.245)

2. DFS로 이동하며 두 색을 번갈아 칠할 수 있으면 **이분 그래프**이며, 같은 색을 할당한 인접한 두 정점을 만나면 **이분 그래프**가 아니다. [U21-017] (K 6.2.5, PDF p.252)

3. 프림 **알고리즘**(Prim's algorithm)은 시작 정점으로 경계를 구성한 뒤 경계를 관통하는 에지 중 가중치가 가장 작은 에지를 선택하여 연결된 정점을 방문한다. [U21-019] (K 6.3, PDF p.253)

4. 프림은 경계로부터의 최소 거리를 사용하지만, 다익스트라는 시작 정점으로부터 각 정점까지의 전체 거리를 사용한다. [U21-024] (K 6.4, PDF p.260)

22 그래프 알고리즘 II

22-1. 다익스트라가 음수 가중치 그래프에서 최적 경로를 놓칠 수 있는 이유는 무엇인가? (K 7.2, PDF p.272)

22-2. 벨만-포드를 수행한 뒤 음수 가중치 사이클을 어떻게 판단하는가? (K 7.4·7.4.1, PDF p.277-278)

22-3. 존슨 알고리즘은 가중치를 변환한 뒤 무엇을 유지하며 어떤 알고리즘을 적용하는가? (K 7.5, PDF p.287)

22-4. 코사라주에서 전치 그래프의 각 DFS가 찾는 정점들은 무엇을 구성하는가? (K 7.7, PDF p.300)

정답과 원문 근거

1. 다익스트라는 방문하지 않은 정점 중 가장 가까운 정점을 선택한 뒤 다시 검토하지 않으므로, 음수 에지를 고려하기 전에 다른 정점을 방문해 최적 경로를 찾지 못할 수 있다. [U22-003] (K 7.2, PDF p.272)

2. 벨만-포드의 (V - 1)번 반복 후 전체 에지를 다시 검사하여 더 짧은 경로가 발견되면 **음수 가중치 사이클**이 있다고 판단한다. [U22-010] (K 7.4·7.4.1, PDF p.277-278)

3. 가중치 변환은 원래의 최단 경로 순서를 유지하면서 음이 아닌 가중치의 **그래프**를 만들며, 각 정점에서 다익스트라를 적용해 모든 정점 쌍의 최단 거리를 구한다. [U22-017] (K 7.5, PDF p.287)

4. 전치 그래프에서는 **스택**에서 꺼낸 정점을 아직 방문하지 않았다면 **DFS**를 시작하며, 각각의 **DFS**에서 만나는 정점들이 **강한 연결 요소**를 구성한다. [U22-026] (K 7.7, PDF p.300)

23 동적 계획법 I

23-1. 동적 계획법으로 해결할 문제에서 원문이 제시한 두 특성은 무엇인가? (K 8.2 동적 계획법이란?, PDF p.318)

23-2. 메모이제이션과 타뷸레이션은 계산을 어디에서 시작하는가? (K 8.3~8.4, PDF p.319-320)

23-3. 부분집합의 합만 캐시 키로 사용하면 어떤 정보가 구분되지 않는가? (K 8.5.7 메모이제이션, PDF p.338)

23-4. 두 부분 문자열의 마지막 문자가 같을 때 LCS 길이는 어떻게 계산하는가? (K 8.6.3 최적 부분 구조 찾기, PDF p.364)

정답과 원문 근거

1. 중복되는 부분 문제와 **최적 부분 구조**이다. [U23-045] (K 8.2 동적 계획법이란?, PDF p.318)

2. 메모이제이션은 하향식이며, **타뷸레이션**은 **기저 조건** 해답에서 시작하는 상향식이다. [U23-046] (K 8.3~8.4, PDF p.319-320)

3. 같은 합에 도달한 서로 다른 부분집합 구성과 **상태**가 구분되지 않아 잘못된 캐시 결과를 쓸 수 있다. [U23-047] (K 8.5.7 메모이제이션, PDF p.338)

4. 각각 마지막 문자를 제외한 문자열의 LCS 길이에 1을 더한다. [U23-048] (K 8.6.3 최적 부분 구조 찾기, PDF p.364)

24 동적 계획법 II

24-1. 부분집합 합의 O(N × M)에서 M은 무엇이며 원문은 이 해법을 어떻게 분류하는가? (K 9.3 부분집합의 합 문제 다시 보기, PDF p.380-381)

24-2. 무한 개수 배낭에서 1차원 DP를 사용할 수 있는 이유는 무엇인가? (K 9.4.4 상태 공간 축소, PDF p.394)

24-3. 플로이드-워셜의 세 중첩 반복문 순서는 무엇인가? (K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)

24-4. 플로이드-워셜은 음수 사이클을 어떤 테이블 원소로 판별하는가? (K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)

정답과 원문 근거

1. M은 부분집합 합의 목표치이고, 원문은 해법을 **의사 다항 시간**으로 분류한다. [U24-043] (K 9.3 부분집합의 합 문제 다시 보기, PDF p.380-381)

2. 상태 전환 조건이 물건 인덱스가 아니라 무게에만 의존하기 때문이다. [U24-044] (K 9.4.4 상태 공간 축소, PDF p.394)

3. 중간 정점 mid, 시작 정점 start, 목표 정점 end 순서이다. [U24-045] (K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)

4. distance[i][i]가 0보다 작은지 확인한다. [U24-046] (K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)

25 실습 문제 풀이

25-1. 공유 프린터 실습에서 요청된 순서대로 인쇄하기 위해 사용한 컨테이너는 무엇인가? (K A.1.3 사무실 공유 프린터의 인쇄 대기 목록 시뮬레이션, PDF p.439)

25-2. 워드카운트 맵 함수에서 key와 value는 각각 무엇인가? (K A.4.3 맵리듀스를 이용하여 워드카운트 구현하기, PDF p.462)

25-3. 욕심쟁이 로봇 풀이에서 전력 값의 부호를 바꾸는 이유는 무엇인가? (K A.7.1 욕심쟁이 로봇, PDF p.478)

25-4. 최대 이익의 상향식 상태를 정의하는 세 요소는 무엇인가? (K A.9.1 최대 이익, PDF p.519-520)

정답과 원문 근거

1. Printer **클래스**는 요청된 순서대로 프린트하기 위해 std::queue를 사용하고, 최대 인쇄 요청 개수를 템플릿 인자로 받는다. [U25-006] (K A.1.3 사무실 공유 프린터의 인쇄 대기 목록 시뮬레이션, PDF p.439)

2. 워드카운트의 맵 함수는 각 입력 파일에서 단어를 추출하고, key를 단어로, value를 1로 설정한 쌍을 생성한다. [U25-018] (K A.4.3 맵리듀스를 이용하여 워드카운트 구현하기, PDF p.462)

3. 욕심쟁이 로봇에서 에너지를 최대로 유지하는 것은 이동 시 에너지 손실을 최소로 하는 것과 같으므로, 실제 구현에서는 전력 값의 부호를 바꾼다. [U25-024] (K A.7.1 욕심쟁이 로봇, PDF p.478)

4. 최대 이익의 상향식 **상태**는 현재 인덱스, 상품 구매 비용, 상품 개수로 정의되며, 구현은 0-1 **배낭 문제** 풀이에서 차원이 하나 추가된 형태이다. [U25-038] (K A.9.1 최대 이익, PDF p.519-520)